

Appunti del corso di Sistemi Concorrenti e Paralleli M

Martino Cito

AA 2025/26

DISCLAIMER:

Questi appunti potrebbero contenere errori e/o imprecisioni, di conseguenza non mi assumo alcuna responsabilità sull'utilizzo di questo pdf. Se comunque vuoi ringraziarmi per il lavoro svolto, puoi offrirmi un caffè qui:

<https://paypal.me/MartinoCito02>

Contents

1	Architetture dei Sistemi di Elaborazione	6
1.1	Il modello di Von Neumann	6
1.2	Memoria Cache	7
1.2.1	Cache Hit e Cache Miss	7
1.3	Instruction Level Parallelism (ILP)	7
1.3.1	Limitazioni dell'ILP	8
1.4	Parallelismo Low Level: Hardware Multithreading	8
1.5	Architetture HPC	9
1.6	Shared Memory Multiprocessors	12
1.7	Distributed Memory	13
2	Programmazione Concorrente	15
2.1	Architetture e Linguaggi per la programmazione concorrente . .	18
2.2	Costrutti linguistici per la specifica della concorrenza	20
3	Modello a memoria comune	22
3.1	Gestore delle Risorse	22
3.2	Mutua Esclusione	25
3.3	Strumenti linguistici per la programmazione di interazioni	26
3.4	problema del rendez-vous	32
4	Modello a scambio di messaggi	43
4.1	Canale di comunicazione	43
4.2	Tipi di Canali	44
4.2.1	Comunicazione asincrona	44
4.2.2	Comunicazione Sincrona: rendez-vous semplice	45
4.3	Costrutti linguistici e primitive per esprimere la comunicazione .	46
4.3.1	Definizione del canale (port)	46
4.3.2	Primitive di comunicazione sincrone	46
4.3.3	Receive Bloccante e Modello Client-Server	47
4.3.4	Comandi con Guardia	48
4.3.5	Primitive di comunicazione asincrone	49
4.3.6	Uso delle primitive di comunicazione sincrone	59
5	Comunicazione con sincronizzazione estesa	62
5.0.1	Chiamata di procedura remota	62
5.0.2	Rendez-vous esteso	63
5.1	Vettore di operazioni di servizio	65
6	Algoritmi di Sincronizzazione Distribuiti	66
6.1	Algoritmi per la gestione del tempo	67
6.1.1	Orologi Fisici (Berkeley)	67
6.1.2	Orologi Logici (Lamport, 1979)	68
6.1.3	Algoritmo di Lamport	68

6.2	Mutua Esclusione Distribuita	69
6.2.1	Mutua esclusione: soluzione centralizzata	70
6.2.2	Mutua esclusione: Algoritmo Ricart-Agrawala	71
6.2.3	Mutua Esclusione: Algoritmo Token ring	73
6.2.4	Confronto tra i 3 algoritmi di mutua esclusione	74
6.2.5	Algoritmi di Elezione	74
7	Virtualizzazione	77
7.1	Virtualizzazione di un Sistema di Elaborazione	77
7.2	Tecniche del VMM	78
7.2.1	Emulazione	78
7.2.2	Macchine virtuali	79
7.3	Realizzaione del VMM	80
7.3.1	Ring di protezione	81
7.3.2	Ring Deprivileging	81
7.3.3	Fast Binary Translation	82
7.3.4	Paravirtualizzazione	82
7.4	Architetture virtualizzabili	83
7.5	Protezione nell'architettura x86	83
7.5.1	Problemi: Ring Aliasing	84
7.6	Stati di una VM	85
7.6.1	Migrazione di VM	86
7.7	XEN	87
7.7.1	Caratteristiche Xen	88
7.7.2	Virtualizzazione della CPU	90
7.7.3	Virtualizzazione dell'I/O	91
7.7.4	Gestione interruzioni e eccezioni	91
7.7.5	gestione del page fault	92
7.7.6	Live migration	92
8	Architettura della macchina corrente	93
8.1	Nucleo di un sistema a Processi	93
8.2	Caratteristiche fondamentali del nucleo	94
8.3	Realizzazione del Nucleo: Architettura monoprocesso	95
8.3.1	Strutture Dati del Nucleo	95
8.3.2	Coda dei processi pronti	96
8.4	Funzioni del Nucleo	97
8.4.1	Funzione di livello inferiore: Cambio di Contesto	98
8.5	Realizzazione Semaforo (caso monoprocesso)	99
8.6	Architettura Multiprocesso: realizzazione del nucleo, Modello a memoria comune	101
8.6.1	Modello SMP: Simmetric Multi Processing	102
8.6.2	Modello a nuclei distinti	103
8.6.3	Organizzazione SMP vs nuclei distinti	103
8.6.4	Realizzazione dei semafori nel modello a nuclei distinti	105
8.6.5	Semafori condivisi	105

8.6.6	Comunicazione tra nuclei	107
8.7	Architettura Monoprocessore: realizzazione del nucleo, Modello a scambio di messaggi	108
8.7.1	Realizzazione delle primitive asincrone	108
9	Calcolo parallelo	115
9.1	Sistemi MIMD	115
9.2	Sistemi operativi per HPC	116
9.2.1	Light Weight Kernel	116
9.2.2	Full Weight Kernel	117
9.2.3	Multi-kernel	117
9.3	Sviluppo di applicazioni parallele in sistemi HPC	118
9.4	Sviluppo di Software Parallelo	118
9.4.1	Programmi embarrassingly parallel	119
9.5	Prestazioni del software parallelo	120
9.5.1	Speedup	120
9.5.2	Efficienza	121
9.6	Legge di Amdahl (1967)	122
9.6.1	Scalabilità strong e weak	125
9.6.2	Legge di Gustafson(1988)	125
10	Sviluppo di programmi paralleli	127
10.1	MPI: Message Passing Interface	127
10.1.1	Caratteristiche di MPI	127
10.1.2	MPI_Init e MPI_Finalize	128
10.2	Communicator	128
10.2.1	Size e Rank	128
10.3	Funzioni di comunicazione	128
10.3.1	MPI_Send	128
10.3.2	MPI_Recv	129
10.4	Primitive collettive	129
11	Programmazione parallela su sistemi shared memory	130
11.1	Sistemi HPC a memoria condivisa	130
11.1.1	OpenMP: Open specifications for Multi-Processing	130
11.2	Clausole principali	131
11.2.1	Scope delle variabili	131
11.2.2	Clausola reduction	132
11.2.3	Clausola if	132
11.2.4	Direttiva master	133
11.2.5	Direttiva single	133
11.2.6	Parallelizzazione di cicli	133
11.2.7	Clausola schedule	134
11.3	Sincronizzazione tra thread in OMP	135
11.3.1	Direttiva barrier	135
11.3.2	Sincronizzazione lock	135

11.4	Confronto OpenMP vs pthreads	136
11.5	Confronto MPI vs OpenMP	137
11.6	Hybridization	137

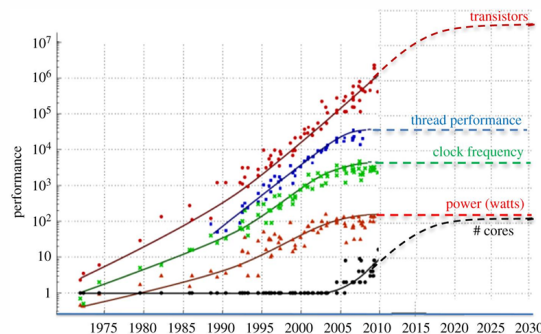
1 Architetture dei Sistemi di Elaborazione

Fino ai primi anni 2000 l'evoluzione dell'hardware è stata guidata dalla Legge di Moore, secondo cui la densità dei transistor nei chip raddoppia approssimativamente ogni 18 mesi. L'aumento della densità permetteva processori sempre più complessi, con miglioramenti costanti delle performance.

A partire dagli anni 2000 ci si è avvicinati ai limiti fisici della miniaturizzazione: gli effetti parassiti (es. surriscaldamento dovuto all'effetto Joule) hanno impedito ulteriori aumenti significativi della frequenza di clock. Da qui nasce la necessità di aumentare la capacità di calcolo senza aumentare la frequenza.

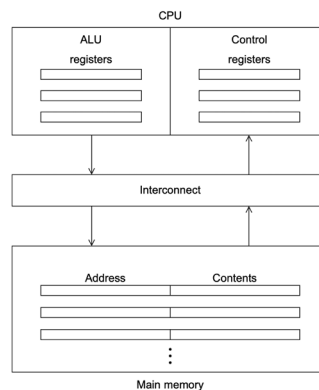
Con clock invariato, le prestazioni vengono migliorate tramite forme di parallelismo hardware:

- presenza di più unità di elaborazione su singolo chip (multicore, GPU, FPGA)
- uso di più processori su più chip (multiprocessori, cluster)



1.1 Il modello di Von Neumann

Il modello di Von Neumann descrive un sistema sequenziale tradizionale con: una sola CPU, memoria centrale contenente dati e istruzioni, un'interconnessione unica (bus)



La separazione tra CPU e memoria crea il cosiddetto *Von Neumann bottleneck*: la CPU è molto più veloce della memoria, e l'accesso a dati/istruzioni tramite il bus diventa un limite.

Poiché CPU e memoria sono collegate tramite bus, la velocità di fetching dipende dalla velocità di trasmissione del bus: anche con una CPU molto potente, la velocità di esecuzione è limitata dalla banda di trasmissione del bus.

Per mitigare il bottleneck si introducono:

- **memorie cache**
- **parallelismo a basso livello**
 - Instruction Level Parallelism
 - hardware multithreading

1.2 Memoria Cache

La cache è una memoria associativa:

- **ad accesso veloce:** risiede sul chip del processore e si colloca a un livello intermedio, tra i registri e la memoria centrale
- **di capacità limitata:** la cache non può contenere tutte le istruzioni e i dati necessari al programma in esecuzione

Viene gestita con criteri basati sul principio di località:

- **Località temporale:** se un dato è stato usato recentemente, è probabile che venga riusato nel prossimo futuro.
- **Località spaziale:** se un dato è stato acceduto, probabilmente serviranno anche i dati vicini in memoria.

Quindi la cache memorizza blocchi di dati/istruzioni presi dalla RAM che la CPU ha appena usato o che è probabile userà a breve.

1.2.1 Cache Hit e Cache Miss

Quando la CPU richiede un'informazione in memoria:

- **cache hit:** l'informazione è in cache → accesso veloce
- **cache miss:** l'informazione è in RAM → accesso lento

L'obiettivo della cache è mantenere un elevato hit-rate, ovvero contenere il numero di cache miss, per ridurre gli effetti del bottleneck.

1.3 Instruction Level Parallelism (ILP)

Ogni istruzione viene eseguita in più fasi (ex: somma floating point $C = A + B$).

1. **fetch** operandi A e B

2. **confronto esponenti** ed eventuale **shift**
3. **somma**
4. **normalizzazione** del risultato C
5. **memorizzazione** di C

Ognuna delle fasi può essere affidata a un'unità funzionale HW indipendente che opera in parallelo alle altre. Tecniche principali:

- **pipelining**: tutte le unità funzionali sono collegate tra loro in una pipeline; fasi diverse di istruzioni diverse possono essere eseguite in parallelo;
- **multiple issue**: più istanze per ogni unità funzionale

Table 2.3 Pipelined Addition. Numbers in the Table Are Subscripts of Operands/Results

Time	Fetch	Compare	Shift	Add	Normalize	Round	Store
0	0						
1	1	0					
2	2	1	0				
3	3	2	1	0			
4	4	3	2	1	0		
5	5	4	3	2	1	0	
6	6	5	4	3	2	1	0
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
999	999	998	997	996	995	994	993
1000		999	998	997	996	995	994
1001			999	998	997	996	995
1002				999	998	997	996
1003					999	998	997
1004						999	998
1005							999

unità funzionali collegate in pipeline

A regime (es. T=6), ogni unità funzionale processa un'istruzione diversa.

1.3.1 Limitazioni dell'ILP

L'efficacia di tecniche ILP può ridursi nel caso di dipendenze tra istruzioni successive: se in un programma è presente una serie di istruzioni tra loro dipendenti, il parallelismo a livello di istruzione è limitato.

Esempio: si consideri la sequenza di istruzioni

$$a = b + c, \quad d = e + f, \quad g = a + d$$

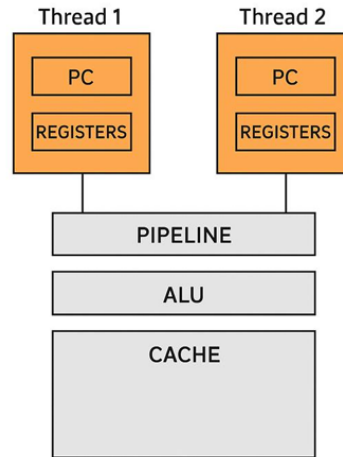
La terza dipende dalle prime due → non può essere eseguita in parallelo. Infatti: la terza operazione aritmetica (a+d) deve attendere che la pipeline completi l'ultima fase (memorizzazione) della prima e della seconda istruzione.

1.4 Parallelismo Low Level: Hardware Multithreading

In alternativa/aggiunta a ILP, i processori moderni offrono **parallelismo a livello di thread** (thread level parallelism) mediante **HW multithreading**: permette a più thread di condividere la stessa CPU (core).

E' reso possibile:

- dalla **duplicazione** dei registri che mantengono lo stato di ogni thread (PC, altri registri..)
- da un meccanismo HW che implementa il context switch tra un thread ed un altro in modo molto efficiente: per ogni cambio di contesto si commuta la CPU verso il set di registri del thread schedulato (nessun salvataggio né ripristino verso/da memoria!).



La gestione dei thread è governata da uno scheduler hardware, che decide ad ogni ciclo a quale thread associare l'unità di elaborazione. L'obiettivo è contrastare gli effetti del Von Neumann bottleneck → limitare le attese dovute a cache miss. Due approcci:

- **Multithreading a grana fine (Fine-grained):** cambio thread ogni istruzione
 - **Vantaggio:** Attese dovute a cache miss vengono "nascoste" allocando la CPU ad altri thread.
 - **Svantaggio:** velocità di esecuzione dei thread ridotta.
- **Multithreading a grana grossa (Coarse-grained):** il context switch avviene quando il thread corrente è in una situazione di attesa (ad esempio: in caso di "cache miss", deve attendere il caricamento dell'informazione dalla memoria centrale).
 - **Vantaggio:** meno context switch, quindi velocità media di esecuzione dei thread più alta.
 - **Svantaggio:** il context switch è più costoso (necessità di vuotare la pipeline ILP). Throughput più basso.

1.5 Architetture HPC

ILP e Hardware multithreading hanno permesso un miglioramento delle prestazioni dei processori, tuttavia tali meccanismi sono trasparenti per gli sviluppatori dei programmi. → **modello Von Neumann Esteso**

Nei sistemi HPC, il parallelismo è esplicito e visibile al programmatore, che deve progettare il software in modo da sfruttare al meglio tutte le risorse computazionali a disposizione. → architetture **non Von Neumann**

Classificazione delle Architetture: Tassonomia di Flynn Basata su:

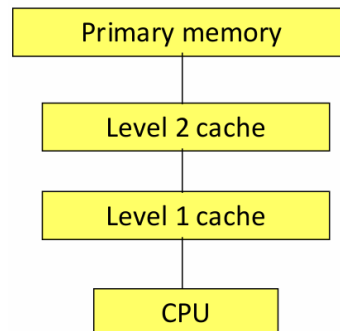
- Flusso di istruzioni (**Instruction Stream**): quanti flussi di istruzioni il sistema è in grado di eseguire contemporaneamente
- Flusso di dati (**Data Stream**): quanti flussi diversi di dati possono essere elaborati contemporaneamente.

		Instruction Streams	
		one	many
Data Streams	one	SISD traditional von Neumann single CPU computer	MISD May be pipelined Computers non prevede applicazioni significative
	many	SIMD Vector processors fine grained data Parallel computers vediamo in sist. di elab. accelerata	MIMD Multi computers Multiprocessors questi vedremo in questo corso

- **SISD**: singola istruzione, singolo dato (modello Von Neumann), in cui non c'è parallelismo né di istruzioni, né di dati: un solo processore esegue un unico flusso di istruzioni su un singolo flusso di dati.
- **SIMD**: stessa istruzione su dati diversi (GPU, processori vettoriali)
- **MISD**: molte istruzioni in parallelo su un singolo flusso di dati (fault tolerance → elaborazioni ridondanti)
- **MIMD**: più istruzioni su più flussi di dati (**cluster, multiprocessori, multicomputers, HPC**)

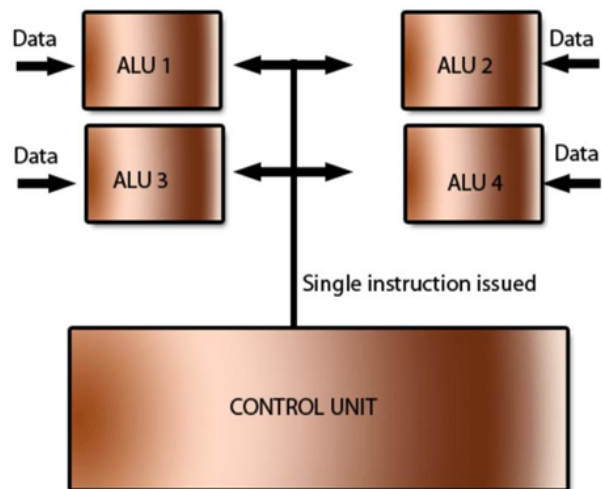
SISD

Classico sistema monoprocesso



SIMD

Array processor con molte unità di elaborazione gestite da un'unica unità di controllo. Esecuzione sincrona della stessa istruzione su dati diversi. Vengono sfruttati per il calcolo ad alte prestazioni e sono particolarmente adatti per applicazioni di calcolo scientifico (dati in forma vettoriale/matriciale) **Più processori ed 1 sola control unit.**



An array processor - a Single Instruction Multiple Data computer

Sistemi MIMD

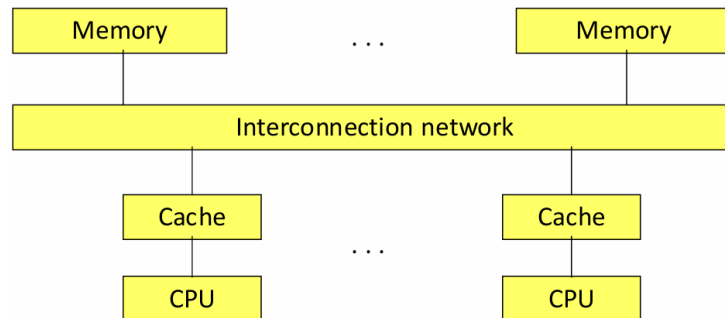
Asincronicità delle attività nei diversi nodi: Processori asincroni, ciascuno con la propria control unit. Due categorie:

- **SHARED MEMORY** (UMA, NUMA) come smartphone e pc
- **DISTRIBUTED MEMORY** (cluster, supercomputer HPC) sistemi composti da insiemi di server

Necessario supporto a livello architetturale per consentire l'interazione:

- reti di interconnessione; possibilmente: bassa latenza e banda elevata.
- Memoria condivisa & cache → problema cache coherence.

1.6 Shared Memory Multiprocessors



Fanno parte di questa categoria i sistemi multicore e multiprocessore.

Sistemi Multiprocessore

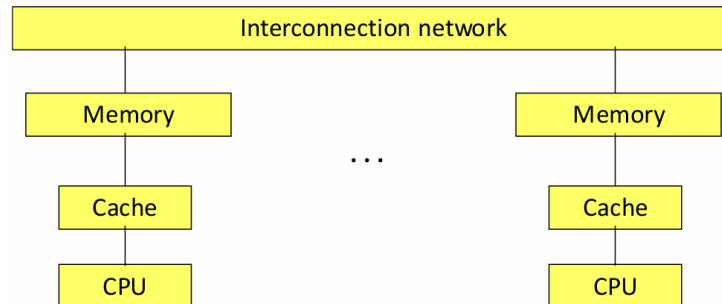
UMA(Uniform Memory Access): sistemi a multiprocessore con un numero ridotto di processori (da 2 a 30 circa):

- la rete di interconnessione è realizzata da un memory bus o da crossbar switch.
- Si chiamano anche symmetric multiprocessors (SMP).
- il tempo di accesso uniforme da ogni processore ad ogni locazione di memoria.

NUMA (Non Uniform Memory Access): sistemi con un numero elevato di processori (decine o centinaia):

- la memoria è organizzata **gerarchicamente** (per evitare la congestione del bus).
- La rete di interconnessione è un insieme di switches e memorie strutturato ad albero. Ogni processore ha memorie che sono più vicine (accesso più veloce) ed altre più lontane (accesso più lento).
- il tempo di accesso dipende dalla distanza tra processore e memoria.

1.7 Distributed Memory



Due modelli:

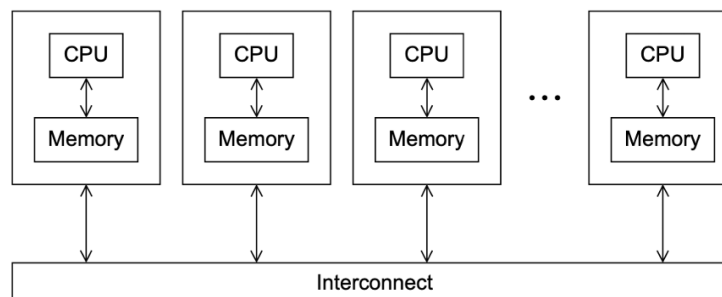
Multicomputer: i processori e la rete sono fisicamente vicini (nella stessa struttura): tightly coupled machine.

La rete di interconnessione offre un cammino di comunicazione tra i processori ad alta velocità e larghezza di banda (es. Cluster of Computers COW, HPC systems).

Network systems: i nodi sono collegati da una rete locale (es.Ethernet) o da una rete geografica (Internet): loosely coupled systems.

I nodi di un distributed memory system possono essere o singoli processori o shared memory multiprocessor (sistemi ibridi o eterogenei).

HPC Distributed Memory

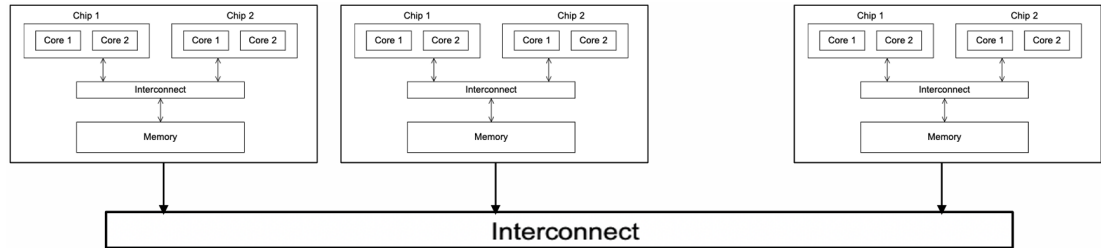


La rete deve garantire:

- bassa latenza
- elevata banda

No Bus, ma reti ad alte prestazioni (topologie mesh, toroidali, ipercubi, ecc).

Modelli Ibridi HPC



La maggior parte dei sistemi HPC oggi combina il modello distributed e shared memory. Esempio: cluster di nodi, ciascuno con architettura multicore/multi-processore.

2 Programmazione Concorrente

La programmazione concorrente è l'insieme delle tecniche, metodologie e strumenti per il supporto all'esecuzione di sistemi software composti da insiemei di attività svolte simultaneamente.

Tipi di Applicazioni (vedi capitolo 1)

- **Multithreaded:** applicazioni strutturate come un insieme di processi con lo scopo di aumentare l'efficienza, ridurre la complessità e la programmazione. I processi possono condividere variabili, generalmente esistono più processi che processori; e i processi vengono schedati ed eseguiti indipendentemente.
- **Sistemi multitasking/Sistemi Distribuiti:** le componenti dell'applicazione (task) vengono eseguite sui nodi, generalmente virtuali, collegati tramite opportuni mezzi di interconnessione. E i processi comunicano scambiandosi messaggi.
- ***Applicazioni Parallele:** applicazioni che sfruttano al massimo il parallelismo disponibile a livello HW. Infatti vengono eseguite su sistemi specificatamente paralleli come HPC o processori vettoriali, facendo largo uso di algoritmi paralleli. A seconda del modello architetturale, l'esecuzione è portata avanti da istruzioni/thread/processi paralleli che interagiscono utilizzando specifiche librerie.

Processi non sequenziali e tipi di interazione

- **Algoritmo:** procedimento logiche che deve essere eseguito per risolvere in determinato problema.
- **Programma:** descrizione di un algoritmo mediante un opportuno formalismo (linguaggio di programmazione), che rende possibile l'esecuzione dell'algoritmo da parte di un particolare elaboratore.
- **Processo:** insieme ordinato degli eventi cui dà luogo un elaboratore quando opera sotto il controllo di un programma.
- **elaboratore:** entità **astratta** realizzata in HW e parzialmente in SW in grado di eseguire programmi.
- **Evento:** esecuzione di un'operazione tra quelle appartenenti all'insieme che l'elaboratore sa riconoscere ed eseguire, ogni evento determina una ****transizione di stato**** dell' elaboratore.

Quindi un programma non descrive un processo, ma un insieme di processi, ognuno dei quali è relativo all'esecuzione del programma da parte dell'elaboratore per un determinato insieme di dati in ingresso. Pertanto più processi possono essere associati allo stesso programma (**istanze**); ciascuno rappresenta

l'esecuzione dello stesso codice con dati di ingresso (possibilmente) diversi. Esempio: Il sistema operativo può generare più processi concorrenti che eseguono shell, ognuno per un diverso utente $\rightarrow$ ogni processo esegue diverse operazioni su dati diversi

Processo Sequenziale

Un processo sequenziale è descritto da una sequenza di stati attraverso i quali passa l'elaboratore durante l'esecuzione di un programma.

L'insieme degli eventi che descrive un processo sequenziale è dotato di una relazione di ordine totale, che esprime la precedenza tra gli eventi.

Relazione d'ordine totale

Definizione: Sia A un insieme e $\leq$ una relazione binaria su A .

La coppia $(A, \leq)$ è un **insieme ordinato totalmente** se $\leq$ soddisfa le seguenti proprietà:

1. **Riflessività:**

$$\forall a \in A, \quad a \leq a$$

2. **Antisimmetria:**

$$\forall a, b \in A, \quad (a \leq b \wedge b \leq a) \Rightarrow a = b$$

3. **Transitività:**

$$\forall a, b, c \in A, \quad (a \leq b \wedge b \leq c) \Rightarrow a \leq c$$

4. **Confrontabilità (totalità):**

$$\forall a, b \in A, \quad a \leq b \vee b \leq a$$

L'insieme degli eventi E che descrive un processo sequenziale è dotato di una relazione di ordine totale, che esprime la precedenza tra gli eventi.

Pertanto, per qualunque coppia di eventi $(e1, e2)$ dell'insieme E , possiamo sempre stabilire se $e1$ precede $e2$, oppure se $e2$ precede $e1$.

Un processo può essere rappresentato tramite un grafo orientato detto **grafo di precedenza del processo**, costituito da nodi ed archi orientati:

- i nodi del grafo rappresentano i singoli eventi del processo;
- gli archi orientati identificano le precedenze temporali tra tali eventi;

Ogni nodo rappresenta quindi un evento corrispondente all'esecuzione di un'operazione tra quelle appartenenti all'insieme che l'elaboratore sa riconoscere ed eseguire.

Processi non sequenziali

L'insieme degli eventi che lo descrive è ordinato secondo una relazione d'ordine parziale. Quindi per descrivere un processo non sequenziale utilizziamo un **grado di precedenza ad ordinamento parziale**.

Per poter eseguire un processo non sequenziale, ovviamente, richiede un elaboratore non sequenziale.

Relazione d'ordine parziale

Definizione: Sia A un insieme e $\leq$ una relazione binaria su A .

$(A, \leq)$ è un insieme ordinato parzialmente se $\leq$ soddisfa le seguenti proprietà:

1. **Riflessività:**

$$\forall a \in A, \quad a \leq a$$

2. **Antisimmetria:**

$$\forall a, b \in A, \quad (a \leq b \wedge b \leq a) \Rightarrow a = b$$

3. **Transitività:**

$$\forall a, b, c \in A, \quad (a \leq b \wedge b \leq c) \Rightarrow a \leq c$$

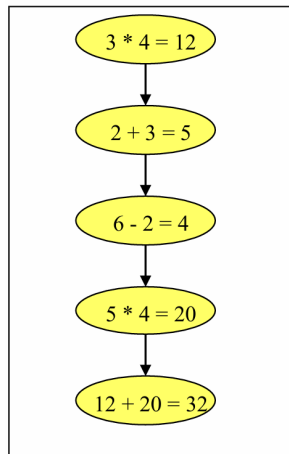
Ordine Parziale vs. Ordine Totale

- **Ordine totale:** ogni coppia di elementi è confrontabile.
- **Ordine parziale:** esistono coppie di elementi che non sono legati dalla relazione di precedenza.

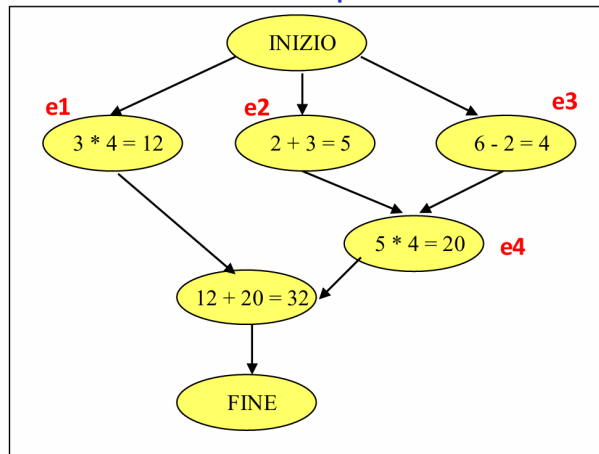
Esempio: valutazione dell'espressione

$$(3 * 4) + (2 + 3) * (6 - 2)$$

Grafo di precedenza ad ordinamento totale:



Grafo di precedenza ad ordinamento parziale:



Processi Interagenti: le interazioni tra processi di lettura, elaborazione e scrittura sono dettati da uno scambio di informazioni.

Quando due processi arrivano ad un punto di interazione (ad esempio, uno scambio di informazioni) devono sincronizzarsi, cioè ordinare i loro eventi come specificato dal grafo di precedenza.

Tipi di Decomposizione:

La decomposizione di un grafo di precedenza ad ordinamento parziale può essere effettuato in modi differenti, orizzontalmente (quindi per archi), oppure verticalmente (per processi), generalmente si sceglie quello che permette di avere il minimo numero di interazioni possibili.

Interazione tra processi:

Comprende tutte le interazioni prevedibili e desiderate, insite quindi nella logica degli algoritmi. Generalmente prevede scambio di informazioni, o dati (quindi messaggi, e quindi comunicazione), oppure segnali temporali (senza trasferimento di dati).

Competizione:

La macchina concorrente su cui i processi sono eseguiti mette a disposizione un numero limitato di risorse condivise tra i processi. L'obiettivo della competizione è il coordinamento dei processi nell'accesso alle risorse condivise. Quindi, nel caso determinate risorse non possono essere accessibili da più processi contemporaneamente, è necessario quindi prevedere dei meccanismi che facciano rispettare le regole di accesso delle risorse e quindi regolino la competizione (in breve è una forma di sincronizzazione in più).

Un esempio è la Mutua Esclusione, nel quale, ad esempio, due processi devono utilizzare in determinati istanti una risorsa condivisa, ma questa risorsa può essere acceduta solamente da un processo alla volta. Quindi gli accessi alla risorsa devono essere fatti in modo mutuamente esclusivo, ovvero un processo alla volta.

Sezione Critica:

Sequenza di istruzioni con le quali un processo accede a un oggetto condiviso con altri oggetti. Ad un oggetto può essere associata ad una sola sezione critica (usata da tutti i processi) o più sezioni critiche (classe di sezioni critiche). **Sezioni critiche appartenenti alla stessa classe devono escludersi mutuamente nel tempo.**

L'interazione tra processi può essere di più tipi:

- Cooperazione: sincronizzazione diretta o esplicita;
- Competizione: sincronizzazione indiretta o implicita;
- Interferenza: interazione provocata da errori di programmazione (dead-lock), interazione non prevista e non desiderata.

2.1 Architetture e Linguaggi per la programmazione concorrente

Proprietà che un linguaggio per la programmazione concorrente deve avere:

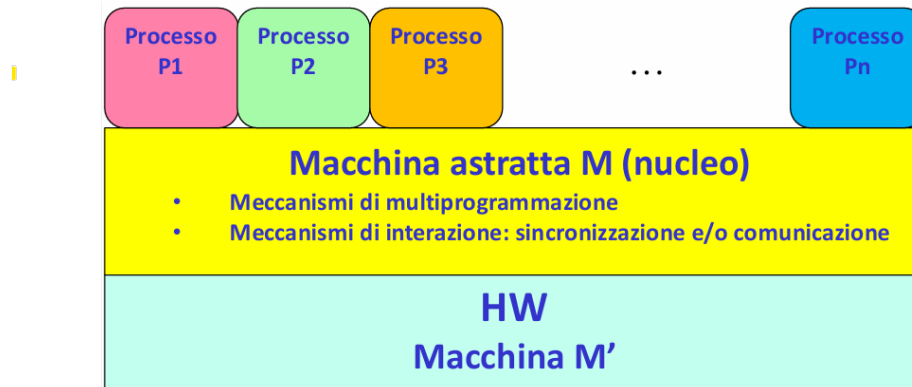
- contenere appositi costrutti con i quali è possibile dichiarare moduli di programma destinati ad essere eseguiti come processi sequenziali distinti;
- non tutti i processi vengono eseguiti contemporaneamente. Alcuni processi vengono svolti solamente se si verificano determinate condizioni;
- occorre che siano presenti strumenti linguistici per specificare le interazioni che dinamicamente potranno verificarsi tra i vari processi.

Una macchina concorrente è costituita da HW e componenti SW che permettono la multiprogrammazione. Ovviamente, in una Macchina astratta M, oltre ai processi di multiprogrammazione e sincronizzazione è presente anche il meccanismo di ****protezione****, e quindi il meccanismo di controllo degli accessi alle risorse, in grado di:

- rilevare eventuali interferenze tra i processi;
- può essere realizzato in HW o in SW nel supporto a tempo di esecuzione;
- capabilities e liste di controllo degli accessi.

Il kernel offre supporto a tempo di esecuzione di un linguaggio concorrente. Nel kernel sono sempre presenti due funzionalità base:

- meccanismo di multiprogrammazione (1 cpu virtuale per ogni processo);
- meccanismo di sincronizzazione e comunicazione tra processi concorrenti.



Architettura di M:

Due diverse organizzazioni logiche:

- gli elaboratori di M sono collegati ad un'unica memoria principale (sistemi multiprocessore) → **modello a memoria comune**;
- gli elaboratori di M sono collegati da una sottorete di comunicazione, senza memoria comune. → **modello a scambio di messaggi**

Queste due organizzazioni logiche di M definiscono due modelli di interazione tra i processi:

- Modello a memoria comune, nel quale l'interazione tra i processi avviene tramite oggetti contenuti nella memoria comune (modello ad ambiente globale / **multithreading**);
- Modello a scambio di messaggi, nel quale la comunicazione e la sincronizzazione tra i processi si basa sullo scambio di messaggi sulla rete che collega i vari elaboratori (modello **multitasking**).

2.2 Costrutti linguistici per la specifica della concorrenza

Fork/Join

L'esecuzione di una **fork** (non la system call unix) coincide con la creazione e l'attivazione di un processo che inizia la propria esecuzione in parallelo con quella del processo chiamante.

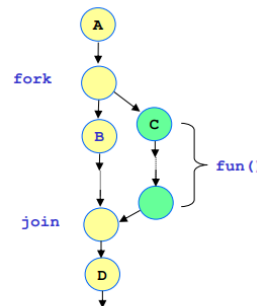
La **join** è una primitiva di sincronizzazione che permette di determinare quando un processo, creato tramite la fork, ha terminato il suo compito, sincronizzandosi con tale evento.

```

/* processo p: */
...
A: .....;
  p = fork fun;
B: .....;
  join p;
D: .....;
...

/* codice nuovo
processo:*/
void fun()
{
  C: .....;
  ...
}

```



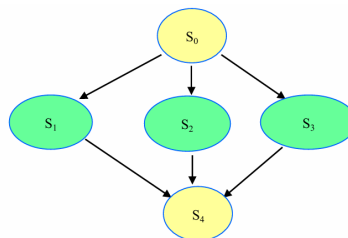
Cobegin-Coend:

Alternativa al modello fork/join, prende ispirazione al modello di programmazione strutturata.

```

S0;
cobegin
  S1;
  S2;
  S3;
coend
S4;

```



Le istruzioni $S_1, S_2, \dots, S_n$ sono eseguite in parallelo. Ogni S_i può contenere altre istruzioni `cobegin..coend` al suo interno.

Proprietà dei programmi

Una delle attività più importanti per chi sviluppa programmi è la verifica di correttezza dei programmi realizzati.

Traccia dell'esecuzione: sequenza degli stati attraversati dal sistema di elaborazione durante l'esecuzione del programma.

Stato: insieme dei valori delle variabili definite nel programma più variabili implicite (es valore del Program Counter).

Proprietà safety: proprietà che garantisce che durante l'esecuzione di P non si entrerà mai in uno stato "errato".

Proprietà liveness: proprietà che garantisce che durante l'esecuzione di P , prima o poi si entrerà in uno stato "corretto".

Safety and Liveness:

Programmi sequenziali:

Le proprietà fondamentali che ogni programma sequenziale deve avere sono:

1. **correttezza del risultato finale:** per ogni esecuzione il risultato ottenuto è giusto. (safety)
2. **terminazione:** prima o poi l'esecuzione termina. (liveness)

Programmi concorrenti:

1. **correttezza del risultato finale e terminazione**
2. **mutua esclusione nell'accesso a risorse condivise:** per ogni esecuzione non accadrà mai che più di un processo acceda contemporaneamente alla stessa risorsa (safety)
3. **assenza di deadlock:** per ogni esecuzione non si verificheranno mai situazioni di blocco critico (safety)
4. **assenza di starvation:** prima o poi ogni processo potrà accedere alle risorse richieste (liveness)

3 Modello a memoria comune

Esistono 2 principali modelli di interazione tra i processi:

- Modello a **memoria comune** (ambiente globale, shared memory)
- Modello a **scambio di messaggi** (ambiente locale, distributed memory)

Il modello a memoria comune rappresenta la più semplice astrazione del funzionamento di un sistema in multiprogrammazione costituito da uno o più processi che hanno accesso ad una memoria comune.

Ogni applicazione viene strutturata come un insieme di componenti, suddiviso in due sottoinsieme disgiunti:

- **Processi** (componenti attivi) **Risorse** (componenti passivi)

Le Risorse rappresentano un qualunque oggetto fisico o logico, di cui un processo necessita per portare a termine il suo compito.

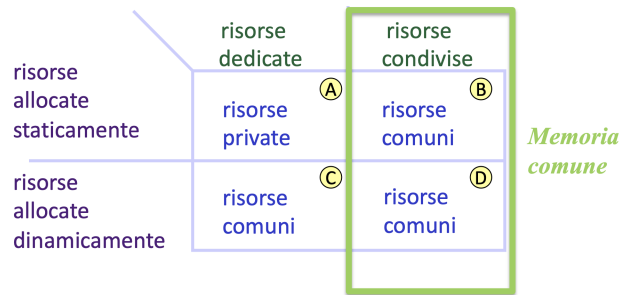
Le risorse vengono raggruppate in classi, dove una classe rappresenta l'insieme di tutte e sole le operazioni che un processo può eseguire per operare su risorse di quella classe. Ogni risorsa si identifica con una struttura dati che la rappresenta, allocata nella memoria comune. (Vale anche per risorse fisiche: descritt del disp) Ovviamente ci deve essere la necessità di specificare quali processi ed in quali istanti possono accedere alla risorsa. Quindi il **meccanismo di controllo degli accessi** si occupa di controllare che gli accessi dei processi alle risorse avvengano correttamente.

3.1 Gestore delle Risorse

Per ogni risorsa **R**, il suo gestore definisce, in ogni istante t , **l'insieme $SR(t)$ dei processi che**, in tale istante, **hanno il diritto di operare su R**.

Classificazione delle risorse:

- Risorsa **R dedicata**: se $SR(t)$ ha una cardinalità sempre ≤ 1
- Risorsa **R condivisa**: in caso contrario
- Risorsa **R allocata staticamente**: se $SR(t)$ è una costante, quindi se $SR(t) = SR(t_0) \forall t$
- Risorsa **R allocata dinamicamente**: se $SR(t)$ è funzione del tempo



Per ogni risorsa **allocata staticamente**, l'insieme $SR(t)$ è definito prima che il programma inizi la propria esecuzione; il gestore della risorsa è il programmatore che, in base alle regole del linguaggio, stabilisce quale processo può vedere e quindi operare su R.

Per ogni risorsa **allocata dinamicamente**, il relativo gestore G_R definisce l'insieme $SR(t)$ in fase di esecuzione e quindi deve essere un componente della stessa applicazione, nel quale l'allocazione viene decisa a run-time in base a politiche date.

Quindi i principali compiti del Gestore delle risorse sono:

- mantenere **aggiornato** l'insieme $SR(t)$ e cioè lo stato di allocazione della risorsa;
- fornire i **meccanismi** che un processo può utilizzare per acquisire il diritto di operare sulla risorsa, entrando a far parte dell'insieme $SR(t)$, e per rilasciare tale diritto quando non è più necessario;
- implementare la **strategia** di allocazione della risorsa e cioè definire quando, a chi e per quanto tempo allocare la risorsa.

Accesso a Risorsa

Consideriamo un processo P che deve operare, ad un certo istante, su una risorsa R di tipo T:

Se R è allocata **staticamente** a P (modalità A e B), il processo, se appartiene a SR, possiede diritto di operare su R in qualunque istante.

```
1 R.op(...); //esecuzione dell'operazione op_i su R
```

Se R è allocata **dinamicamente** a P (modalità C e D), è necessario prevedere un gestore GR, che implementa le funzioni di Richiesta e Rilascio della risorsa; quindi il processo P deve seguire il seguente protocollo:

```
1 GR.Richiesta(...); // acquisizione della risorsa
2 R.op(...); // esecuzione dell'operazione op su R
3 GR.Rilascio(...); // rilascio della risorsa R
```

Se R è allocata come **risorsa condivisa**, (modalità B e D) è necessario assicurare che gli accessi avvengano in modo non divisibile: nel senso che le funzioni di accesso alla risorsa devono essere programmate come una **classe di sezioni critiche**, utilizzando meccanismi di sincronizzazione offerti dal linguaggio di programmazione e supportati dalla macchina concorrente.

Se R è allocata come **risorsa dedicata**, (modalità A e C), essendo P l'unico processo che accede alla risorsa, non è necessario prevedere alcuna forma di sincronizzazione.

Regione critica condizionale [Hoare, Brinch-hansen]

Formalismo che permette di esprimere la specifica di qualunque vincolo di sincronizzazione. Data una risorsa R condivisa:

```
1 region R << Sa; when(C) Sb; >>
```

- tra doppie parentesi angolari il **corpo** della **region** che rappresenta una operazione da eseguire su una risorsa condivisa R e quindi costituisce una sezione critica che deve essere eseguita in **mutua esclusione** con le altre operazioni definite su R
- il corpo della region è costituito da due istruzioni da eseguire in sequenza: l'istruzione **Sa** e successivamente l'istruzione **Sb**.
- In particolare, una volta terminata l'esecuzione di Sa viene valutata la condizione **C**:
 - se C è **vera** l'esecuzione continua con Sb
 - se C è **falsa** il processo che ha invocato l'operazione attende che la condizione C diventi vera. Non appena C sarà vera l'esecuzione della **region** potrà riprendere ed eseguire Sb.

Esistono però dei casi particolari di regioni critiche:

- **region R << S; >>**: specifica della sola mutua esclusione, senza ulteriori vincoli.
- **region R << when(C) >>**: specifica di un semplice vincolo di sincronizzazione, nel quale il processo deve attendere che C sia verificata prima di proseguire.
- **region R << when(C) S; >>**: specifica il caso in cui la condizione C di sincronizzazione caratterizza lo stato in cui la risorsa R deve trovarsi per poter eseguire l'operazione S (C quindi è una **precondizione** di S).

3.2 Mutua Esclusione

Il problema della mutua esclusione nasce quando più di un processo alla volta può e deve accedere a variabili comuni. Quindi è di fondamentale importanza che le operazioni con le quali i processi accedono alle variabili comuni non si sovrappongano nel tempo, tenendo presente che nessun vincolo è imposto sull'ordine con il quale le operazioni sulle variabili vengono eseguite.

Con **sezione critica** s'intende la sequenza di istruzioni con le quali un processo accede e modifica un insieme di variabili comuni. Ad un insieme di variabili comuni possono essere associate una sola sezione critica (usata da tutti i processi) oppure più sezioni critiche (classe di sezioni critiche).

La regola della mutua esclusione stabilisce che:

Sezioni critiche appartenenti alla stessa classe devono escludersi mutuamente nel tempo.

oppure

Ad ogni istante può essere "in esecuzione" al più una sezione critica di ogni classe.

Per specificare una sezione critica **S** che opera su una risorsa condivisa **R**:

1	<prologo>
2	S;
3	<epilogo>

Attraverso il **prologo** si ottiene l'autorizzazione ad eseguire la sezione critica, quindi **R** viene acquisita in modo esclusivo. Invece attraverso l'**epilogo** la risorsa **R** viene liberata.

Le principali soluzioni possibili alla mutua esclusione sono:

- **Algoritmiche:** (es. Algoritmi di Dekker, ecc.) la soluzione non necessita di meccanismi di sincronizzazione (es. semafori, lock, ecc.), ma sfrutta solo la possibilità di condivisione di variabili; l'attesa di un processo che trova la variabile condivisa già occupata viene modellata attraverso cicli di attesa attiva.
- **Hardware-based:** ad esempio disabilitazione delle interruzioni, lock/unlock. Quindi il supporto è fornito direttamente dall'architettura HW.
- **Strumenti software di sincronizzazione realizzati dal nucleo della macchina concorrente:** prologo ed epilogo sfruttano strumenti di sincronizzazione che consentono l'effettiva sospensione dei processi in attesa ed eseguire sezioni critiche.

3.3 Strumenti linguistici per la programmazione di interazioni

Il Semaforo

Il semaforo è uno strumento linguistico di basso livello che consente di risolvere qualunque problema di sincronizzazione nel modello a memoria comune. È realizzato dal nucleo della macchina concorrente. L'eventuale attesa nella esecuzione può essere realizzata utilizzando i meccanismi di gestione dei thread offerti dal nucleo. Inoltre viene utilizzato per realizzare strumenti di sincronizzazione di più alto livello ad esempio le *condition*.

Definizione: un semaforo è una **variabile intera non negativa**, alla quale è possibile accedere solo **tramite le due operazioni P e V**.

Definizione di un oggetto di tipo **semaphore**:

```
1 semaphore s = i; //dove i (i >= 0) e' il valore iniziale
```

Al tipo semaphore sono associati:

Insieme di valori = $\{X | X \in N\}$

Insieme delle operazioni = $\{P, V\}$

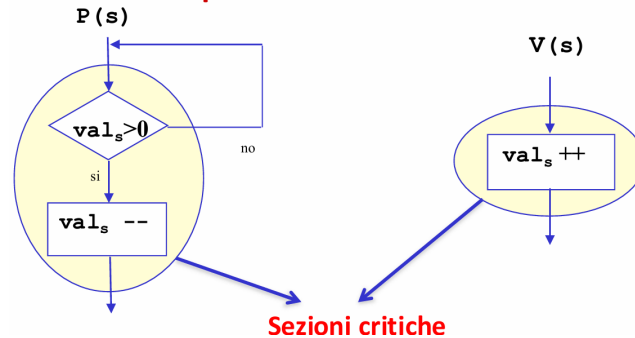
Operazioni sul semaforo

Un oggetto di tipo semaphore è condivisibile da due o più threads, che operano su di esso attraverso le operazioni **P** e **V**.

```
1 void P(semaphore s):  
2     region s << when(val_s > 0) val_s--; >>  
3  
4 void V(semaphore s):  
5     region s << val_s++; >>  
6  
7 // dove val_s rappresenta il valore del semaforo
```

Essendo **s** l'oggetto condiviso, le due operazioni P e V vengono definite come **sezioni critiche** da eseguire in mutua esclusione e in forma atomica.

Operazioni sul semaforo



Quindi il semaforo viene utilizzato come strumento di sincronizzazione tra processi concorrenti:

- **attesa:** $P(s)$, $val_s == 0$
- **risveglio:** $V(s)$, se vi è almeno un processo sospeso

Proprietà del Semaforo

Dato un semaforo S , siano:

- val_s : valore dell'intero non negativo associato al semaforo;
- I_s : valore intero ≥ 0 con cui il semaforo s viene inizializzato in PC;
- nv_s : numero di volte che l'operazione $V(s)$ è stata eseguita;
- np_s : numero di volte che l'operazione $P(s)$ è stata completata.

Ad ogni istante possiamo esprimere il valore del semaforo come:

$$val_s = I_s + nv_s - np_s$$

da cui ($val_s \geq 0$):

Relazione di Invarianza

$$np_s \leq I_s + nv_s$$

La relazione di invarianza è sempre soddisfatta, per ogni semaforo, qualunque sia il suo valore e comunque sia strutturato il programma concorrente che lo usa (**Safety** property).

Il semaforo è quindi uno strumento generale che consente la risoluzione di qualunque problema di sincronizzazione.

Esistono molti casi d'uso del meccanismo semaforico:

- semafori di mutua esclusione
- semafori condizione
- semafori evento
- semafori risorsa
- semafori binari composti
- semafori privati

Semaforo di mutua esclusione

Il semaforo di mutua esclusione viene inizializzato ad 1. Principalmente viene utilizzato per realizzare le sezioni critiche di una stessa classe, secondo il protocollo:

```

1  class tipo_risorsa {
2      <struttura dati di ogni istanza della classe>;
3
4      semaphore mutex = 1;
5
6      public void op1() {
7          P(mutex);    // prologo
8          <sezione critica: corpo della funzione op1>;
9          V(mutex);    // epilogo
10     }
11
12     public void opN() {
13         P(mutex);    // prologo
14         <sezione critica: corpo della funzione opN>;
15         V(mutex);    // epilogo
16     }
17 }
18 ...
19 tipo_risorsa ris;
20 ris.opi();

```

(il semaforo di mutua esclusione può assumere solo i valori 0 e 1) **semaforo binario**

Dimostriamo che il protocollo:

```

Semaphore mutex = 1;
...
p(mutex);
<sezione critica>
v(mutex);
...

```

risolve correttamente il problema della mutua esclusione.

Condizioni Necessarie Ipotesi:

```

Semaphore mutex = 1;
...

```

```

p(mutex);
<sezione critica>
v(mutex);
...

```

Teorema: Le condizioni necessarie per la mutua esclusione sono soddisfatte:

- a) Le sezioni critiche della stessa classe devono essere eseguite in modo **mutuamente esclusivo**.
- b) Non deve essere possibile il verificarsi di situazioni in cui i processi impediscono mutuamente la prosecuzione della loro esecuzione (**deadlock**).
- c) Quando un processo si trova all'esterno di una sezione critica **non può impedire** l'accesso alla stessa sezione (o a sezioni della stessa classe) ad altri processi.

Dimostrazione della Proprietà a) Teorema: Il numero N_{sez} dei processi nella sezione critica è sempre minore o uguale a uno:

$$0 \leq N_{sez} \leq 1$$

Dimostrazione:

$$N_{sez} = n_p - n_v$$

Dalla relazione invariante:

$$1 + n_v - n_p \geq 0 \Rightarrow n_p - n_v \leq 1 \Rightarrow N_{sez} \leq 1$$

Inoltre, poiché il protocollo impone che $p(mutex)$ preceda $v(mutex)$ in ogni processo, in ogni istante dell'esecuzione vale:

$$n_p \geq n_v \Rightarrow n_p - n_v \geq 0 \Rightarrow N_{sez} \geq 0 \quad \text{c.v.d.}$$

Corollario: Il semaforo può assumere solo 2 valori $\{0, 1\}$:

$$\text{Val}_{mutex} = 1 + n_v - n_p \leq 1$$

$$n_p - n_v \geq 0 \Rightarrow n_v - n_p \leq 0 \Rightarrow 1 + n_v - n_p \leq 1$$

Dimostrazione della Proprietà b) Teorema: Assenza di deadlock.

Dimostrazione per assurdo: Supponiamo ci sia deadlock:

1. Tutti i processi sono in attesa su $p(mutex)$
2. Nessun processo è nella sezione critica

Da (1) segue: $Val_{mutex} = 0$

Da (2): $N_{sez} = n_p - n_v = 0$

Sappiamo che:

$$Val_{mutex} = I_{mutex} - n_p + n_v = 1 - N_{sez}$$

$$\Rightarrow 0 = 1 - 0 \quad \text{assurdo!} \quad \mathbf{c.v.d.}$$

Dimostrazione della Proprietà c) Teorema: Un processo all'esterno della sezione critica non può impedire ad altri di entrare nella sezione critica.

Dimostrazione: Se nessun processo è nella sezione critica, deve essere possibile ad un qualunque processo di entrare nella sezione critica senza ritardi.

Se la sezione critica è libera $\rightarrow N_{sez} = n_p - n_v = 0$

$$Val_{mutex} = I_{mutex} - n_p + n_v \rightarrow Val_{mutex} = 1$$

$$\Rightarrow p \text{ non è bloccante} \quad \mathbf{c.v.d.}$$

In alcuni casi è consentito a più processi di eseguire contemporaneamente la stessa operazione su una risorsa, ma non operazioni diverse.

Quindi una soluzione potrebbe essere:

- definisco un semaforo mutex per la mutua esclusione tra operazioni
- prologo ed epilogo di op_i sono sezioni critiche quindi introduco un ulteriore semaforo di mutua esclusione m_i

Un esempio potrebbe essere il **problema dei lettori/scrittori**.

Sia data una risorsa condivisa F (ad esempio un file) che può essere acceduta dai thread concorrenti in due modi:

- **lettura;**
- **scrittura**

Una possibile soluzione di sincronizzazione potrebbe essere:

- la lettura è consentita a più thread contemporaneamente;
- la scrittura è consentita ad un thread alla volta;
- lettura e scrittura su F non possono avvenire contemporaneamente

```

1  semaphore mutex = 1;
2  semaphore ml = 1;
3  int contl = 0;
4
5  public void lettura(...) {
6      P(ml);
7      contl++;
8
9      if(contl == 1) {
10         P(mutex);}
11
12     V(ml);
13     <lettura del file>;
14     P(ml);
15     contl--;
16
17     if(contl == 0) {
18         V(mutex);}
19     v(ml);
20 }
21
22 public void scrittura(...) {
23     P(mutex);
24     <scrittura del file>;
25     V(mutex);
26 }

```

Semaforo evento

Un semaforo evento è un semaforo binario utilizzato per imporre un **vincolo di precedenza** tra le operazioni dei processi. (ad es. op_a deve essere eseguita da P1 solo dopo che P2 ha eseguito op_b).

Introduciamo quindi un semaforo **sem** inizializzato a **zero**:

- prima di eseguire op_a , P1 esegue P(sem)
- dopo aver eseguito op_b , P2 esegue V(sem)

Dimostriamo che la soluzione basata sui semafori evento garantisce il rispetto del vincolo di precedenza:

HP I due processi interagenti sono strutturati secondo il protocollo:

<pre> 1 semaphore sem = 0; 2 P1: 3 ... 4 p(sem); 5 op_a'; </pre>	<pre> P2: ... op_b'; v(sem); </pre>
---	-------------------------------------

Th L'operazione `op_a` viene eseguita sempre dopo `op_b`.

Dimostrazione per assurdo

Supponiamo per assurdo che sia possibile che `op_a` venga eseguita in un istante precedente a quello in cui viene eseguita `op_b`.

Consideriamo l'istante T : è già stata eseguita la `p(sem)`, ma non ancora la `v(sem)`.

$$n_{p_sem} - n_{v_sem} > 0 \quad (1)$$

L'invariante del semaforo, in questo caso, sarebbe data da:

$$0 \geq n_{p_sem} - n_{v_sem}$$

Ma dalla (1) abbiamo:

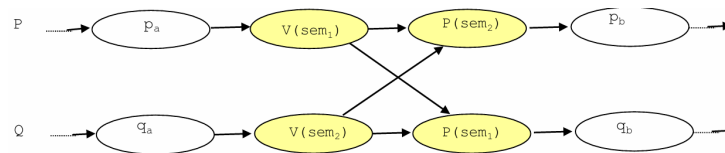
$$0 < n_{p_sem} - n_{v_sem} \Rightarrow \text{ASSURDO! (c.v.d.)}$$

3.4 problema del rendez-vous

Due processi P e Q eseguono ciascuno due operazioni p_a e p_b il primo e q_a e q_b il secondo.

Vincolo del rendez-vous: l'esecuzione di p_b da parte di P e q_b da parte di Q possono iniziare solo dopo che entrambi i processi hanno completato la loro prima operazione (p_a e q_a).

- Scambio di segnali temporali in modo simmetrico: ogni processo quando arriva all'appuntamento segnala di esserci arrivato e attende l'altro.
- Introduciamo due semafori evento `sem1` e `sem2`



Rendez-vous: struttura dei processi

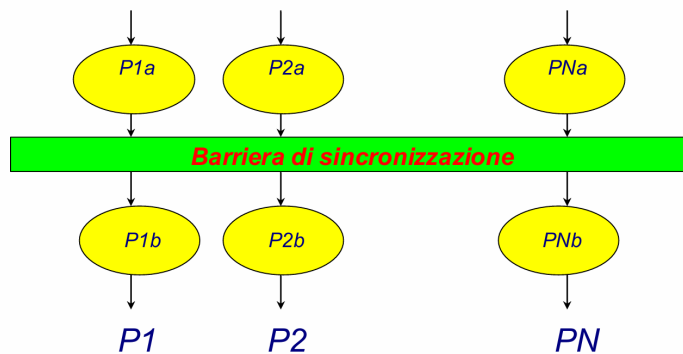
```
semaphore sem1= 0;
semaphore sem2= 0;
```

Q:	P:
...	...
<code>v(sem2);</code>	<code>v(sem1);</code>
<code>p(sem1);</code>	<code>p(sem2);</code>
...	...

la sincronizzazione è ottenuta con **2 semafori evento**

Soluzione: Barriera di sincronizzazione

L'esecuzione di ogni operazione P_{ib} è subordinata al completamento di tutte le istruzioni P_{ia} ($i=1,..N$)



```
1 //Variabili condivise
2 semaphore mutex=1;
3 semaphore barriera=0;
4 int completati=0; // numero dei thread che hanno
5 eseguito la prima operazione P_ia
6
7 //Struttura del thread i-simo P_i
8 <operazione a di Pi>
9 p(mutex);
10 completati++;
11 if (completati==N)
12 v(barriera);
13 v(mutex);
14 p(barriera);
15 v(barriera);
16 <operazione b di Pi>
```

Semafori binari composti

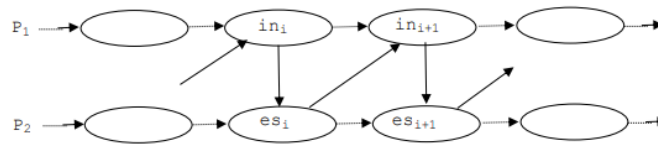
Un insieme di semafori usato in modo tale che:

- uno solo di essi sia inizializzato a 1 e tutti gli altri a 0
- ogni processo che usa questi semafori esegue sempre sequenze che iniziano con la P su uno di questi e termina con la V su un altro.

Due processi P1 e P2 si scambiano dati di tipo T utilizzando una memoria condivisa (buffer).

Quindi devono esserci dei vincoli di sincronizzazione:

- accessi al buffer mutuamente esclusivi
- P2 può prelevare un dato solo dopo che P1 lo abbia inserito
- P1, prima di inserire un dato, deve attendere che P2 abbia estratto il precedente



Utilizziamo quindi due semafori:

- **vu**, per realizzare l'attesa di P1, in caso di buffer pieno
- **pn**, per realizzare l'attesa di P2, in caso di buffer vuoto

Buffer inizialmente vuoto, $vu = 1$, $pn = 0$

```

1 void invio(T dato) {
2     P(vu);
3     inserisci(dato);
4     V(pn);
5 }

```

```

1 T ricezione() {
2     T dato;
3     P(pn);
4     dato = estrai();
5     V(vu);
6     return dato;
7 }

```

Semaforo condizione

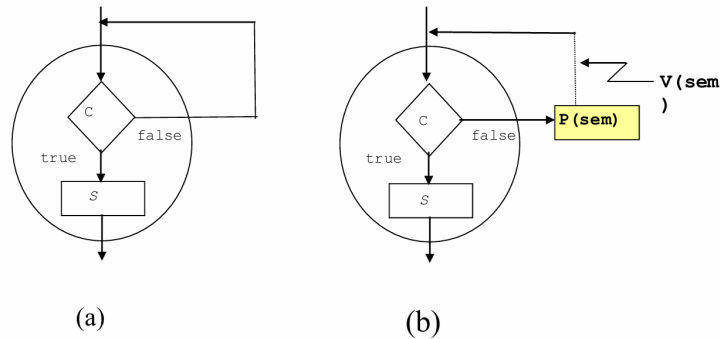
In alcuni casi l'esecuzione di un'istruzione S1 su una risorsa R è subordinata al verificarsi di una condizione C:

```

1 void op1(): region R << when(C) S1; >>

```

Il processo deve sospendersi se la condizione non è verificata e deve uscire dalla regione per consentire ad altri processi di eseguire altre operazioni su R per rendere vera la condizione C.



- Lo schema (a) presuppone una forma di attesa attiva da parte del processo che non trova soddisfatta la condizione.
- Nello schema (b) si realizza la region sospendendo il processo sul semaforo sem da associare alla condizione.
 - è necessaria un'altra operazione op2 che, chiamata da un altro processo, modifichi lo stato interno di R in modo che C diventi vera.
 - nell'ambito di op2 viene eseguita la **V(sem)** per risvegliare il processo.

Schema con attesa circolare

```

1 semaphore mutex = 1;
2 semaphore sem = 0;
3 int csem = 0;

```

```

1 public void op1() {
2     P(mutex);
3     while(!C) {
4         csem++;
5         V(mutex);
6         P(sem);
7         P(mutex);
8     }
9     S1;
10    V(mutex);
11 }

```

```

1 public void op2() {
2     P(mutex);
3     S2;
4     if(csem > 0) {
5         csem--;
6         V(sem);
7     }
8     V(mutex);
9 }

```

Schema con passaggio di testimone:

```

1 semaphore mutex = 1;
2 semaphore sem = 0;
3 int csem = 0;

```

```

1 public void op1() {
2     P(mutex);
3     if(!C) {
4         csem++;
5         V(mutex);
6         P(sem);
7         csem--;
8     }
9     S1;
10    V(mutex);
11 }

```

```

1 public void op2() {
2     P(mutex);
3     S2;
4     if(C && csem > 0) {
5         V(sem);
6     } else {
7         V(mutex);
8     }
9 }

```

Questo secondo schema (**passaggio del testimone**) è più efficiente del primo ma ha comunque delle limitazioni. Permette di risvegliare un solo processo alla volta poichè ad uno solo può passare il diritto di operare in mutua esclusione. Inoltre la condizione C (precondizione di S1) deve essere verificabile anche all'interno di op2. Ciò significa che non deve contenere variabili locali o parametri della funzione op1.

Semaforo condizione

I semafori risorsa sono semafori generali, quindi possono assumere qualunque valore ≥ 0 . Vengono generalmente impiegati per realizzare l'allocazione di risorse equivalenti, nel quale il valore del semaforo rappresenta il numero di risorse libere.

```

1 class tipo_gestore {
2     semaphore mutex = 1;    // semaforo di mutua esclusione
3     semaphore n_ris = N;    // semaforo risorsa
4     boolean libera[N];      // indicatori di risorsa libera
5
6     public tipo_gestore() {
7         for(int i = 0; i < N; i++) {
8             libera[i] = true;    // inizializzazione
9         }
10    }
11
12    public int richiesta() {
13        int i = 0;
14        P(n_ris);
15        P(mutex);
16        while(libera[i] == false) {
17            i++; }
18        libera[i] = false;
19        V(mutex);
20        return i; }
21 }

```

```

22     public void rilascio(int r) {
23         P(mutex);
24         libera[r] = true;
25         V(mutex);
26         V(n_ris);
27     } }

```

****Leggere esempi sulle condizioni di sincronizzazione. 04(sl 69-74)****

Semaforo privato

Un semaforo S si dice privato per un processo quando solo tale processo può eseguire la primitiva P sul semaforo S. La primitiva V sul semaforo può essere invece eseguita da qualunque processo. Generalmente un semaforo privato viene inizializzato con il valore zero.

I semafori privati vengono utilizzati per realizzare particolari politiche di allocazione di risorse:

- il processo che acquisisce la risorsa può eventualmente sospendersi sul suo semaforo privato (se la condizione di sincronizzazione non è rispettata);
- chi rilascia la risorsa, risveglierà uno tra i processi sospesi, in base alla politica scelta, mediante una V sul semaforo privato del processo scelto.

Allocazione di risorse, **primo schema**:

```

1  class tipo_gestore_risorsa {
2      <struttura dati del gestore>;
3      semaphore mutex = 1;
4      semaphore priv[n] = {0, 0, ..., 0}; // semafori privati
5
6      public void acquisizione(int i) {
7          P(mutex);
8          if(<condizione di sincronizzazione>) {
9              <allocazione della risorsa>;
10             V(priv[i]);
11         } else {
12             <registra la sospensione del processo>;
13         }
14         V(mutex);
15         P(priv(i));
16     }
17
18     public void rilascio() {
19         int i;
20         P(mutex);
21         <rilascio della risorsa>;

```

```

22         if(<esiste almeno un processo sospeso per il quale
           ↳la condizione risulta true>) {
23             <scelta fra i processi sospesi quello destinato
               ↳alla riattivazione (Pi)>;
24             <allocazione della risorsa a Pi>;
25             <registrare che Pi non e piu sospeso>;
26             V(priv[i]);
27         }
28         V(mutex);
29     }
30 }

```

Proprietà del primo schema:

- la sospensione del processo, nel caso in cui la condizione di sincronizzazione non sia soddisfatta, non può avvenire entro la sezione critica in quanto ciò impedirebbe ad un processo che rilascia la risorsa di accedere a sua volta alla sezione critica e di riattivare il processo sospeso. E quindi la sospensione avviene fuori dalla sezione critica.
- la specifica del particolare algoritmo di assegnazione della risorsa non è opportuno che sia realizzata nella primitiva V.

questo schema può presentare i seguenti problemi:

- l'operazione P sul semaforo privato viene sempre eseguita anche quando il processo richiedente non deve essere bloccato.
- Il codice relativo all'assegnazione della risorsa viene duplicato nelle procedure acquisizione e rilascio.

Allocazione di risorse, **secondo schema** (supera i limiti del primo):

```

1  class tipo_gestore_risorsa {
2      <struttura dati del gestore>;
3      semaphore mutex = 1;
4      semaphore priv[n] = {0, 0, ..., 0};
5
6      public void acquisizione(int i) {
7          P(mutex);
8          if(! <condizione di sincronizzazione>) {
9              <registrare la sospensione del processo>;
10             V(mutex);
11             P(priv[i]);
12             <registrare che il processo non e piu sospeso>;
13         }
14         <allocazione della risorsa>;
15         V(mutex);
16     }

```

```

17
18     public void rilascio() {
19         int i;
20         P(mutex);
21         <rilascio della risorsa>;
22         if(<esiste almeno un processo sospeso per il quale
23             ↳la condizione risulta true>) {
24             <scelta del processo Pi da riattivare>;
25             V(priv[i]);
26         } else {
27             V(mutex);
28         }
29     }

```

Rispetto al primo schema, in questa soluzione risulta più complesso realizzare la riattivazione di più processi per i quali risulti vera contemporaneamente la condizione di sincronizzazione. Infatti il processo che rilascia la risorsa attiva al più un processo sospeso, il quale dovrà a sua volta provvedere alla riattivazione di eventuali altri processi.

Soluzione esempio 1: prod e cons con priorità (schema 1)

```

1  class buffer {
2      int richiesta[num_proc]=0; /*richiesta[i]= numero di celle
3          ↳richieste da Pi*/
4      int sospesi=0; /*numero dei processi prod.sospesi*/
5      int vuote=n; /*numero di celle vuote del buffer*/
6      semaphore mutex=1;
7      semaphore priv[num_proc]={0,0,...0};
8
9      public void acquisizione(int m, int i) {
10         /* m dim. messaggio, i id.del processo chiamante */
11         P(mutex);
12         if (sospesi==0 && vuote>=m) { //assegnaz. m celle a Pi
13             vuote=vuote-m;
14             V(priv[i]);
15         }
16         else {
17             sospesi++;
18             richiesta[i]=m;
19         }
20         V(mutex);
21         P(priv[i]);
22     }
23     public void rilascio(int m){ /* m num. celle rilasciate*/
24         int k;
25         P(mutex);
26         vuote+=m;

```

```

26     while (sospesi!=0) {
27         <individuazione tra i processi sospesi del processo
           ↪ Pk con la max richiesta> ;
28         if (richiesta[k]<=vuote) { //ass. a Pk
29             vuote= vuote-richiesta[k];
30             richiesta[k]=0;
31             sospesi--;
32             V(priv[k]);
33         }
34         else break; /* fine while */
35     }
36     V(mutex);
37 }
38 }

```

Soluzione esempio 2: pool di risorse equivalenti + priorità (schema 2: passaggio di testimone)

Introduciamo le variabili:

PS[i]: variabile logica che assume il valore vero se il processo Pi è sospeso; il valore falso diversamente.

libera[j]: variabile logica che assume il valore falso se la risorsa j esima è occupata; il valore vero diversamente.

disponibili: esprime il numero delle risorse non occupate;

sospesi: è il numero dei processi sospesi;

mutex: semaforo di mutua esclusione;

priv[i]: il semaforo privato del processo Pi.

```

1  class tipo_gestore {
2      semaphore mutex=1;
3      semaphore priv[num_proc]={0,0,...0} /*sem. privati */
4      int sospesi=0; /*numero dei processi sospesi*/
5      boolean PS[num_proc]={false, false,..., false};
6      int disponibili=num_ris; /*numero di risorse disp.*/
7      boolean libera[num_ris]={true, true,..., true};
8
9      public int richiesta(int proc)
10     {
11         int i =0;
12         P(mutex);
13         if (disponibili==0)
14         {
15             sospesi ++;
16             PS[proc]=true;
17             V(mutex);
18             P(priv[proc]);
19             PS[proc]=false;
20             sospesi -- ;
21         }
22         while (! libera[i]) i++;
23         libera[i]=false;

```



```

22     disponibili--;
23     V(mutex );
24     return i;
25 }
26
27 public void rilascio (int r) { /* r indice risorsa ril. */
28     libera[r]=true;
29     disponibili++;
30     if (sospesi>0)
31     { <seleziona il processo Pj a massima
32       priorit  tra quelli sospesi utilizzando PS>;
33       V(priv[j]);
34     }
35     else V(mutex);
36 }
37 }

```

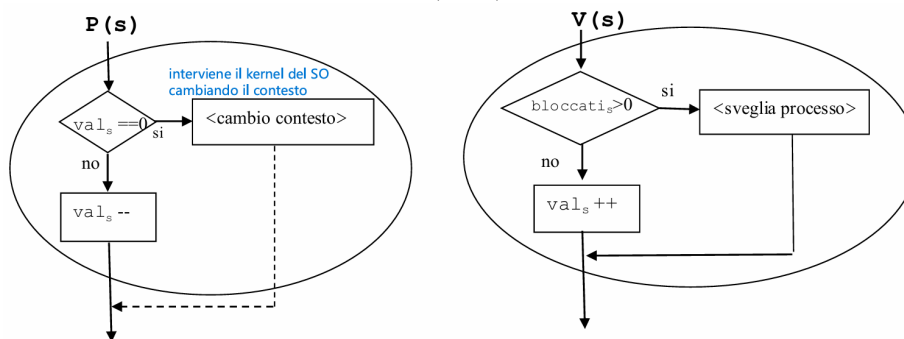
Considerazioni: Ogni processo che nella fase di acquisizione della risorsa trova la condizione di sincronizzazione non soddisfatta, deve lasciare traccia in modo esplicito della sua sospensione entro la sezione critica.

Il processo che libera la risorsa deve infatti eseguire la primitiva V(priv(i)) solo se esistono processi sospesi. In tutte le soluzioni   stata introdotta un'apposita variabile per indicare il numero dei processi sospesi.

Realizzazione dei semafori

In sistemi operativi multiprogrammati, il semaforo viene realizzato dal kernel, che, sfruttando i meccanismi di gestione dei processi (sospensione e riattivazione) elimina la possibilit  di attesa attiva.

  possibile definire la P e la V nel modo seguente, garantendo comunque la validit  delle propriet  del semaforo (s.p.d).



```

1 typedef struct{
2     int contatore;
3     coda queue;
4 }semaforo;

```

Una p su un semaforo con contatore a 0, sospende il processo nella coda queue, altrimenti contatore viene decrementato.

Una v su un semaforo la cui coda queue non è vuota, estrae un processo dalla coda, altrimenti incrementa il contatore

Architettura monoprocesso

Supponendo di avere le interruzioni disabilitate (per garantire l'atomicità di p e v)

```
1 void P(semaphore s) {
2     if(s.contatore==0)
3         <sospensione del processo nella coda associata a s>;
4     else s.contatore--;
5 }
6
7 void V (semaphore s)
8     { if(s.queue!=NULL)
9         <estrazione del primo processo dalla coda s.queue, che
          ↳viene riportato nello stato di ready>
10        else s.contatore++;
11 }
```

L'implementazione di p e v è parte del nucleo della macchina concorrente e dipende dal tipo di architettura hardware (monoprocesso, multiprocesso, ecc.) e da come il nucleo rappresenta e gestisce i processi concorrenti.

4 Modello a scambio di messaggi

Il modello architetturale di una macchina concorrente è caratterizzato da:

- Ogni processo può accedere esclusivamente alle risorse allocate nella propria memoria locale.
- Ogni risorsa del sistema è accessibile direttamente ad un **solo processo (il Gestore della risorsa)**
- Se una risorsa è necessaria a più processi applicativi, ciascuno di questi (processi clienti) dovrà delegare l'unico processo che può operare sulla risorsa (processo server) all'esecuzione delle operazioni richieste; al termine di ogni operazione il server restituirà al cliente gli eventuali risultati
- In questo modello il concetto di **gestore** di una risorsa coincide con quello di processo server
- Ogni processo, per usufruire dei servizi offerti da una risorsa, dovrà comunicare con il gestore
- Il meccanismo di base utilizzato dai processi per qualunque tipo di interazione è costituito dal **meccanismo di scambio di messaggi**

4.1 Canale di comunicazione

Il concetto fondamentale del modello a scambio di messaggi è il **canale**, ovvero un **collegamento logico mediante il quale due o più processi comunicano**.

È il nucleo della macchina concorrente a realizzare l'astrazione "canale" come meccanismo primitivo per lo scambio di informazioni. Ed è compito del linguaggio di programmazione offrire gli strumenti linguistici di alto livello per:

- specificare i canali di comunicazione
- utilizzarli per esprimere le interazioni tra i processi

Caratteristiche del canale di comunicazione

I parametri che caratterizzano il concetto di canale sono:

- la **direzione del flusso dei dati** che un canale può trasferire
- la **designazione del canale** e dei processi origine e destinatario di ogni comunicazione
- il tipo di **sincronizzazione** fra i processi comunicanti

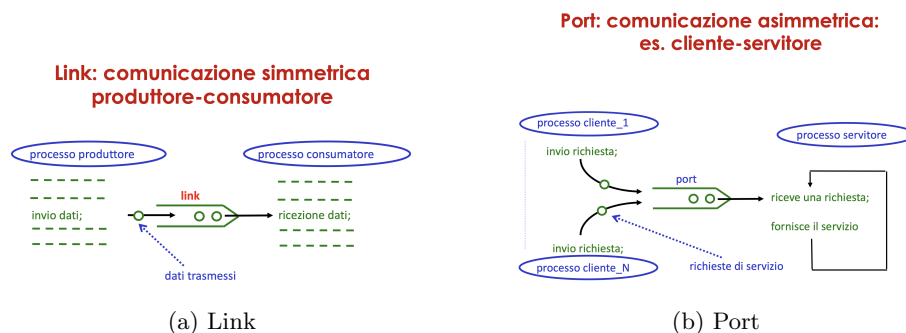
4.2 Tipi di Canali

I canali possono essere distinti in più gruppi. Facendo riferimento alla **direzione del flusso dei dati**, possiamo distinguere due tipi di canali:

- Canale **monodirezionale**: consente il flusso dei messaggi in una sola direzione (mittente verso ricevente)
- Canale **bidirezionale**: può essere usato sia per inviare che per ricevere informazioni

Facendo invece riferimento alla **designazione dei processi comunicanti**, è possibile definire tre tipi di canale:

- **Link**: da uno-a-uno (canale simmetrico)
- **Port**: da molti-a-uno (canale asimmetrico)
- **Mailbox**: da molti-a-molti (canale asimmetrico perché la cardinalità dei mittenti può essere diversa da quella dei destinatari)



Con riferimento alla modalità di **sincronizzazione** tra i processi comunicanti, possiamo individuare tre tipi di canale:

- Comunicazione **asincrona**
- Comunicazione **sincrona**
- Comunicazione con **sincronizzazione estesa**

4.2.1 Comunicazione asincrona

Semantica:

Il processo mittente **continua la sua esecuzione** immediatamente dopo l'invio del messaggio.

Effetti:

- La ricezione del messaggio può avvenire in un istante successivo all'invio: il messaggio ricevuto contiene informazioni che non possono essere attribuite allo stato attuale del mittente e quindi c'è una difficoltà nella verifica dei programmi.
- L'invio di un messaggio non è un punto di sincronizzazione per mittente e destinatario.

Proprietà:

- **Carenza espressiva** (difficoltà di verifica dei programmi)
- **L'assenza di vincoli di sincronizzazione** tra mittente e destinatario favorisce il **grado di concorrenza**
- Da un punto di vista realizzativo, sarebbe necessario un **buffer di capacità illimitata**

Realizzazione:

Ogni implementazione prevede inevitabilmente un limite alla capacità del buffer. Nel caso di un buffer pieno, se si vuole mantenere immutata la semantica, occorre che il supporto a tempo di esecuzione provveda a sospendere il processo che invia il messaggio.

4.2.2 Comunicazione Sincrona: rendez-vous semplice

Semantica:

Il primo dei due processi comunicanti che esegue l'invio (mittente) o la ricezione (destinatario) si sospende in attesa che l'altro sia pronto ad eseguire l'operazione corrispondente.

Effetti:

L'invio di un messaggio è un **punto di sincronizzazione**: ogni messaggio ricevuto contiene informazioni attribuibili allo stato attuale del processo mittente. Ciò semplifica la scrittura e la verifica dei programmi.

Inoltre **non è necessaria l'introduzione di un buffer**: un messaggio può essere inviato solamente se il ricevente è pronto a riceverlo.

Comunicazione asincrona vs sincrona

- L'invio con semantica sincrona è più espressivo
- La semantica asincrona consente di raggiungere maggiore concorrenza/parallelismo
- La realizzazione del canale nella comunicazione asincrona richiede opportuna struttura per l'accodamento dei messaggi.

Comunicazione con sincronizzazione estesa: rendez-vous esteso

Assunzione: ogni messaggio inviato rappresenta una richiesta al destinatario dell'esecuzione di una certa azione

Semantica: il processo mittente rimane in attesa fino a che il ricevente non ha terminato di svolgere l'azione richiesta

4.3 Costrutti linguistici e primitive per esprimere la comunicazione

4.3.1 Definizione del canale (port)

Un canale **port** identifica un canale asimmetrico **multi-a-uno**

port <tipo> <identificatore>;

```
1 port int ch1;
```

L'identificatore `ch1` denota un canale utilizzato per trasferire messaggi di tipo intero. **Port** viene dichiarato locale ad un processo (il ricevente) ed è visibile ai processi mittenti mediante la dot notation:

<nome del processo>.<identificatore del canale>

4.3.2 Primitive di comunicazione sincrone

Send è la primitiva che esprime l'invio di un messaggio:

send(<valore>) to <porta>;

- <**porta**>: identifica in modo univoco il canale a cui inviare il messaggio;
- <**valore**>: identifica una espressione dello stesso tipo di <porta> e rappresenta il contenuto del messaggio inviato.

Semantica: può essere

- **asincrona** → canale bufferizzato;
- **sincrona** → canale a capacità nulla;

A seconda della semantica e delle caratteristiche del canale la send può sospendere o meno il processo che la esegue. Ad esempio:

- **Send sincrona:** il processo attende che il destinatario esegua la primitica di ricezione (receive) corrispondente;
- **Send asincrona:** il processo attende solo se il canale è pieno.

Receive

P = receive(<variabile>) from <porta>;

- **<porta>**: identifica il **canale**, locale al processo ricevente, dal quale ricevere il messaggio;
- **<variabile>**: è l'identificativo della variabile, dello stesso tipo di **<porta>**, a cui assegnare il valore del messaggio ricevuto.

Semantica:

- **Default: Semantica Bloccante.** La primitiva sospende il processo se non ci sono messaggi sul canale; quando c'è almeno un messaggio nel canale, ne estrae il primo e ne assegna a **<variabile>** il valore. La receive restituisce un valore del tipo predefinito **process** che identifica il nome del processo mittente.
- Alcuni linguaggi offrono anche una semantica **non bloccante**: se il canale è vuoto, il processo continua; se contiene almeno un messaggio, estrae il primo e lo assegna a **<variabile>**.

4.3.3 Receive Bloccante e Modello Client-Server

Il **Server** è un processo che si occupa di servire le richieste di altri processi, detti **Client**. Il server ha la possibilità di eseguire diversi servizi, ognuno attivato in seguito alla ricezione di un messaggio di tipo diverso. A questo scopo il server gestisce più **canali d'ingresso** (ad esempio porte), ognuno dedicato alla ricezione delle richieste di un particolare servizio.

Il problema principale della receive bloccante su un modello client-server sta nel fatto che il server specifica il canale sul quale eseguire ogni receive. Se i canali sono più di uno, il server deve eseguire in sequenza le receive su ogni canale: poiché la receive ha semantica bloccante, c'è la possibilità di blocco su un canale mentre contemporaneamente ci sono messaggi in attesa di essere ricevuti su altri canali.

Una possibile soluzione potrebbe essere una **receive con semantica non bloccante**: verifica lo stato del canale, restituisce un messaggio (se presente) o un'indicazione del canale vuoto (non bloccante). Non sospende mai il processo che la esegue. Il server può eseguire un ciclo di ispezione/ricezione di/da tutti i canali.

In caso di presenza contemporanea di messaggi su più canali la scelta può essere fatta secondo diversi criteri (priorità, stato della risorsa) oppure in modo non deterministico.

Problema dell'attesa attiva: se tutti i canali sono vuoti, il server continua a iterare.

Meccanismo di ricezione ideale:

- consente al processo server di verificare contemporaneamente la disponibilità di messaggi su più canali;

- abilità di ricezione di un messaggio da un qualunque canale contenente messaggi;
- quando tutti i canali sono vuoti, blocca il processo in attesa che arrivi un messaggio, qualunque sia il canale su cui arriva.

Questo meccanismo è realizzabile tramite i **comandi con guardia**.

4.3.4 Comandi con Guardia

`<guardia> -> <istruzione>`

dove **guardia** è costituita dalla coppia: (**espressione booleana**; **receive**)

- **espressione booleana** viene detta guardia logica;
- **receive** con semantica bloccante e viene detta guardia d'ingresso;

Valutazione della guardia

`<guardia> := (<espressione booleana>; <receive>)`

La valutazione della guardia può fornire tre diversi valori:

- **guardia fallita**: se l'espressione ha valore false e il comando eseguito con guardia fallisce;
- **guardia ritardata**: se l'espressione booleana ha valore true e nel canale su cui viene eseguita non ci sono messaggi, il processo che esegue il comando viene sospeso e quando arriverà il primo messaggio nel canale riferito dalla guardia d'ingresso il processo verrà riattivato, eseguirà la receive e successivamente l'istruzione;
- **guardia valida**: se l'espressione booleana ha valore true e nel canale c'è almeno un messaggio, quindi il processo esegue la receive e successivamente l'istruzione.

Comando con guardia alternativo

```

1 select {
2   [] <guardia_1> -> <istruzione_1>;
3   ...
4   [] <guardia_n> -> <istruzione_n>;
5 }
```

Il comando con guardia alternativo (select) racchiude un numero arbitrario di comandi con guardia semplici.

Vengono valutate le guardie di tutti i rami e si possono verificare 3 casi:

- **se una o più guardie sono valide:** viene scelto, in maniera non deterministica, uno dei rami con guardia valida e la relativa guardia viene eseguita, viene quindi eseguita (eseguita la receive) l'istruzione relativa al ramo scelto, e con ciò termina l'esecuzione dell'intero comando alternativo;
- **se tutte le guardie non fallite sono ritardate:** il processo in esecuzione si sospende in attesa che arrivi un messaggio che abilita la transizione di una guardia da ritardata a valida e a quel punto procede come nel caso precedente;
- **se tutte le guardie sono fallite:** il comando termina (e la select fallisce).

Comando con guardia ripetitivo

```

1 do {
2   [] <guardia_1> -> <istruzione_1>;
3   ...
4   [] <guardia_n> -> <istruzione_n>;
5 }
```

Vengono valutate le guardie di tutti i rami e si possono verificare 3 casi:

- **se una o più guardie sono valide:** viene scelto in maniera non deterministica uno dei rami con guardia valida e la relativa guardia viene eseguita (eseguita la receive), viene quindi eseguita l'istruzione relativa al ramo scelto, e poi si passa al ciclo successivo (ripetizione);
- **se tutte le guardie non fallite sono ritardate:** il processo in esecuzione si sospende in attesa che arrivi un messaggio che abilita la transizione da una guardia ritardata a valida; e a quel punto procede come nel caso precedente (si itera);
- **se tutte le guardie sono fallite:** l'esecuzione del comando termina.

4.3.5 Primitive di comunicazione asincrona

Nel modello a scambio di messaggi, lo strumento di comunicazione di più basso livello è la **send asincrona**. Nel quale ad ogni operazione associata alla risorsa corrisponde un diverso servizio.

Accesso a risorse "condivise" → processi servitori

Verranno ora analizzate delle **soluzioni di problemi di sincronizzazione con send asincrona**.

Prendendo in considerazione alcuni tipici problemi di interazione nel modello a scambio di messaggi. Quindi ci si chiede: data una risorsa, come implementare nel modello a scambio di messaggi (con send asincrona) il gestore della risorsa nel caso in cui sia previsto:

- una sola operazione;
- più operazioni mutuamente esclusive;
- più operazioni con condizioni di sincronizzazione.

1. Risorsa condivisa con una sola operazione

Risorsa condivisa che mette a disposizione di un insieme di processi "clienti" una sola operazione con il solo vincolo della mutua esclusione.

Soluzione nel modello a memoria comune: il gestore è un **monitor** con una operazione entry

```

1 monitor gestore {
2     tipo_var var;
3     <eventuale inizializzazione>;
4     entry tipo_out fun(tipo_in x) {
5         <corpo della funzione fun>;
6     }
7 }
8 gestore ris; // istanza del gestore

```

```

1 thread client {
2     tipo_in a;
3     tipo_out b;
4
5     b = ris.fun(a);
6 }

```

Soluzione nel modello a scambio di messaggi:

La risorsa viene gestita da un **processo server** che offre un unico servizio senza condizioni di sincronizzazione.

```

1 // cliente: un solo
2   ↳ canale risposta
3   ↳ di tipo tipo_out
4
5 process cliente {
6     port tipo_out
7       ↳ risposta;
8     tipo_in a;
9     tipo_out b;
10    process p;
11    ...
12    send(a) to server.
13      ↳ input;
14    p = receive(b) from
15      ↳ risposta;
16    ...
17 }

```

```

1 // server: un solo canale input
2   ↳ di tipo tipo_in
3
4 tipo_out fun(tipo_in x);
5
6 process server {
7     port tipo_in input;
8     tipo_var var;
9     process p;
10    tipo_in x;
11    tipo_out y;
12    <eventuale inizializzazione>;
13    while(true) {
14        p = receive(x) from input;
15        y = fun(x);
16        send(y) to p.risposta;
17    }
18 }

```

2. Risorsa condivisa con più operazioni

Risorsa condivisa che mette a disposizione di un insieme di processi clienti due operazioni con il solo vincolo della mutua esclusione.

Soluzione nel modello a memoria comune: il gestore è un monitor con due operazioni entry:

```
1 monitor gestore {  
2     tipo_var var;  
3     ...  
4     entry tipo_out1 fun1(tipo_in1 x1)  
5         { <corpo della funzione fun1>; }  
6     entry tipo_out2 fun2(tipo_in2 x2)  
7         { <corpo della funzione fun2>; }  
8 }  
9  
10 gestore ris;
```

```
1 thread client {  
2     tipo_in1 a1;  
3     tipo_in2 a2;  
4     tipo_out1 b1;  
5     tipo_out2 b2;  
6     ...  
7     b1 = ris.fun1(a1);  
8     ...  
9     b2 = ris.fun2(a2);  
10 }
```

Soluzione nel modello a scambio di messaggi:

La risorsa è gestita da un processo servitore che offre 2 servizi senza condizioni di sincronizzazione.

- soluzione senza comandi con guardia: un solo canale per entrambi i tipi di richiesta.
- Soluzione con comandi con guardia: per ogni servizio il server apre un canale distinto.

```

1 //senza comandi con guardia
2 typedef struct{
3     enum (fun1,fun2)servizio;
4     union{
5         tipo_in1 x1;// parametri
6             ↪fun1
7         tipo_in2 x2;// parametri
8             ↪fun2
9     }parametri;
10    }in_mess;//tipo del mess.
11
12 tipo_out1 fun1 (tipo_in1 x1);
13 tipo_out2 fun2 (tipo_in2 x2);
14
15 process cliente {
16     port tipo_out1 risposta1;
17     port tipo_out2 risposta2;
18     tipo_in1 a1; tipo_in2 a2;
19     in_mes M; process p;
20     tipo_out1 b1; tipo_out2 b2;
21     <init M, in base al servizio
22         ↪ scelto>
23     if (<servizio1>) then {
24         send(M)to server.input;
25         p=receive(b1)from
26             ↪risposta1;
27     } else {
28         send(M)to server.input;
29         p=receive(b2)from
30             ↪risposta2;
31     }
32     .. }

```

```

1 process server {
2     port in_mes input;
3     tipo_var var; process p;
4     in_mes richiesta;
5     ↪tipo_out1 y1;
6     tipo_out2 y2;
7     while (true) {
8         p=receive (richiesta)
9             ↪from input;
10        switch (richiesta.
11            ↪servizio) {
12            case fun1: {y1=
13                fun1(richiesta.
14                    ↪parametri.x1);
15                send(y1) to p.
16                ↪risposta1;
17                break; }
18            case fun2: {y2=
19                fun2(richiesta.
20                    ↪parametri.x2);
21                send(y2) to p.
22                ↪risposta2;
23                break; }
24        }
25    }
26 }

```

<pre> 1 //con comandi con guardia 2 tipo_out1 fun1 (tipo_in1 x1); 3 tipo_out2 fun2 (tipo_in2 x2); 4 5 process cliente { 6 port tipo_out1 risposta1; 7 port tipo_out2 risposta2; 8 tipo_in1 a1; tipo_in2 a2; 9 process p; 10 tipo_out1 b1; tipo_out2 b2; 11 12 <inizializzazione a1 o a2> 13 if (<servizio1>) then { 14 send(a1)to server.input1; 15 p=receive(b1)from 16 ↳risposta1; 17 } else { 18 send(a2)to server.input2; 19 p=receive(b2)from 20 ↳risposta2; 21 } 22 .. } </pre>	<pre> 1 process server { 2 port tipo_in1 input1; // 3 ↳chan per le rich. 4 ↳relative a fun1 5 port tipo_in2 input2; // 6 ↳chan per le rich. 7 ↳relative a fun2 8 tipo_var var; process p; 9 tipo_in1 x1; tipo_in2 x2; 10 tipo_out1 y1;tipo_out2 y2; 11 {< eventuale init>;} 12 do { 13 []p = receive (x1) from 14 ↳input1;-> 15 y1=fun1(x1); 16 send (y1) to p. 17 ↳risposta1; 18 []p = receive (x2) from 19 ↳input2;-> 20 y2=fun2(x2); 21 send (y2) to p. 22 ↳risposta2; 23 } 24 } </pre>
---	---

3. Risorsa condivisa con più operazioni e condizioni di sincronizzazione

La risorsa è gestita da un processo servitore che offre **due operazioni di sincronizzazione**

Soluzione nel modello a memoria comune: la risorsa è gestita da un monitor con 2 entry e 2 variabili condizione *condition c1, c2*;

<pre> 1 entry tipo_out1 op1(2 ↳tipo_in1 x1) { 3 ... 4 if(!cond1) 5 wait(c1); 6 ... 7 signal(c2); 8 ... 9 } </pre>	<pre> 1 entry tipo_out2 op2(2 ↳tipo_in2 x2) { 3 ... 4 if(!cond2) 5 wait(c2); 6 ... 7 signal(c1); 8 ... 9 } </pre>
--	--

Soluzione nel modello a scambio di messaggi:

La risorsa è gestita da **un processo servitore con due servizi:**

- viene associato ad ogni servizio un canale distinto;
- la ricezione delle richieste in arrivo viene realizzata con un **comando con guardia ripetitivo**, con due rami (uno per servizio); in ogni ramo la guardia logica esprime la **condizione di sincronizzazione**

```

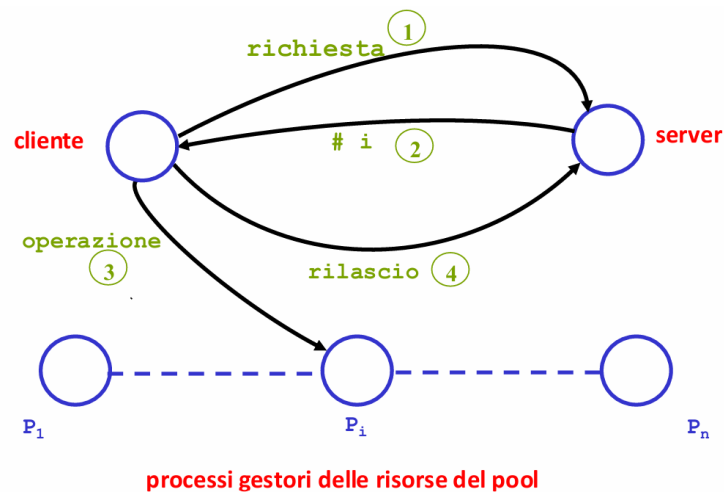
1  tipo_out1 fun1(tipo_in1 x1);
2  tipo_out2 fun2(tipo_in2 x2);
3
4  process server {
5      port tipo_in1 input1; port tipo_in2 input2;
6      tipo_var var;
7      tipo_in1 x1; tipo_in2 x2;
8      tipo_out1 y1; tipo_out2 y2;
9      ...
10     do { //la condizione non deve dipendere dal contenuto
11         ↪del messaggio
12         [] (cond1);
13             p = receive (x1) from input1; -> {
14                 y1 = fun1(x1);
15                 send(y1) to p.risposta1;
16             }
17         [] (cond2);
18             p = receive (x2) from input2; -> {
19                 y2 = fun2(x2);
20                 send (y2) to p.risposta2;
21             }
22     }
23 }
```

Esempi di processi servitori: gestore di un pool di risorse equivalenti

Ogni risorsa è gestita da un processo.

n risorse $R_1, R_2, \dots, R_n \Rightarrow$ n processi gestori: $P_1, P_2, \dots, P_n$.

Processo server (gestore del pool): nell'allocazione di una risorsa R_i , indica al processo cliente richiedente il processo P_i che gestisce la risorsa. Il cliente invia al server una richiesta per ottenere l'indice di una risorsa disponibile. Il server restituisce l'indice i del processo che gestisce la risorsa disponibile; successivamente il cliente interagisce direttamente con P_i per richiedere l'esecuzione di operazioni sulla risorsa R_i . Quando il client avrà terminato di utilizzare la risorsa R_i , invierà al server un messaggio per indicare il <<rilascio>> di R_i .



```

1 process cliente{
2   port int risorsa; signal s; int r;
3   process p, server;
4   ...
5   send(s) to server.richiesta; // richiesta di una risorsa
6   p=receive (r) from risorsa; //attesa della risposta
7   <uso delle risorsa r-sima>
8   send (r) to server.rilascio; //rilascio della risorsa
9   ...
10 }
11
12 process server{
13   // la richiesta e un messaggio privo di contenuto
14   ⇔ informativo (tipo signal)
15   port signal richiesta;
16   port int rilascio; int disponibili=N;
17   boolean libera[N];
18   process p ; signal s; int r;
19   for (int i=0; i<N; i++) libera[i]=true; //inizializza
20   do {
21     [] (disponibili>0); p= receive(s)from richiesta; ->
22     int i=0;
23     while (!libera[i]) i++;
24     libera[i] = false;
25     disponibili--;
26     send (i) to p.risorsa;
27     [] p=receive(r)from rilascio;->
28     disponibili++;
29     libera[r] = true;
30   }
31 }

```

Specifica di strategie di priorità

Si consideri un server che gestisce un pool di risorse adottando una politica basata su priorità. **Criterio di priorità:** il server deve privilegiare le richieste di P0 rispetto a quelle di P1 e queste rispetto a quelle di P2...

- Quando un qualunque processo P_i invia al server una richiesta, questa viene servita se ci sono risorse disponibili, mentre, in caso contrario, P_i rimane sospeso.
- Quando una risorsa viene rilasciata, il server deve essere in grado di riconoscere se ci sono processi clienti sospesi e, in questo caso, quali sono.
- Tra i clienti sospesi, va scelto quello a priorità più alta (indice più basso).

```
1 process cliente{
2   port int risorsa; signal s; int r; ...
3   send(s) to server.richiesta;
4   p=receive (r) from risorsa;
5   <uso delle risorsa r-sima>
6   send (r) to server.rilascio;
7   ...
8 }
9
10 process server{
11   port signal richiesta; port int rilascio;
12   int disponibili=N; boolean libera[N];
13   process p ; signal s; int r;
14   int sospesi=0; boolean bloccato[M];
15   process client[M];
16   { //inizializzazione
17     for (int i=0; i<N; i++) libera[i]=true;
18     for (int j=0; j<M; j++) bloccato[j]=false;
19     client[0]="P0";... client[M-1]="PM-1";
20   } do {
21     [] p=receive(s) from richiesta;->
22     if (disponibili>0) {
23       int i=0;
24       while (!libera[i]) i++;
25       libera[i] = false;
26       disponibili--;
27       send (i) to p.risorsa;}
28     else { int j=0; sospesi++;
29           while(client[j]!=p) j++;
30           bloccato[j]=true;}
31     [] p:= receive (r) from rilascio; ->
32     if (sospesi == 0) {
33       disponibili++;
34       libera[r] = true; }
35     else { int i=0;
```

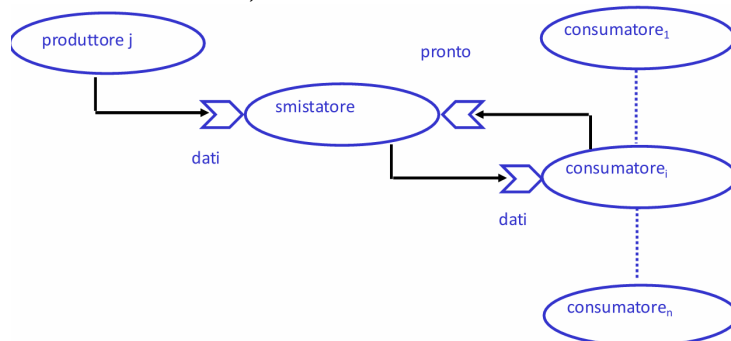


```

36     while (!bloccato[i]) i++;
37     sospesi --;
38     bloccato[i] = false;
39     send (r)to client[i].risorsa;}
40 }
41 }

```

Esempi di processi servitori: produttori e consumatori (scambio di dati multi-a-molti)



Il processo **smistatore** ha due porte:

- **dati** per ricevere dati dai produttori
- **pronto** per ricevere il messaggio inviato dai consumatori pronti a ricevere il messaggio. Il messaggio inviato dal consumatore è privo di contenuto informativo (messaggio di tipo signal).

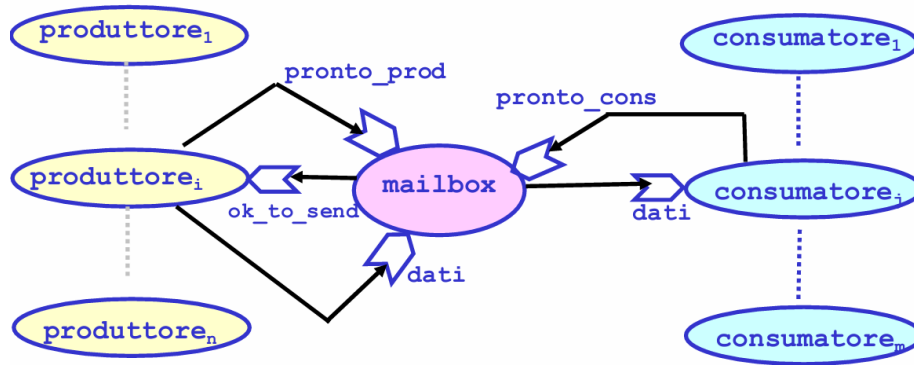
Se T è il tipo generico dei messaggi inviati dai produttori:

- La porta dati del server è di tipo T.
- La porta dati del consumatore è di tipo T.

Quando desidera ricevere un messaggio, ogni consumatore invia un messaggio sulla porta pronto del server e si mette in attesa di ricevere il messaggio sulla propria porta dati.

Esempi di servitori: mailbox (multi-a-molti) con capacità limitata

- Il produttore non può inviare un nuovo messaggio se il buffer è pieno (cioè, i precedenti N messaggi non sono stati ancora ricevuti); in tal caso attende.
-> prevediamo la porta "pronto-prod" per la verifica della condizione di inserimento (tramite una guardia)
- Il consumatore non può estrarre un messaggio se il buffer è vuoto; in tal caso attende.
-> prevediamo la porta "pronto-cons" per la verifica della condizione di estrazione (tramite una guardia)



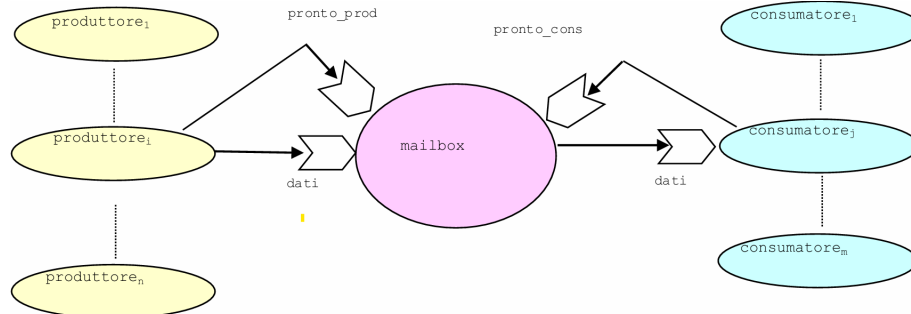
```

1 process mailbox{
2     port T dati; // implementa il buffer
3     port signal pronto_prod, pronto_cons;
4     T messaggio;
5     process prod, cons;
6     signals;
7     int contatore=0;
8     do {
9         [] (contatore<N); prod=receive(s) from pronto_prod; ->
10        { contatore++;
11          send(s) to prod.ok_to_send; }
12        [] (contatore>0); cons=receive(s) from pronto_cons; ->
13          { prod=receive(messaggio) from dati;
14            contatore--;
15            send(messaggio) to cons.dati; }
16    } }
17
18 process produttore_i {
19     port signal ok_to_send;
20     T messaggio; process p; signal s;
21     ...
22     <produci il messaggio>;
23     send(s) to mailbox.pronto_prod;
24     p=receive (s) from ok_to_send;
25     send (messaggio) to mailbox.dati; }
26
27 process consumatore_j {
28     port T dati;
29     T messaggio; process p;
30     signal s;
31     ...
32     send(s) to mailbox.pronto_cons;
33     p=receive (messaggio) from dati;
34     <consumi il messaggio>; }

```

4.3.6 Uso delle primitive di comunicazione sincrone

Confrontando le primitive asincrone e sincrone si nota: un minore grado di concorrenza delle primitive sincrone rispetto alle asincrone; con le primitive sincrone non è più necessaria la presenza di buffer nei canali di comunicazione.



Rispetto al caso della send asincrona: il canale non è in grado di bufferizzare i messaggi inviati quindi c'è necessità di una struttura interna al processo mailbox: coda di messaggi.

Nel descrivere la soluzione, per semplicità, non viene dettagliata la realizzazione del tipo astratto coda-messaggi di cui, locale al processo mailbox, viene dichiarata l'istanza coda. In particolare, supporremo che su oggetti del tipo coda-messaggi si possa operare con le seguenti funzioni:

```

1 void inserimento(T mes); //per inserire mes nella coda
2 T estrazione(); //restituisce il primo elemento della coda
3 boolean piena(); // restituisce true se la coda e piena
4 booleana vuota(); // restituisce true se la coda e vuota
5
6 process mailbox{
7     port T dati;
8     port signal pronto_prod, pronto_cons;
9     T messaggio; process p; signal s;
10    coda_messaggi coda; //
11    <inizializzazione>;
12    do {
13        [] (!coda.piena()); p=receive(s)from pronto_prod;-> //
14        p=receive(messaggio)from dati;
15        coda.inserimento(messaggio);
16        [] (!coda.vuota()); p=receive(s)from pronto_cons;-> //
17        messaggio = coda.estraneazione;
18        send(messaggio)to p.dati;
19    } }
20
21 process produttore_i{
22     T messaggio; signal s;
23     ...

```

```

24     <produci il messaggio>;
25     send(s)to mailbox.pronto_prod; //.
26     send(messaggio)to mailbox.dati; //.
27     ...}
28
29 process consumatore_j{
30     port T dati; T messaggio;
31     process p; signal s; ...
32     send(s)to mailbox.pronto_cons; //.
33     p=receive(messaggio)from dati; //.
34     <consuma il messaggio>;
35     ... }

```

Mailbox di dimensioni finite

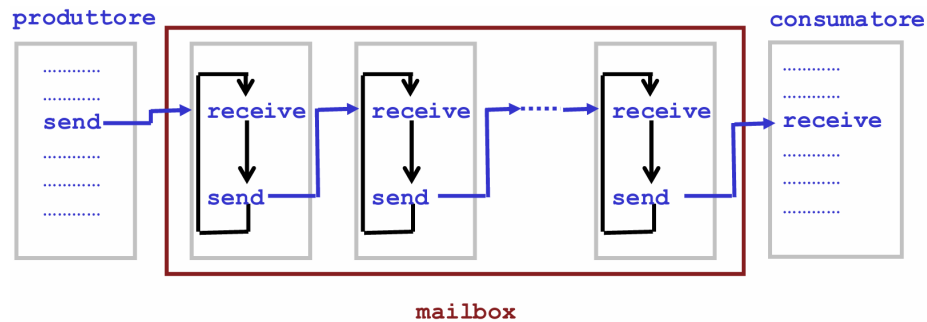
```

1  //ottimizzazione lato produttore
2  process mailbox{
3      port T dati;
4      port signal pronto_cons;
5      T messaggio; process p; signal s;
6      coda_messaggi coda;
7      <inizializzazione>;
8      do {
9          [] (!coda.piena()); p=receive(messaggio)from dati;-> //.
10         coda.inserimento(messaggio);
11         [] (! coda.vuota()); p=receive(s)from pronto_cons;-> //.
12         messaggio = coda.estrazione;
13         send(messaggio)to p.dati;                //.
14     }}
15
16 process produttore_i{
17     T messaggio;
18     process p;
19     signal s;
20     ...
21     <produci il messaggio>;
22     send(messaggio)to mailbox.dati; //.
23     ... }

```

Mailbox concorrente

Un buffer di dimensione N può essere implementato da un insieme di processi concorrenti collegati secondo uno schema a cascata (pipeline). Ogni processo ha una struttura ciclica e, sfruttando la semantica sincrona della send, può trattenere un singolo messaggio inserito, ma non ancora estratto.



```

1  //struttura del processo i-simo nella pipeline
2  process m_i{ //(0<=i<=N-2)
3      port T dati;
4      T buffer;
5      process p;
6      while (true){
7          p=receive(buffer) from dati;
8          send(buffer) to m_i+1.dati; }}
9
10 process server{ //pool con 1 sola risorsa
11     port signal richiesta;
12     port int rilascio;
13     boolean libera; process p; signal s;
14     int sospesi=0; boolean bloccato[M];
15     process client[M]
16     { //inizializzazione
17         libera=true;
18         for (int j=0; j<M; j++) bloccato[j]=false;
19         client[0]="P0";...client[M-1]="PM-1";
20     }
21     do {
22         [] p = receive(s)from richiesta;-> //.
23         if (libera) {
24             libera = false;
25             send (s) to p.risorsa;}
26         else {
27             sospesi++;
28             int j=0;
29             while(client[j]!=p) j++;
30             bloccato[j]=true;}
31         [] p = receive (s) from rilascio;-> //.
32         if (sospesi == 0) libera= true;
33         else {
34             int i = 0;
35             while (!bloccato[i]) i++;
36             sospesi --;
37             bloccato[i] = false;
38             send (s)to client[i].risorsa;}} //.

```

5 Comunicazione con sincronizzazione estesa

Modello di comunicazione/sincronizzazione in cui:

- il processo mittente richiede l'esecuzione di un servizio al processo destinatario
- il processo mittente rimane sospeso fino al completamento del servizio richiesto.

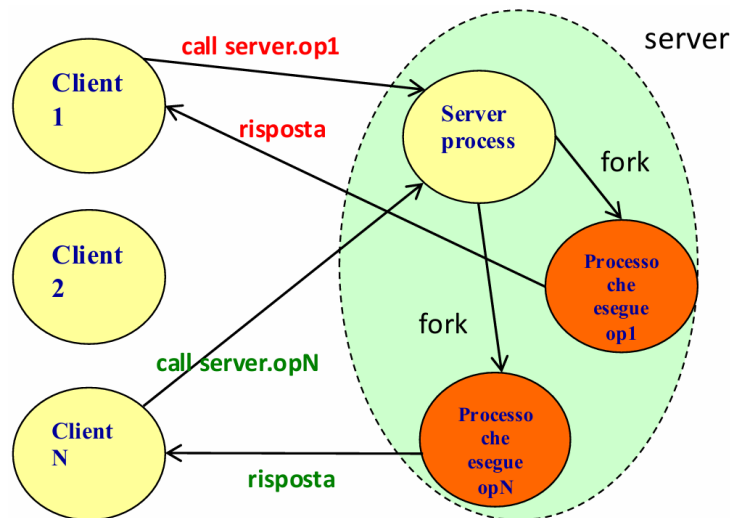
I processi rimangono sincronizzati durante l'esecuzione del servizio da parte del ricevente fino alla conclusione del servizio richiesto e all'eventuale ricezione dei risultati da parte del mittente. Il meccanismo è noto anche con il nome di chiamata di operazione remota. C'è una forte analogia semantica con una normale chiamata di funzione: il chiamante prosegue solo dopo che l'esecuzione della funzione è terminata. La differenza sostanziale sta nel fatto che nella sincronizzazione estesa la funzione (servizio) **viene eseguita remotamente** da un processo diverso dal chiamante.

Implementazione lato ricevente(server):

- **chiamata di procedura remota** (RPC -Remote Procedure Call)
- **rendez-vous esteso**.

5.0.1 Chiamata di procedura remota

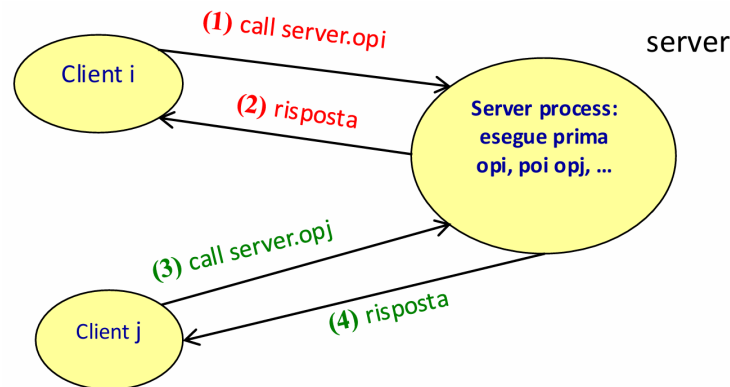
- Per ogni operazione che un processo client può richiedere viene dichiarata, lato server, una procedura;
- per ogni richiesta di operazione al server viene creato un nuovo processo (thread) che ha il compito di eseguire la procedura corrispondente.



5.0.2 Rendez-vous esteso

L'operazione richiesta viene specificata come un insieme di istruzioni che può comparire in un punto qualunque del processo servitore (es. linguaggio ADA) . Il processo servitore utilizza un'istruzione di input (accept) che lo sospende in attesa dell'arrivo di una richiesta dell'operazione.

All'arrivo della richiesta il processo esegue il relativo insieme di istruzioni ed i risultati ottenuti sono inviati al chiamante.



RPC vs Rendez-vous esteso

- RPC rappresenta solo un **meccanismo di comunicazione tra processi**: la possibilità che più operazioni siano eseguite concorrentemente implica la necessità di sincronizzazione tra i vari processi servitori. La sincronizzazione è a carico del programmatore. (ES: Java RMI, Distributed Processes)
- Rendez-vous esteso combina **comunicazione con sincronizzazione**. Esiste, infatti, un solo processo servitore al cui interno sono definite le istruzioni che consentono di realizzare il servizio richiesto. Il processo servitore si sincronizza con il processo cliente quando esegue l'operazione di accept (es: ADA).

RPC Chiamata di procedura remota

I singoli moduli operano in spazi di indirizzamento diversi e possono quindi essere allocati su nodi distinti di una rete.

Il server crea un thread che esegue l'operazione richiesta (op-i).

Ad ogni istante è possibile che **più thread concorrenti** all'interno del modulo server accedano a variabili interne.

->**Necessità di sincronizzazione**-> monitor, semafori, ...

```

1 module nome_del_modulo {
2     // server
3     <dichiarazione delle variabili locali>;
4     <inizializzazione delle variabili locali>;
5
6     public void op_1 (<parametri formali>){
7     <corpo della procedura op1>;}
8     ...
9     public void op_n (<parametri formali>){
10    <corpo della procedura op_n >;}
11
12    <dichiarazione di procedure locali>;
13    <dichiarazione di processi locali>; }
14
15    module client {
16        ...
17        call nome_del_modulo.op_i (<parametri attuali>;
18        ... }

```

Rendez-vous esteso

Il servizio richiesto viene specificato dal server come un insieme di istruzioni che può comparire in un punto qualunque del processo server tramite l'istruzione accept (v. linguaggio ADA, 1976):

```

1     accept <servizio>(in <par-ingresso>, out<par-uscita>)
2         -> {S1,...,Sn};

```

in: parametri di input; out: parametri di output (risultato del servizio)

Il client richiede il servizio con l'istruzione:

```

1     call <server>.<servizio>(<parametri attuali>;

```

Accept

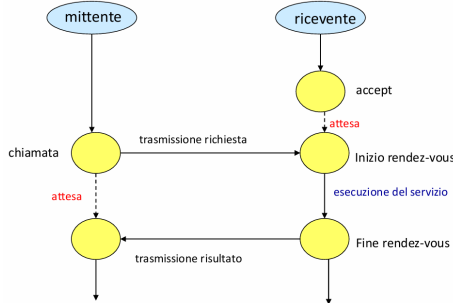
```

1     accept <S>(in <par-ingresso>, out<par-uscita>)->{S1,...,Sn};

```

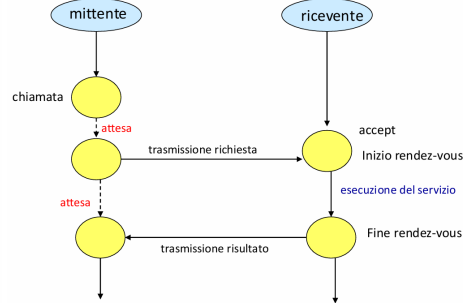
- Se non sono presenti richieste del servizio S l'accept è **sospensiva**.
- Se lo stesso servizio S è richiesto da più processi prima che il servitore esegua la accept, le richieste vengono inserite in una **coda associata al servizio**, gestita con politica FIFO.
- Ad uno stesso servizio possono essere associate **più accept** nel codice eseguito dal server -> ad una richiesta possono corrispondere azioni diverse.
- Lo schema di comunicazione realizzato dal meccanismo di rendez vous è di tipo **asimmetrico da molti a uno**.

Possibili sequenze di eventi in una comunicazione di tipo rendez-vous



(a)

Possibili sequenze di eventi in una comunicazione di tipo rendez-vous



(b)

Accept: selezione delle richieste Nel modello rendez-vous, il server può selezionare le richieste da servire in base al suo stato interno (es. lo stato delle risorse gestite), utilizzando i comandi con guardia:

```

1 select
2   [] <cond1>; accept <servizio1>(in <par-ingresso>,
3     out <par-uscita>) -> {S11,...,S1n}; ...
4   [] <cond2>; accept <servizio2>(in <par-ingresso>,
5     out <par-uscita>) -> {S21,...,S2n}; ...
6   ...
7 end; //cond1 e cond2 sono condizioni logiche che dipendono
      ↪ dallo stato interno del server.

```

Rendez-vous esteso: Selezione delle richieste in base ai parametri di ingresso

La decisione se servire o no una richiesta può dipendere, oltre che dallo **stato della risorsa**, anche dai parametri della richiesta stessa (esempio: politiche di accettazione delle richieste basate su priorità). In questo caso, infatti, condizione per l'esecuzione dell'azione richiesta deve essere espressa anche in termini dei parametri di ingresso.

Ciò non è possibile tramite la guardia di ingresso: è necessaria una **doppia interazione** tra processo cliente e processo servitore; la prima per trasmettere i parametri della richiesta e la seconda per richiedere il servizio.

5.1 Vettore di operazioni di servizio

Nell'ipotesi di un **dominio limitato del parametro delle richieste** per un dato servizio S si può ottenere una semplice soluzione al problema associando ad ogni possibile valore del parametro della richiesta una differente operazione (operazione di servizio).

Ciò è reso possibile in alcuni linguaggi dalla possibilità di aggregare le richieste possibili in strutture di tipo vettore: **vettore delle operazioni di servizio**.

6 Algoritmi di Sincronizzazione Distribuiti

Il modello a scambio di messaggi è la naturale astrazione di un sistema distribuito, nel quale processi distinti eseguono su nodi fisicamente separati, collegati tra di loro attraverso una rete.

Caratteristiche dei sistemi distribuiti:

- concorrenza/parallelismo delle attività nei nodi;
- assenza di risorse condivise tra nodi;
- assenza di un clock globale;
- possibilità di malfunzionamenti indipendenti:
 - nei nodi (crash/attacchi/...);
 - nella rete di comunicazione (ritardi, perdite di messaggi, ecc..).

Nelle applicazioni distribuite è importante poter soddisfare alcune proprietà:

- **scalabilità:** nell'applicazione distribuita le prestazioni dovrebbero aumentare al crescere del numero di nodi utilizzati;
- **tolleranza ai guasti:** l'applicazione è in grado di funzionare anche in presenza di guasti.

Tolleranza ai guasti

Il sistema distribuito tollerante ai guasti riesce a erogare i propri servizi anche in presenza di guasti in uno o più nodi.

Tipi di guasto:

- Transienti; Intermittenti; Persistenti.

Un sistema tollerante ai guasti deve nascondere i guasti agli altri processi. Questo obiettivo può essere raggiunto, ad esempio, con tecniche di **ridondanza**: vengono mantenute più istanze degli stessi componenti, in modo da poter rimpiazzare l'elemento guasto con un elemento equivalente.

Algoritmi di Sincronizzazione

Come nel modello a memoria comune, anche nel modello a scambio di messaggi è importante poter disporre di algoritmi di sincronizzazione tra i processi concorrenti, che consentano di risolvere alcune problematiche comuni, coordinando i vari processi.

È importante che gli algoritmi distribuiti godano delle proprietà di scalabilità e di tolleranza ai guasti.

6.1 Algoritmi per la gestione del tempo

Tempo nei sistemi distribuiti

In un sistema distribuito, ogni nodo è dotato di un proprio orologio. Se gli orologi locali di due nodi non solo sono sincronizzati, è possibile che se un evento e_2 accade nel nodo N_2 dopo un altro evento e_1 nel nodo N_1 , ad e_2 sia associato un istante temporale precedente quello di e_1 .

Quindi in applicazioni distribuite può essere necessario:

- disporre di un orologio fisico universale, se c'è bisogno di utilizzare l'ora esatta: a questo scopo vengono utilizzati degli algoritmi di sincronizzazione dei nodi del sistema (es. **algoritmo di Berkeley**, Christian)
- disporre di un orologio logico, che permetta di associare ad ogni evento un istante logico (timestamp) coerente con l'ordine in cui essi si verificano. (es. **Lamport**)

6.1.1 Orologi Fisici (Berkeley)

Assumiamo che la comunicazione tra ogni coppia di nodi A e B è affetta da un ritardo di comunicazione, che è lo stesso in entrambi i versi di comunicazione ($A \rightarrow B$, $B \rightarrow A$); inoltre approccio centralizzato: per determinare l'ora universale, tutti i nodi interagiscono con un nodo Coordinatore.

1. Il master interagisce con ogni nodo N_i nel modo seguente:

T1) invia una richiesta R_i al nodo N_i .

T2) il nodo N_i risponde con la propria ora locale T_i .

T3) Il master riceve l'ora T_i da N_i e misura il Round Trip Time (RTT_i): **$RTT = T_3 - T_1$**

→ il ritardo stimato in ogni comunicazione master $\leftrightarrow N_i$ è **$d = RTT_i / 2$** .

– Calcola l'ora effettiva del nodo N_i : $T_i + d$.

– Determina la differenza $\Delta_i = (T_i + d) - T_3$.

2. Una volta calcolati tutti i Δ_i , il master calcola la media:

$$\Delta_{medio} = \frac{\sum_{i=0}^n \Delta_i}{n + 1}$$

3. Infine il master invia ad ogni nodo N_i il valore della correzione C_i dell'orario locale:

$$C_i = \Delta_{medio} - \Delta_i$$

4. Ogni nodo aggiornerà il proprio orologio locale all'orario: $T'_i = T_i + C_i$

I vantaggi di questo algoritmo sono:

Nessuna dipendenza da un singolo orologio (diversamente da altri approcci).

Sincronizzazione cooperativa e robusta: il fallimento di un nodo N_i non compromette l'efficacia dell'algoritmo. Di contro si assume che la latenza sia sim-

metrica; ed è meno accurato in reti con ritardi variabili.

6.1.2 Orologi Logici (Lamport, 1979)

In un'applicazione distribuita, gli eventi sono legati da vincoli di precedenza che danno origine ad una relazione d'ordine parziale (v. algoritmi non sequenziali)

Relazione di precedenza tra eventi (happened-before, $\rightarrow$):

- se a e b sono eventi in uno stesso processo ed a si verifica prima di b : $a \rightarrow b$
- se a è l'evento di invio di un messaggio e b è l'evento di ricezione dello stesso messaggio: $a \rightarrow b$
- se $a \rightarrow b$ e $b \rightarrow c$ allora $a \rightarrow c$

Data una coppia di eventi (a, b) sono possibili 3 casi:

- $a \rightarrow b$, cioè a avviene prima di b ;
- $b \rightarrow a$, b avviene prima di a ;
- a e b non sono legati dalla relazione **HB** $\rightarrow$ a e b sono concorrenti.

Assumiamo che ad ogni evento e venga associato un timestamp $C(e)$.

Si vuole definire un modo per misurare il concetto di tempo tale per cui ad ogni evento a possiamo definire un timestamp $C(a)$ sul quale tutti i processi siano d'accordo.

Il tempo $C(a)$ deve soddisfare la seguente proprietà:

$$\text{se } a \rightarrow b \text{ allora } C(a) < C(b) \quad [*]$$

Quindi:

- se all'interno di un processo a precede b allora $C(a) < C(b)$
- se a è l'evento di **invio** (in un processo P_s) e b l'evento di **ricezione** (in un processo P_r) dello stesso messaggio m allora $C(a) < C(b)$

6.1.3 Algoritmo di Lamport

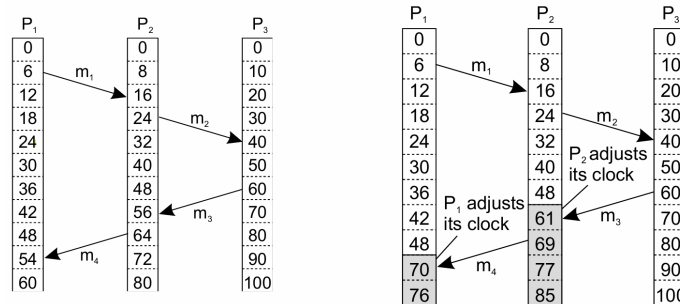
Per garantire il rispetto della proprietà $[*]$ ogni processo P_i gestisce localmente un contatore del tempo logico C_i , che viene gestito nel modo seguente:

- ogni nuovo evento all'interno di P_i provoca un incremento del valore di C_i :
 $C_i = C_i + 1$
- ogni volta che P_i invia un messaggio m , il contatore C_i viene incrementato:
 $C_i = C_i + 1$ e successivamente al messaggio viene associato il timestamp C_i :
 $ts(m) = C_i$

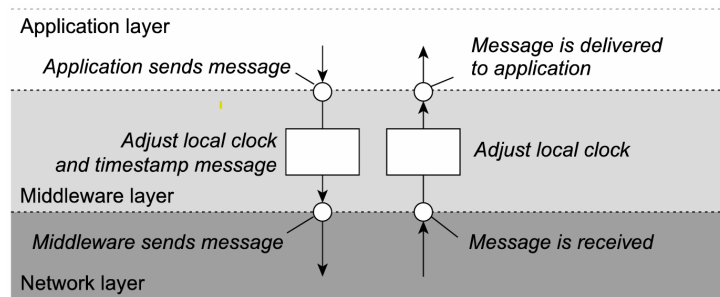
- quando un processo P_j riceve un messaggio m , assegna al proprio contatore C_j un valore uguale al massimo tra C_j e $ts(m)$: $C_j = \max\{C_j, ts(m)\}$ e successivamente lo incrementa di 1: $C_j = C_j + 1$

Orologio logico di Lamport: esempio

Considerando 3 processi in esecuzione su 3 nodi, ognuno con il proprio clock:



Nei sistemi distribuiti l'algoritmo di Lamport viene generalmente eseguito da uno strato software (middleware) che interfaccia i processi alla rete: i processi vedono il tempo logico.



6.2 Mutua Esclusione Distribuita

L'obiettivo della mutua esclusione distribuita è quello di garantire che due o più processi non possano eseguire contemporaneamente certe attività. Ad esempio accesso a risorse condivise come file, stampanti, ecc..

Esistono 2 principali soluzioni:

- **centralizzata:** la risorsa è gestita da un processo dedicato (coordinatore), al quale tutti i processi si rivolgono per accedervi;
- **decentralizzata:** non è previsto un coordinatore, quindi i processi in competizione si sincronizzano tra loro tramite opportuni algoritmi, la cui logica è distribuita tra tutti i processi.

Una soluzione decentralizzata è, in generale, più scalabile di soluzioni centralizzate, nelle quali il processo gestore della risorsa rappresenta un collo di bottiglia.

Soluzioni al problema della mutua esclusione distribuito

In generale, possiamo suddividere le soluzioni in due categorie:

- **Permission-based:** ogni processo che vuole eseguire la sua sezione critica, richiede un permesso ad uno o più processi;
- **Token-based:** un "testimone" (token) viene passato tra i vari processi in competizione: il processo che possiede il token può:
 - eseguire la sua sezione critica;
 - passare il token ad un altro processo, se non intenzionato ad entrare nella sezione critica.

Gli algoritmi token-based sono sempre decentralizzati, mentre gli algoritmi permission-based possono essere sia centralizzati che decentralizzati.

6.2.1 Mutua esclusione: soluzione centralizzata

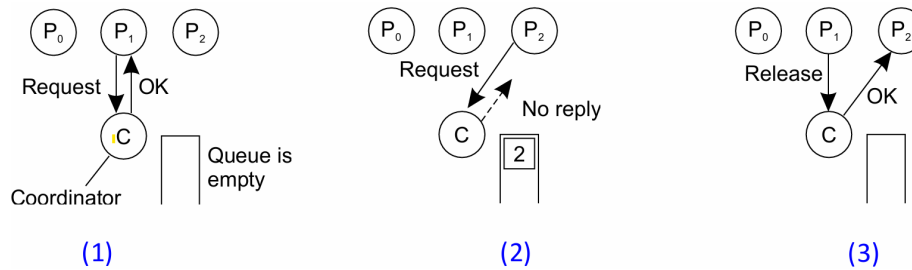
L'algoritmo si basa su **permessi** (permission-based):

- la risorsa viene gestita da un processo coordinatore al quale ogni processo che vuole eseguire la sua sezione critica si rivolge per ottenere il permesso.
- per eseguire la propria sezione critica ogni processo P_i :
 - **Richiesta:** P_i invia una richiesta di autorizzazione al coordinatore; quando verrà accolta, il processo P_i otterrà il permesso.
 - viene eseguita la sezione critica
 - **Rilascio:** P_i comunica al coordinatore il termine della sezione critica

Il coordinatore **concede un permesso alla volta:** ogni Richiesta ricevuta da un processo P_i mentre l'autorizzazione è concessa al processo P_j viene messa in attesa. Inoltre il coordinatore mantiene una coda delle Richieste in attesa.

Esempio:

- P_1 chiede il permesso al coordinatore, che autorizza l'accesso;
- P_2 chiede il permesso al coordinatore, che non può concederle perché P_1 sta eseguendo la sua sezione critica; il coordinatore non risponde e inserisce la richiesta di P_2 nella coda;
- quando P_1 termina la sua sezione critica, invia un messaggio di Rilascio al coordinatore, che quindi può successivamente autorizzare P_2 .



Vantaggi:

L'algoritmo è equo, e quindi non c'è starvation. Inoltre prevede solo 3 messaggi per ogni sezione critica: richiesta, autorizzazione e rilascio.

Svantaggi:

Scalabilità: la soluzione è poco scalabile perché al crescere del numero dei processi il coordinatore può diventare un collo di bottiglia.

Tolleranza ai guasti:

- il coordinatore può essere soggetto a guasto e, in questo caso, l'intero sistema si blocca (single point of failure)
- inoltre, se un processo P che ha fatto una richiesta non ottiene una risposta, P non può distinguere le due possibili cause:
 - autorizzazione non concessa;
 - coordinatore guasto

6.2.2 Mutua esclusione: Algoritmo Ricart-Agrawala

Algoritmo decentralizzato basato su permessi (permission-based). Il sistema è costituito da un insieme di processi in competizione. Ad ogni processo sono associate 2 attività concorrenti (thread):

- **main:** il thread che esegue la sezione critica
- **receiver:** thread dedicato alla ricezione delle autorizzazioni

Questo algoritmo ha un requisito. Ovvero è richiesta la presenza di un temporizzatore globale per tutti i nodi (orologio logico): ogni messaggio è corredato da un timestamp che ne indica l'istante di invio.

Struttura del main:

Quando un processo vuole entrare nella sezione critica:

1. invia (n-1) richieste di autorizzazione ai receiver degli altri (n-1) nodi:
Request(Pid, timestamp);
2. attende le (n-1) autorizzazioni;

3. esegue la **sezione critica**;
4. invia **ok** a tutte le richieste in attesa.

Struttura del receiver:

Quando riceve una richiesta, il receiver può trovarsi in uno di tre possibili stati:

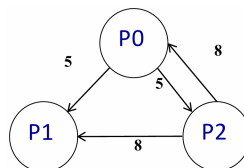
1. **Released:** il processo non è interessato ad entrare nella sezione critica: risponde OK
2. **Wanted:** il processo è in procinto di entrare nella sezione critica (attende autorizzazione): confronta il timestamp T_r della richiesta ricevuta con quello della richiesta inviata T_s :
 - se $T_r < T_s$ risponde OK
 - altrimenti non risponde e mette la richiesta ricevuta in coda
3. **Help:** il processo sta eseguendo la sezione critica: la richiesta viene messa in coda.

Esempio:

Consideriamo 3 processi: P0, P1, P2.

Se P0 e P2 vogliono eseguire una sezione critica, cioè sono entrambi nello stato W:

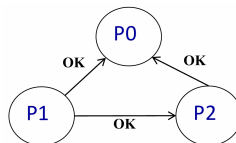
- P0 invia a P1 e P2 una Richiesta (timestamp=5)
- P2 invia a P0 e P1 una Richiesta (timestamp=8)



P0 è in stato WANTED: non risponde a P2 perchè $5 < 8$ e mette in coda la sua richiesta.

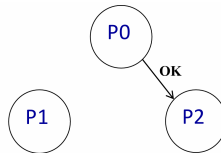
P1 è in stato RELEASED: risponde ad entrambi OK.

P2 è in stato WANTED: risponde OK a P0 perchè $5 < 8$.



P0 esegue la sezione critica.

Terminata la sezione critica, P0 invia OK a P2, la cui richiesta era stata accodata



P2 esegue la sezione critica.

Vantaggi e svantaggi dell'algoritmo Ricart-Agrawala

Vantaggi: distribuzione ed una seguente scalabilità.

Svantaggi: maggior costo di comunicazione per il singolo partecipante, $2*(N-1)$ messaggi per ogni sezione critica. Inoltre non c'è più un single point of failure, ma N points of failure: se uno dei nodi va in crash, non risponderà alle richieste degli altri. In più è impossibile rilevare il guasto, quindi impossibile sapere se l'attesa è dovuta ad un guasto o a sezioni critiche in esecuzione.

Guasti nell'algoritmo Ricart-Agrawala

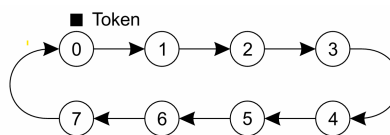
Per la rilevazione di guasti si può modificare il protocollo prevedendo che ad ogni richiesta venga sempre fornita una risposta:

- OK
- Attendi (autorizzazione negata): in questo caso il richiedente si mette in attesa di un OK

Il richiedente può impostare un timeout per rilevare guasti del destinatario. Invece per gli N points of failure, ogni processo ha un ruolo equivalente agli altri; se è guasto può essere escluso dal gruppo.

6.2.3 Mutua Esclusione: Algoritmo Token ring

Algoritmo distribuito basato su token. Il sistema è costituito da un insieme di processi in competizione, collegati tra loro secondo una topologia logica ad anello. Ogni processo conosce i suoi vicini nell'anello. Un messaggio (detto token), circola attraverso l'anello, nel verso relativo all'ordine dei processi nella topologia.



Il token rappresenta il permesso unico di eseguire sezioni critiche: chi detiene il token può eseguire la sua sezione critica.

Il token viene inizialmente inviato da P0 a P1:

- P1 può essere nello stato WANTED: in questo caso trattiene il token ed esegue la sezione critica; al termine passa il token al processo successivo P2;
- P1 può essere nello stato RELEASED: in questo caso passa il token al processo successivo P2.

P2 si comporta allo stesso modo di P1, come tutti i processi successivi nell'anello. In questo modo il token circola nell'anello nel verso stabilito dall'ordine dei processi.

Vantaggi: distribuzione ed una seguente scalabilità.

Svantaggi: il numero di messaggi per ogni sezione critica non è limitato. Non c'è più un single point of failure ma N points of failure: se uno dei nodi va in crash interrompe l'intera catena. Per far fronte a questo problema si può modificare il protocollo prevedendo che ad ogni invio del token (da P_i a P_{i+1}) venga sempre restituita una risposta, se la risposta non arriva (allo scadere di un timeout) P_{i+1} viene escluso dall'anello ed il token viene inviato a P_{i+2} . Un ultimo problema è che c'è la possibilità di perdere il token, se ad esempio il processo che detiene il token va in crash il token va perduto.

6.2.4 Confronto tra i 3 algoritmi di mutua esclusione

Algoritmo	Num messaggi per sezione critica	Ritardo in entrata nella sezione critica	Problemi
Centralizzato	3	2	scarsa scalabilità e crash coordinatore
Ricart-Agrawala	$2*(n-1)$	$2*(n-1)$	crash qualunque processo
Token Ring	$1..∞$	$0..n-1$	crash qualunque processo e perdita token

6.2.5 Algoritmi di Elezione

In alcuni algoritmi è previsto che un processo rivesta un ruolo speciale nella sincronizzazione tra tutti i nodi. Tale processo viene detto **coordinatore**. La designazione del coordinatore può essere:

- statica: il coordinatore viene deciso in modo arbitrario dal programmatore/amministratore prima dell'esecuzione;
- dinamica: il coordinatore viene designato a runtime dai processi del sistema con un **algoritmo di elezione**.

Ad esempio se il coordinatore di un gruppo di processi subisce un crash, per ripristinare l'operatività del sistema, è necessario individuare a runtime un altro processo del gruppo a cui attribuire il ruolo di nuovo coordinatore.

Assunzioni di base:

- ogni processo è identificato da un unico ID. Il dominio degli id è totalmente ordinario.
- ogni processo conosce gli ID di tutti gli altri, ma non il loro stato e quindi non se sono attivi o terminati.

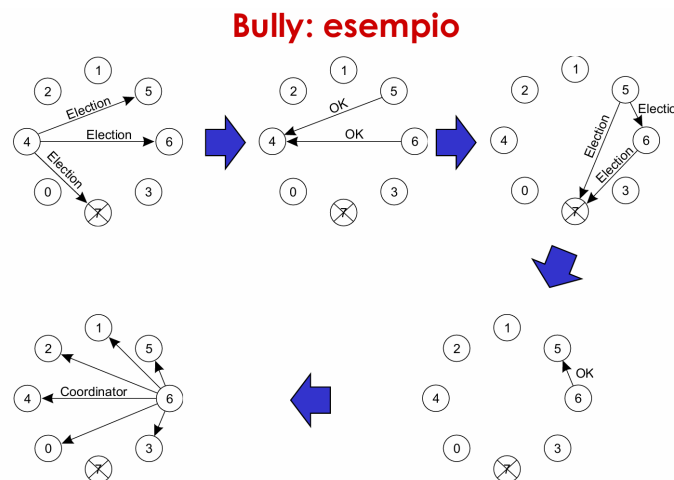
L'obiettivo è quello di designare vincitore il processo attivo che ha l'ID più alto.

Algoritmo Bully

Consideriamo un sistema composto da N processi $\{P_0, \dots, P_{N-1}\}$; sia $\text{id}(P_i) = i$. Quando un processo P_k rileva che il coordinatore non è più attivo, organizza un'elezione:

1. P_k invia un messaggio "ELEZIONE" a tutti i processi con ID più alto: $P_{k+1}, P_{k+2}, \dots, P_{N-1}$
2. Se nessun processo risponde, P_k vince l'elezione e diventa il nuovo coordinatore; comunica quindi a tutti gli altri processi il nuovo ruolo tramite il messaggio "COORDINATORE"
3. Se un processo P_j ($j > k$) risponde, P_j prende il controllo e P_k rinuncia ed esce dall'elezione

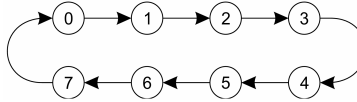
Ogni processo attivo risponde ad ogni messaggio di ELEZIONE ricevuto.



Algoritmo di elezione ad Anello

I processi del gruppo sono collegati tramite una topologia logica ad anello. La posizione (ID) che ogni processo occupa all'interno dell'anello rappresenta la sua priorità.

Il processo attivo con la massima priorità viene eletto **coordinatore**.

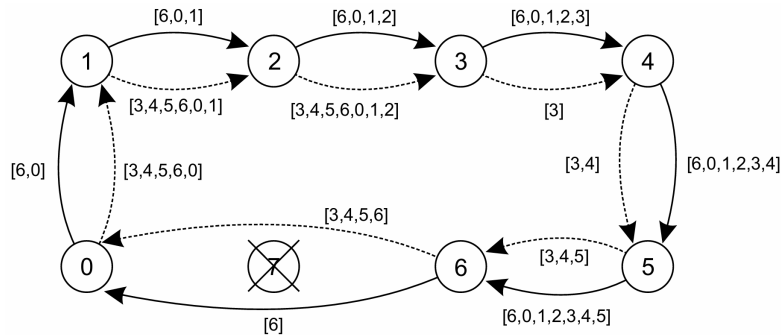


Quando un qualunque processo P_i si rende conto che il coordinatore non risponde più, inizia un'elezione:

1. P_i invia un messaggio ELEZIONE contenente il suo ID al suo successore P_{i+1} . Se il successore P_{i+1} è in crash, il messaggio viene spedito al successore di P_i , ecc
2. Quando un processo P_j riceve un messaggio ELEZIONE:
 - se il messaggio non contiene l'ID di P_j , aggiunge il suo ID al messaggio e lo spedisce al successivo, ecc
 - se il messaggio contiene l'ID di P_j , significa che è stato compiuto un giro completo dell'anello: P_j designa come coordinatore il processo corrispondente all'ID più alto nel messaggio ricevuto e invia al successivo processo un messaggio COORDINATOR contenente l'ID del processo designato come nuovo coordinatore
3. Quando un processo riceve un messaggio COORDINATOR, ne prende atto e inoltra lo stesso messaggio al successivo.

Esempio

P_3 e P_6 avviano entrambi un'elezione: entrambe le elezioni designeranno P_6 come nuovo coordinatore. Ogni messaggio COORDINATOR dopo aver percorso l'anello verrà eliminato.



7 Virtualizzazione

Virtualizzare un sistema (hardware e software) significa presentare all'utilizzatore una visione delle risorse del sistema diversa da quella reale. Ciò è possibile introducendo un **livello di indirizzione** tra la vista logica e quella fisica delle risorse.

Quindi l'obiettivo della virtualizzazione è quello di **disaccoppiare** il comportamento delle risorse di un calcolatore dalla loro realizzazione fisica. Quindi apparendo diverse da quelle effettive della macchina. Il software che si occupa di virtualizzare in parole semplici divide le risorse reali nel numero di macchine virtuali necessarie. Quindi ogni macchina virtuale avrà la sua CPU, GPU, RAM, ecc...

Esempi di virtualizzazione:

- **Virtualizzazione a livello di processo:** i sistemi multitasking permettono l'esecuzione contemporanea di più processi, ognuno dei quale dispone di una macchina virtuale dedicata. Questo tipo di virtualizzazione viene realizzata dal kernel del sistema operativo.
- **Virtualizzazione della memoria:** in presenza di memoria virtuale, ogni processo vede uno spazio di indirizzamento di dimensioni indipendenti dallo spazio fisico effettivamente a disposizione. Anche questa virtualizzazione è realizzata dal kernel.
- **Astrazione:** un oggetto astratto (risorsa virtuale) è la rappresentazione semplificata di un oggetto (risorsa fisica), quindi esibendo le proprietà significative per l'utilizzatore e nascondendo i dettagli realizzativi non importanti
- **Linguaggi di Programmazione.** La capacità di portare lo stesso programma (scritto in un linguaggio di alto livello) su architetture diverse è possibile grazie alla definizione di una macchina virtuale in grado di interpretare ed eseguire ogni istruzione del linguaggio, indipendentemente dall'architettura del sistema (S.O. e HW) come fanno i compilatori o gli Interpreti (JVM).

7.1 Virtualizzazione di un Sistema di Elaborazione

Tramite la virtualizzazione una singola piattaforma hardware viene condivisa da più elaboratori virtuali, ognuno gestito da un proprio sistema operativo. Il disaccoppiamento viene realizzato dal **Virtual Machine Monitor (VMM)**, il cui compito è quello di consentire la condivisione da parte di più macchine virtuali di una singola piattaforma hardware.

Quindi il **VMM** è il **mediatore unico** nelle interazioni tra le macchine virtuali e l'hardware, il quale garantisce: **isolamento tra le VM** e **stabilità del sistema**.

7.2 Tecniche del VMM

7.2.1 Emulazione

L'emulazione è l'insieme di tutti quei meccanismi che permettono l'esecuzione di un programma compilato su un determinato sistema di girare su un qualsiasi altro sistema differente da quello nel quale è stato compilato. Quindi vengono emulate interamente le singole istruzioni dell'architettura ospitata.

I vantaggi dell'emulazione sono l'interoperabilità tra ambienti eterogenei, mentre gli svantaggi sono le ripercussioni sulle performances.

Esistono principalmente due tecniche di emulazione: **interpretazione** e **ricompilazione dinamica**.

Interpretazione:

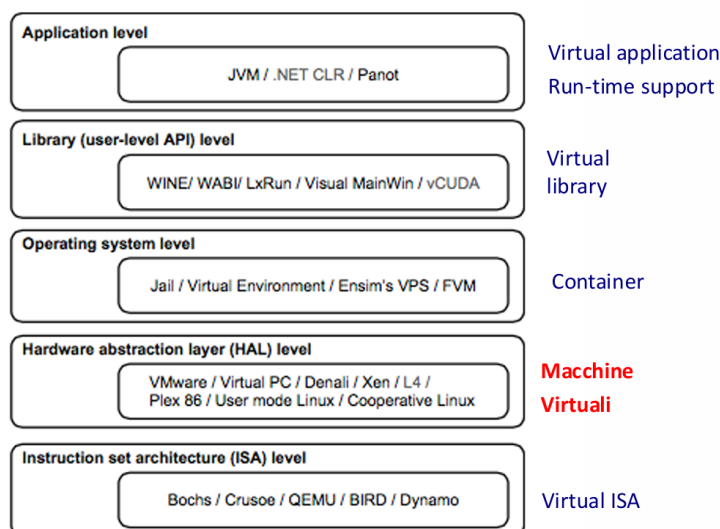
L'interpretazione si basa sulla lettura di ogni singola istruzione del codice macchina che deve essere eseguito e sulla esecuzione di più istruzioni sull'host virtualizzante. Produce un sovraccarico elevato in quanto potrebbero essere necessarie molte istruzioni dell'host per interpretare una singola istruzione sorgente.

Compilazione dinamica:

Invece di leggere una singola istruzione del sistema ospitato, legge interi blocchi di codice, vengono analizzati, tradotti per la nuova architettura, ottimizzati e messi in esecuzione. Il vantaggio in termini prestazionali rispetto all'interpretazione è notevolmente maggiore.

Ad esempio parti di codice utilizzati frequentemente vengono bufferizzati nella cache per evitare di doverli ricompilare in seguito.

Tipi (livelli) di virtualizzazione*

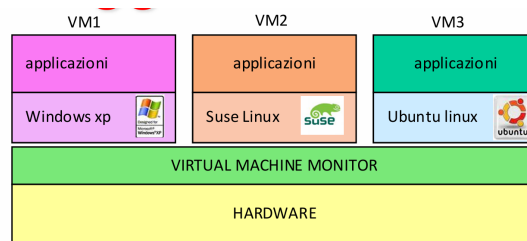


7.2.2 Macchine virtuali

Una singola piattaforma hardware viene condivisa da più elaboratori virtuali (macchine virtuali o VM) ognuno gestito da un proprio sistema operativo. Il disaccoppiamento è realizzato da un componente chiamato Virtual Machine Monitor (VMM, o hypervisor) il cui compito è consentire la **condivisione da parte di più macchine virtuali di una singola piattaforma hardware**. Ogni VM contiene un proprio sistema operativo, che definisce un ambiente di esecuzione distinto e isolato dalle altre macchine virtuali, che consente l'esecuzione di applicazioni all'interno di esso.

Il VMM è il **mediatore unico**, garantisce **isolamento** tra le VM e **stabilità** del sistema

Vantaggi della virtualizzazione:



La virtualizzazione consente l'utilizzo di **più sistemi operativi sulla stessa macchina fisica**, permettendo a un singolo utente di accedere a diversi ambienti di esecuzione, anche eterogenei. Questo approccio è utile per mantenere sistemi legacy e per eseguire applicazioni progettate per specifici SO.

Ogni macchina virtuale opera in un ambiente isolato, una sorta di sandbox, che garantisce la separazione dagli altri ambienti. Questo isolamento è fondamentale per effettuare test di applicazioni senza compromettere l'integrità degli altri ambienti o del Virtual Machine Monitor (VMM). Inoltre, migliora la sicurezza, poiché eventuali attacchi da parte di malware o spyware restano confinati alla singola macchina virtuale.

La virtualizzazione permette il **consolidamento dell'hardware**, ovvero la concentrazione di più macchine, come server, su un'unica architettura fisica. Questo porta a un utilizzo più efficiente delle risorse hardware, con conseguente riduzione dei costi sia per HW, sia per la sua amministrazione.

La **gestione delle macchine virtuali** risulta **semplificata**. È possibile

- **creare** facilmente nuove macchine virtuali,
- **amministrarle** con operazioni come il riavvio o la ricompilazione del kernel,
- effettuare la **migrazione a caldo** tra macchine fisiche. Quest'ultima funzionalità consente la **manutenzione** dell'hardware senza interrompere i servizi, facilita il **bilanciamento del carico computazionale** grazie a meccanismi di migrazione automatica

- contribuisce alla strategia di **disaster recovery**.

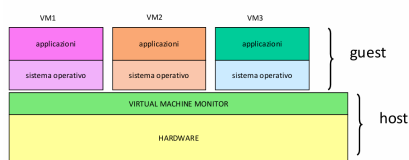
7.3 Realizzazione del VMM

Requisiti di Popek e Goldberg del 1974:

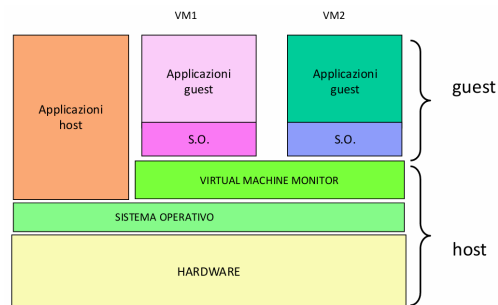
- **Ambiente di esecuzione per i programmi sostanzialmente identico a quello della macchina reale:** Gli stessi programmi che eseguono nel sistema non virtualizzato possono essere eseguiti nelle VM senza modifiche e problemi.
- **Garantire un'elevata efficienza nell'esecuzione dei programmi:** Il VMM deve permettere l'esecuzione diretta delle istruzioni impartite dalle macchine virtuali, quindi le istruzioni non privilegiate vengono eseguite direttamente in hardware senza coinvolgere il VMM
- **Garantire la stabilità e la sicurezza dell'intero sistema:** Il VMM deve sempre rimanere sempre nel pieno controllo delle risorse hardware, e i programmi in esecuzione nelle macchine virtuali non possono accedere all'hardware in modo privilegiato

Parametri e classificazione

- **Livello** nel quale è collocato il VMM:
 - **VMM di sistema:** eseguono direttamente sopra l'hardware del elaboratore (vmware, esx, xen, kvm)
 - **VMM ospitati:** eseguiti come applicazioni sopra un S.O. esistente (parallels, virtualbox)
- **Modalità di dialogo:** per l'accesso alle risorse fisiche tra le macchine virtuali ed il VMM:
 - **Virtualizzazione pura** (vmware): le macchine virtuali usano la stessa interfaccia dell'architettura fisica
 - **Paravirtualizzazione** (xen): il VMM presenta un'interfaccia diversa da quella dell'architettura HW



VMM di Sistema



VMM ospitato

7.3.1 Ring di protezione

La CPU prevede due livelli di protezione: **supervisore o kernel (0)** e **utente (<0)**.

Ogni ring corrisponde a una diversa modalità di funzionamento del processore:

- a livello 0 vengono eseguite le istruzioni privilegiate della CPU
- nei ring di livello superiore a 0 le istruzioni privilegiate non vengono eseguite

Alcuni programmi sono progettati per eseguire nel ring 0, ad esempio il Kernel del S.O. infatti è l'unico componente che ha pieno controllo dell'hardware.

VMM (vmm di sistema)

In un sistema virtualizzato il VMM deve essere l'unica componente in grado di mantenere il controllo completo dell'hardware. Infatti solo il VMM opera nello stato supervisore, mentre il S.O. e le applicazioni eseguono in un ring di livello superiore.

Sorgono però due problemi:

- **Ring deprivileging:** il s.o. della macchina virtuale esegue in un ring che non gli è proprio
- **Ring compression:** se i ring utilizzati sono solo 2, applicazioni e s.o. della macchina virtuale eseguono allo stesso livello: scarsa protezione tra spazio del s.o. e delle applicazioni.

7.3.2 Ring Deprivileging

Con Ring Deprivileging si indica una situazione nel quale l'esecuzione di istruzioni privilegiate richieste dal sistema operativo nell'ambiente guest non possono essere eseguite in quanto richiederebbero un ring 0, ma il kernel della macchina virtuale esegue in un ring di livello superiore.

Una possibile prima soluzione è il **Trap & Emulate**: nel quale se il guest tenta di eseguire un'istruzione privilegiata

- la CPU notifica un'eccezione al VMM (**trap**) e gli trasferisce il controllo
- il VMM controlla la correttezza dell'operazione richiesta e ne emula il comportamento (**emulate**)

Quindi in poche parole la CPU notifica e delega al VMM il controllo e l'esecuzione dell'istruzione privilegiata.

Esempio: Il guest tenta di disabilitare le interruzioni (**popf**), se la richiesta della macchina virtuale fosse eseguita direttamente sulla CPU sarebbero disabilitati tutti gli interrupt di sistema e quindi il VMM non potrebbe riottenere il controllo. Invece, con Trap&Emulate riceve la notifica di tale richiesta e ne emula

il comportamento sospendendo gli interrupt solamente per la macchina virtuale richiedente.

Supporto HW alla virtualizzazione

L'architettura della CPU si dice **naturalmente virtualizzabile** se e solo se prevede l'invio di trap al VMM per ogni istruzione privilegiata invocata da un livello di protezione differente dal quello del VMM.

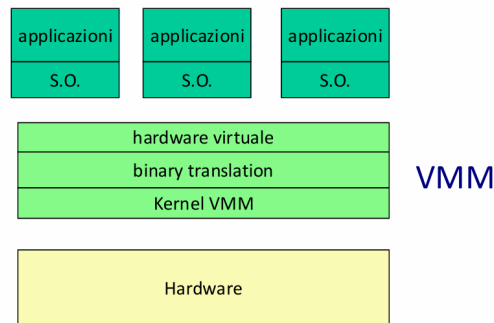
Se la CPU è naturalmente virtualizzabile viene implementato il trap & emulate; un ulteriore problema è il **Ring Aliasing**: Alcune istruzioni non privilegiate, eseguite in modo user, permettono di accedere in lettura ad alcuni registri la cui gestione dovrebbe essere riservata al VMM e ciò può creare inconsistenza.

Se il processore non fornisce alcun supporto alla virtualizzazione vi sono 2 possibilità: **Fast Binary Translation** e **Paravirtualizzazione**.

7.3.3 Fast Binary Translation

Il VMM scansiona dinamicamente il codice dei sistemi operativi guest prima dell'esecuzione per sostituire a run time i blocchi contenenti istruzioni privilegiate in blocchi equivalenti dal punto di vista funzionale e contenenti chiamate al VMM. Inoltre i blocchi tradotti sono eseguiti e conservati in cache per eventuali riusi futuri.

Il principale limite della Fast Binary Translation è che la traduzione dinamica è molto costosa. Però, con questa tecnica, ogni macchina virtuale è una esatta copia della macchina fisica, con la possibilità di installare gli stessi s.o. di architetture non virtualizzate.



7.3.4 Paravirtualizzazione

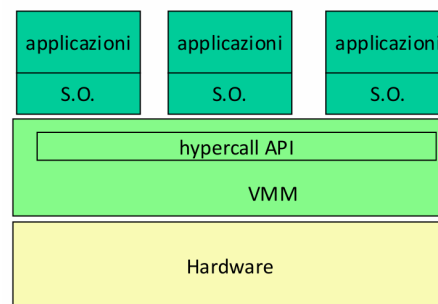
Il VMM (hypervisor) offre al sistema operativo guest un'interfaccia virtuale (ovviamente differente da quello hardware del processore) chiamata **hypercall API** alla quale i s.o. guest devono riferirsi per avere accesso alle risorse (system call).

Queste Hypercall API permettono di:

- richiedere l'esecuzione di istruzioni privilegiate, senza generare un interrupt al VMM

- i kernel dei s.o. guest devono quindi essere modificati per avere accesso all'interfaccia del particolare VMM
- la struttura del VMM è semplificata perchè non deve più preoccuparsi di tradurre dinamicamente i tentativi di operazioni privilegiate dei s.o. guest

Le prestazioni rispetto alla Fast Binary Translation sono notevolmente superiori, però ovviamente c'è una necessità di porting dei dei s.o. guest (non sempre facile).



7.4 Architetture virtualizzabili

L'uscita sul mercato di processori con supporto nativo alla virtualizzazione (Intel VT, AMD-V) ha dato l'impulso allo sviluppo di VMM semplificati, basati su virtualizzazione pura:

- No Ring Compression/Aliasing: il s.o. guest esegue in un ring separato (livello di protezione intermedio) da quello delle applicazioni
- Ring Deprivileging: ogni istruzione privilegiata richiesta dal s.o. guest genera un trap gestito dal VMM.

I vantaggi sono maggior **Efficienza** e quindi non è presente il binary translation, **Trasparenza**, l'API presentata dall'hypervisor è la stessa offerta dal processore.

7.5 Protezione nell'architettura x86

Nelle prime versioni dell'architettura x86, fino al modello 80186, non erano presenti meccanismi di protezione. Il sistema operativo e le applicazioni condividevano lo stesso livello di privilegio, il che permetteva alle applicazioni di accedere direttamente ai sottosistemi di input/output e di allocare memoria senza alcun controllo da parte del sistema operativo. Questo causava gravi problemi di sicurezza e stabilità, come nel caso di MS-DOS.

Con l'introduzione della seconda generazione, a partire dal 80286, vennero implementati meccanismi di **protezione** che distinguevano chiaramente tra il sistema operativo, con pieno controllo della macchina fisica, e le applicazioni, che

potevano interagire solo tramite richieste al sistema operativo. Fu introdotto il concetto di **anelli di protezione**, rappresentati dai bit meno significativi del **registro CS**, che indicano il **livello di privilegio corrente (CPL)**. Esistono quattro anelli: l'anello 0 è il più privilegiato ed è riservato al kernel del sistema operativo, mentre l'anello 3 è il meno privilegiato ed è destinato alle applicazioni utente. Solo l'anello 0 può eseguire istruzioni privilegiate, e ogni violazione genera un'eccezione gestita dal sistema operativo.

La **protezione della memoria** si basa sulla **segmentazione**, dove ogni segmento è descritto da un descrittore in una tabella (GDT o LDT) che specifica il livello di protezione e i permessi di accesso. Se il livello di privilegio corrente è superiore a quello richiesto dal segmento, si genera un'eccezione. Sebbene l'architettura x86 preveda quattro anelli, nella pratica se ne utilizzano solo due: l'anello 0 per il sistema operativo e l'anello 3 per le applicazioni. Gli anelli 1 e 2 sono raramente impiegati, talvolta per componenti del sistema operativo che non necessitano del massimo privilegio, al fine di garantire portabilità.

Nel contesto della virtualizzazione, i Virtual Machine Monitor (VMM) operano sfruttando il meccanismo del **ring deprivileging**. In questo schema, l'anello 0 è riservato al VMM, mentre i sistemi operativi guest vengono spostati in anelli con minore privilegio. Esistono due tecniche principali:

- la configurazione **0/1/3**, in cui il sistema operativo guest è collocato nell'anello 1 e le applicazioni nell'anello 3, mentre il VMM rimane nell'anello 0; è ad oggi largamente la più usata nei VMM software, il livello applicativo (livello 3) non può danneggiare il sistema operativo virtuale sul quale è in esecuzione (livello 1) andando a scrivere su porzioni di memoria ad esso dedicate e le eccezioni generate sono catturate dal VMM sul ring 0 e passate da esso, praticamente invariate, al livello 1 dove è presente il sistema operativo virtualizzato.
- la configurazione **0/3/3**, detta ring compression, in cui sia il sistema operativo guest che le applicazioni operano nell'anello 3, lasciando il VMM nell'anello 0. Quest'ultima è compatibile anche con sistemi operativi a 64 bit; non essendo possibile generare alcuna eccezione (sia le applicazioni, che il sistema operativo condividono lo stesso privilegio), devono essere intrapresi meccanismi molto sofisticati con un controllo continuo da parte del VMM, funzionamento che molto si avvicina all'emulazione.

7.5.1 Problemi: Ring Aliasing

Alcune istruzioni non privilegiate, eseguibili in modalità utente, consentono l'accesso in lettura a registri del sistema che, dovrebbero essere gestiti esclusivamente dal Virtual Machine Monitor. Questo può portare alla possibilità, da parte del sistema operativo guest, di rilevare il proprio livello di privilegio, generando potenziali incoerenze nel contesto virtualizzato.

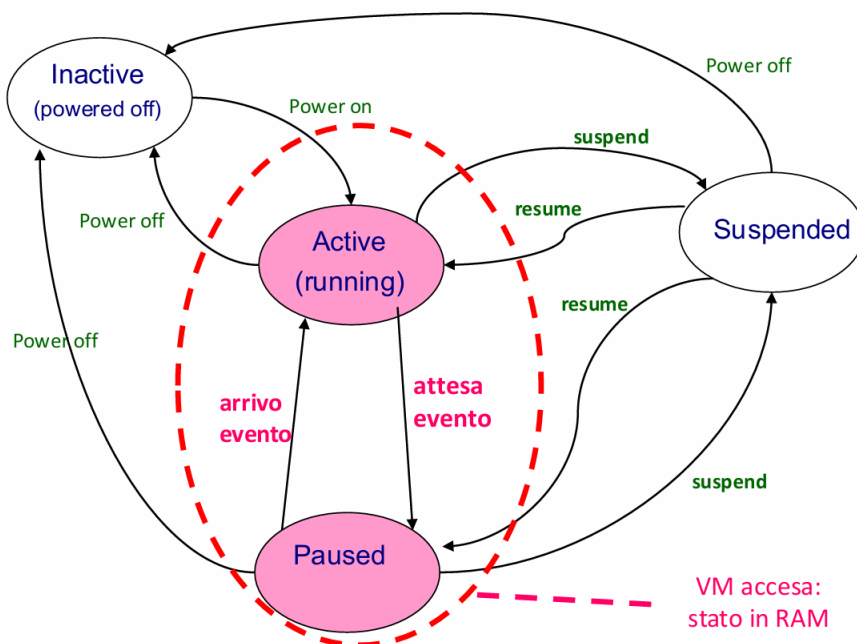
Un esempio significativo si trova nell'architettura x86, dove l'istruzione **PUSH** applicata al registro CS può essere eseguita da qualsiasi livello di privilegio.

Poiché il registro CS contiene il Current Privilege Level (CPL), il sistema operativo virtualizzato può leggere direttamente il valore e determinare il ring in cui sta operando.

In un ambiente virtualizzato che adotta la configurazione 0/1/3, il sistema operativo guest, aspettandosi di trovarsi nell'anello 0, rileva invece di essere confinato nell'anello 1, riservato dal VMM. Questo tipo di rilevamento può compromettere l'illusione di pieno controllo che il VMM cerca di mantenere, evidenziando una delle sfide tecniche nella virtualizzazione su architetture non progettate nativamente per essa.

7.6 Stati di una VM

- **Running** (o attiva): la macchina è accesa e occupa memoria nella ram del server sul quale è allocata.
- **Inactive** (Powered Off): la macchina è spenta ed è rappresentata nel file system tramite un file immagine.
- **Paused**: la macchina virtuale è in attesa di un evento (es. I/O richiesto da un processo nell'ambiente guest).
- **Suspended**: la macchina virtuale è stata sospesa dal VMM; il suo stato e le risorse utilizzate sono salvate nel file system (file immagine). L'uscita dallo stato di sospensione avviene tramite l'operazione resume da parte del VMM.



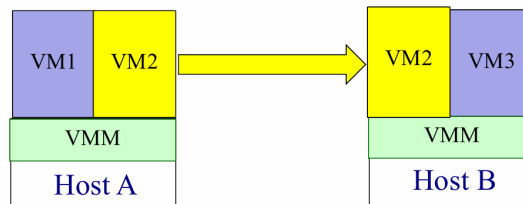
7.6.1 Migrazione di VM

Nei datacenter caratterizzati dall'utilizzo di server virtualizzati, emerge sempre più la necessità di una gestione flessibile e dinamica delle macchine virtuali. Questa esigenza nasce dalla necessità di affrontare variazioni del carico di lavoro, che richiedono soluzioni di bilanciamento e consolidamento per garantire prestazioni ottimali. Allo stesso tempo, diventa fondamentale poter effettuare interventi di manutenzione sui server senza interrompere i servizi, mantenendo la continuità operativa.

Un altro obiettivo cruciale è la gestione orientata al risparmio energetico, che consente di ottimizzare l'utilizzo delle risorse riducendo i consumi e i costi. Infine, la tolleranza ai guasti e le strategie di disaster recovery rappresentano elementi indispensabili per assicurare affidabilità e resilienza dei sistemi.

In tutti questi scenari, la possibilità di spostare le macchine virtuali tra server diversi si rivela un meccanismo centrale, capace di garantire flessibilità, efficienza e sicurezza nella gestione complessiva dell'infrastruttura.

- Le macchine virtuali possono essere spostate da un server fisico a un altro senza essere spente.



Il Virtual Machine Monitor ha la possibilità di mettere in stand-by una macchina virtuale attraverso l'operazione **suspend**, durante la quale lo stato della VM viene salvato nella memoria secondaria. Una volta sospesa, la macchina può riprendere l'esecuzione grazie all'operazione **resume**, che ripristina lo stato precedentemente salvato nella memoria centrale, consentendo di continuare esattamente dal punto in cui era stata interrotta.

Poiché una macchina virtuale è quasi del tutto indipendente dal server fisico su cui è ospitata, il ripristino può avvenire anche su un nodo diverso da quello originario. Questo rende possibile una gestione più flessibile delle risorse, favorendo la mobilità delle VM e garantendo continuità operativa anche in scenari di manutenzione o riorganizzazione dell'infrastruttura.

È fondamentale ridurre al minimo alcuni fattori critici. Il **downtime**, ossia il tempo in cui la macchina non è in grado di rispondere alle richieste degli utenti, deve essere contenuto per garantire continuità di servizio. Allo stesso modo, è importante limitare il **tempo della migrazione** e il **consumo di banda** di rete, così da rendere l'operazione più efficiente e meno impattante sull'infrastruttura.

Quando il file system è condiviso tra i nodi coinvolti, ad esempio tra A e B, non è necessario trasferire l'intero file immagine della macchina virtuale. Questo

semplifica notevolmente il processo, riducendo i tempi e le risorse richieste, e rende la migrazione più rapida e meno onerosa.

Soluzione: Precopy

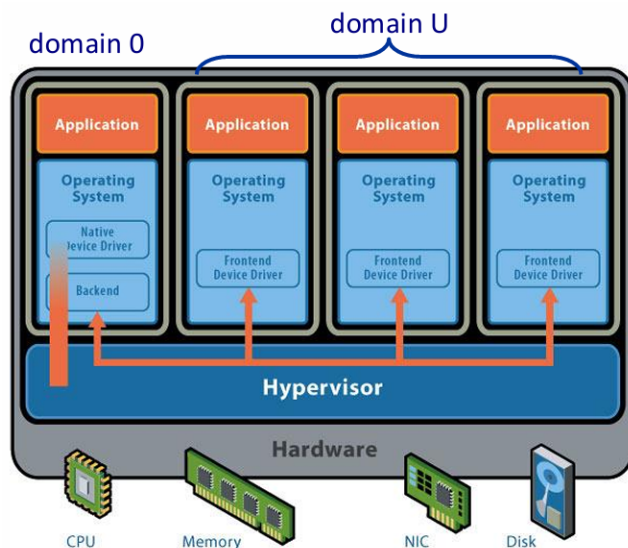
1. **Pre-migrazione:** individuazione della VM da migrare e dell'host (B) di destinazione.
2. **Reservation:** viene inizializzata una VM (container) sul server di destinazione.
3. **Pre-copia iterativa delle pagine:** viene eseguita una copia nell'host B di tutte le pagine allocate in memoria sull'host A per la VM da migrare; successivamente vengono iterativamente copiate da A a B tutte le pagine modificate (**dirty pages**) fino a quando il numero di dirty pages è inferiore a una soglia data.
4. **Sospensione della VM e copia dello stato + dirty pages** da A a B.
5. **Commit:** la VM viene eliminata dal server A
6. **Resume:** la VM viene attivata nel server B

Soluzione: Postcopy la macchina viene sospesa e vengono copiate (non iterativamente) pagine e stato. Tempo di migrazione più basso, ma downtime molto più elevato.

7.7 XEN

Xen è un VMM open source sviluppato presso l'Università di Cambridge nel 2003, basato sui principi della paravirtualizzazione. Uno dei primi risultati concreti fu il porting di Linux su Xen, con la creazione di XenLinux e l'integrazione in distribuzioni come Suse e CentOS. Per consentire al kernel di Linux di dialogare con le API di Xen furono necessarie modifiche pari a circa 3000 linee di codice, corrispondenti a circa l'1,36% del totale.

Fu avviato anche un progetto di porting di Windows XP, denominato XenXP, in collaborazione con Microsoft, che tuttavia non venne completato. Nel tempo molte distribuzioni Linux, tra cui Suse, Red Hat, CentOS e Debian, hanno reso disponibili pacchetti precompilati per facilitare l'installazione di Xen.

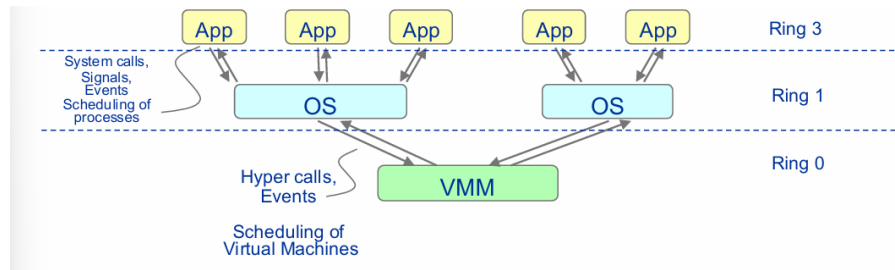


Il VMM, o hypervisor, ha il compito di virtualizzare la CPU, la memoria e i dispositivi per ciascuna macchina virtuale ($domain_i$). Xen, in particolare, mette a disposizione un'interfaccia di controllo che consente di gestire la suddivisione di queste risorse tra i vari domini. L'accesso a tale interfaccia non è libero, ma riservato a una macchina virtuale dedicata, denominata **domain 0**. In questo dominio viene eseguito il software che si occupa della gestione e del controllo dell'intero sistema.

Il software di controllo presente nel domain 0 è distinto dall'hypervisor stesso, realizzando una chiara separazione tra i meccanismi di virtualizzazione e le politiche di gestione. Questa architettura garantisce maggiore modularità e flessibilità, permettendo di mantenere l'hypervisor leggero e focalizzato sulle funzioni di base, mentre le decisioni di amministrazione e coordinamento vengono demandate al dom0.

7.7.1 Caratteristiche Xen

- **Paravirtualizzazione:** Le macchine virtuali eseguono direttamente le istruzioni non privilegiate, inoltre l'esecuzione di istruzioni privilegiate viene delegata al VMM tramite chiamate al VMM (hypercalls).
- **Protezione (x86):** I sistemi operativi guest OS sono collocati nel ring 1 e il VMM nel ring 0.



La **gestione della memoria** nei sistemi virtualizzati con Xen si basa su un modello in cui i sistemi operativi guest continuano a occuparsi della memoria virtuale attraverso i tradizionali meccanismi di paginazione, mentre il VMM interviene per garantire coerenza e sicurezza. Nell'architettura x86 i page fault vengono gestiti direttamente a livello hardware tramite la TLB, ma nelle macchine virtuali le tabelle delle pagine non sono gestite dai kernel guest: esse vengono mappate nella memoria fisica dal VMM, risultano accessibili in sola lettura ai guest e possono essere modificate esclusivamente dal VMM. Ogni richiesta di aggiornamento da parte dei sistemi operativi viene validata e applicata dal monitor di virtualizzazione.

Memory Split: Per migliorare l'efficienza, Xen risiede nei primi 64 MB dello spazio di indirizzamento virtuale di ogni VM. In questo modo non è necessario eseguire un flush della TLB a ogni hypercall, e lo spazio di indirizzamento è strutturato in segmenti separati che contengono Xen e il kernel del sistema operativo guest.

I guest OS si occupano della paginazione, delegando al VMM la scrittura delle page table entries. Le tabelle della pagine devono essere create e verificate dal VMM su richiesta dei guest: una volta create, le tabelle rimangono Read-only per il guest.

Quando un guest richiede la creazione di un nuovo processo, il VMM genera una nuova tabella delle pagine, vi aggiunge le pagine relative al segmento di Xen e ne registra la gestione, acquisendo il read-only. Ogni successivo tentativo di aggiornamento da parte del guest provoca un **protection fault**, che viene intercettato e gestito dal VMM per verificare e applicare l'aggiornamento.

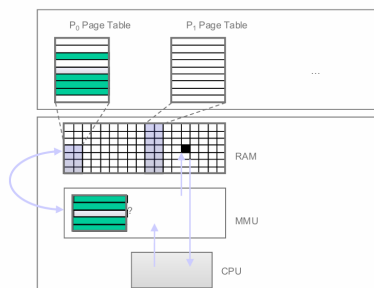


Figure 3: gestione della memoria

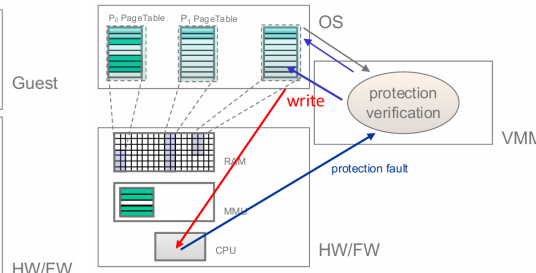


Figure 4: creazione di un processo

Il **balloon process**, pensato per consentire al VMM di reclamare memoria dalle VM in caso di necessità. Su ogni macchina virtuale è attivo un processo che comunica direttamente con il VMM e che può essere “gonfiato” su richiesta: ciò significa che il sistema operativo guest assegna nuove pagine al balloon process, che le restituisce al VMM. Questo approccio permette di liberare memoria meno utilizzata e di ridistribuirla, ad esempio, per l’attivazione di nuove macchine virtuali.

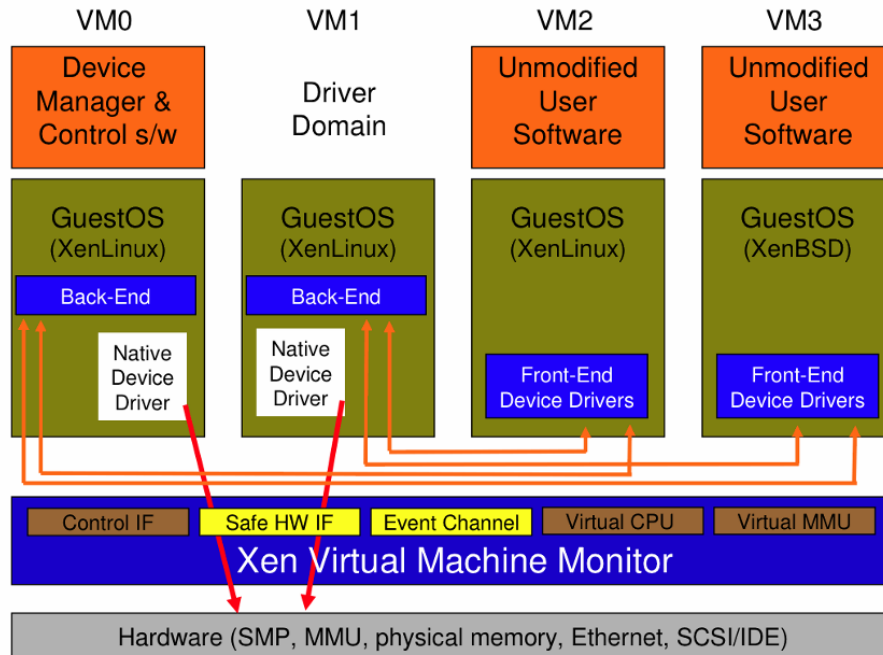
7.7.2 Virtualizzazione della CPU

Il VMM definisce un’architettura virtuale che riproduce quella del processore, ma sostituisce le istruzioni privilegiate con specifiche hypercalls. L’invocazione di una hypercall comporta il passaggio dal sistema guest a Xen, ossia dal ring 1 al ring 0, e per questo motivo i kernel dei sistemi operativi guest devono essere opportunamente modificati per poter interagire con l’hypervisor.

Un altro compito fondamentale del VMM è lo scheduling delle macchine virtuali, realizzato attraverso il **Borrowed Virtual Time scheduling algorithm**. Questo algoritmo si basa sul concetto di virtual-time ed è di tipo general-purpose, consente di ottenere schedulazioni efficienti anche in presenza di vincoli temporali stringenti, come nel caso di applicazioni sensibili al tempo, protocolli TCP/IP o servizi real-time.

Il sistema utilizza due diversi clock: il **real-time**, che rappresenta il tempo del processore e inizia con il boot della macchina, e il **virtual-time**, associato a ciascuna VM e che avanza solo quando la macchina virtuale è effettivamente in esecuzione. I valori di questi tempi vengono comunicati ai sistemi guest tramite eventi, garantendo una gestione coerente e sincronizzata delle risorse.

7.7.3 Virtualizzazione dell'I/O



La virtualizzazione dell'I/O in Xen si basa su un'architettura a due livelli di driver. Da un lato troviamo i **back-end driver**, che per ogni dispositivo sono isolati all'interno di una macchina virtuale dedicata, tipicamente il domain 0, e hanno accesso diretto all'hardware. Dall'altro lato ci sono i front-end driver, presenti in ciascun sistema operativo guest, che rappresentano versioni semplificate dei driver e consentono l'accesso ai dispositivi attraverso il collegamento con il back-end.

Questa soluzione offre diversi vantaggi: garantisce portabilità, facilitando la migrazione delle macchine virtuali; assicura isolamento, poiché i driver hardware sono confinati in un dominio specifico; e semplifica la struttura del VMM, che non deve gestire direttamente la complessità dei dispositivi. Tuttavia, comporta anche un limite: la necessità di comunicare costantemente con il back-end introduce un livello di intermediazione che si traduce nell'uso di asynchronous I/O rings, con possibili implicazioni sulle prestazioni.

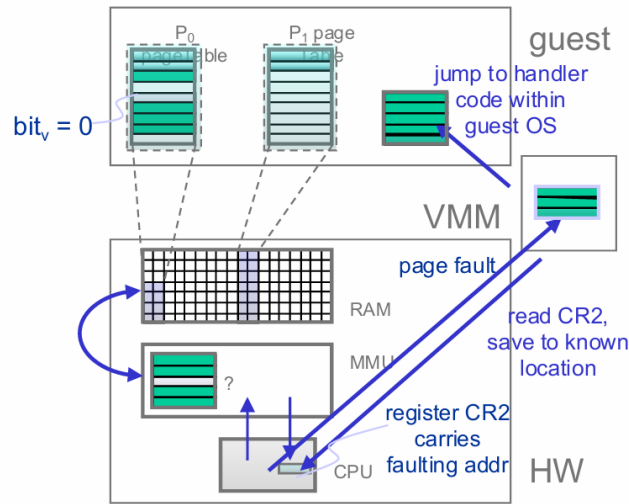
7.7.4 Gestione interruzioni e eccezioni

La virtualizzazione della gestione delle interruzioni in Xen è realizzata in modo semplice: il vettore delle interruzioni punta alle routine del kernel guest, che si occupa di gestirle autonomamente. Tuttavia, esiste un caso particolare che riguarda i **page fault**. La loro gestione non può essere delegata completamente al sistema operativo guest, poiché richiede l'accesso al registro **CR2**, contenente l'indirizzo che ha provocato l'errore. Questo registro è accessibile soltanto dal

ring 0, e quindi deve necessariamente coinvolgere il VMM.

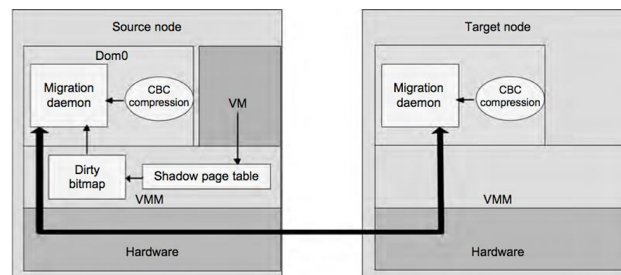
7.7.5 gestione del page fault

In tali circostanze, l'handler punta a codice Xen eseguito nel ring 0. La routine di gestione legge il contenuto del registro CR2 e lo copia in una variabile nello spazio del guest. Successivamente il controllo viene trasferito al sistema operativo guest, che può così completare la gestione del page fault.



7.7.6 Live migration

Per quanto riguarda la live migration in Xen, essa è di tipo **guest-based**. Il comando di migrazione viene avviato da un demone presente nel domain 0 del server di origine. La procedura si basa sulla tecnica di **pre-copy**, che prevede la copia delle pagine di memoria verso l'host di destinazione. Per ridurre l'occupazione di banda, le pagine vengono compresse durante il trasferimento, rendendo l'operazione più efficiente e meno impattante sulla rete.



8 Architettura della macchina corrente

La concorrenza è resa possibile da un adeguato supporto a run-time, realizzato dalla <macchina concorrente>, realizzata in SW sull'architettura HW disponibile. Nella maggior parte dei casi la macchina concorrente viene realizzata dal **kernel** del sistema operativo. La struttura interna della macchina concorrente dipende principalmente:

- Dalle caratteristiche fisiche del sistema HW:
 - Monoprocessore Multiprocessore (UMA/NUMA)
- Dal modello di interazione adottato:
 - Modello a memoria comune Modello a scambio di messaggi

Inoltre la macchina concorrente implementa **Meccanismi di multiprogrammazione**: 1 cpu virtuale per ogni processo (thread o processo pesante); e **Mecanismi di interazione**: sincronizzazione e/o comunicazione

8.1 Nucleo di un sistema a Processi

In un sistema multiprogrammato (o <a processi>) vengono offerte tante unità di elaborazione astratte (macchine virtuali) quanti sono i processi; ogni macchina possiede come **set di istruzioni elementari quelle corrispondenti all'unità centrale reale** più le istruzioni relative alla **creazione** ed **eliminazione** dei processi, al meccanismo di **comunicazione** e **sincronizzazione** (compresa la comunicazione con i dispositivi di I/O visti come processori esterni). Questo modello consente di mettere in evidenza le proprietà logiche di comunicazione e sincronizzazione tra processi senza doversi occupare degli aspetti implementativi legati alle particolari caratteristiche del processore fisico (es. gestione delle interruzioni).

Definizione: Si chiama nucleo (**kernel**) il modulo (o insieme di funzioni) realizzato in software, hardware o firmware che supporta il concetto di processo e realizza gli strumenti necessari per la gestione dei processi.

Il nucleo costituisce il livello più interno di un qualunque sistema basato su processi, ad esempio:

- il livello più elementare di un sistema operativo multiprogrammato;
- il supporto a tempo di esecuzione di un linguaggio per la programmazione concorrente.

Il nucleo è il solo modulo che è conscio dell'esistenza delle **interruzioni**. I processi che colloquiano con i dispositivi utilizzano opportune primitive del nucleo che provvedono a sospenderli in attesa del completamento dell'azione richiesta. Non appena l'azione viene completata, il relativo segnale di interruzione inviato

dal dispositivo alla CPU viene catturato dal nucleo che provvede a risvegliare il processo sospeso.

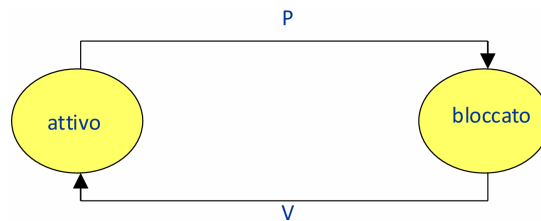
La gestione delle interruzioni è quindi invisibile ai processi ed ha come unico effetto rilevabile di rallentare la loro esecuzione sulle rispettive macchine virtuali.

8.2 Caratteristiche fondamentali del nucleo

Efficienza: condiziona l'intera struttura a processi. Infatti esistono dei sistemi in cui alcune o tutte le operazioni del nucleo sono realizzate in hardware o attraverso microprogrammi.

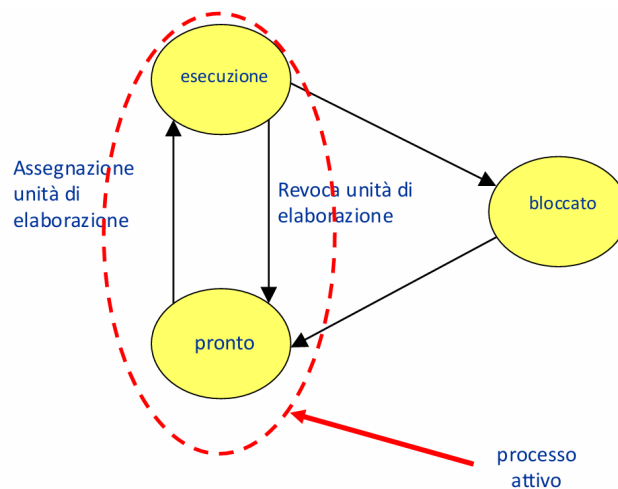
Dimensioni: la semplicità delle funzioni richieste al nucleo fa sì che la sua dimensione risulti estremamente limitata.

Separazione tra meccanismi e politiche: il nucleo deve contenere solo meccanismi consentendo così, a livello di processi, di utilizzare tali meccanismi per la realizzazione di diverse politiche di gestione a seconda del tipo di applicazione.



Le transizioni tra i due stati sono implementate dai meccanismi di sincronizzazione realizzati dal nucleo. **P** per sospensione e **V** per risveglio.

Stati di un processo in un sistema in cui il numero di processi supera il numero delle unità di elaborazione:



Quando un processo perde il controllo del processore, il contenuto dei registri del processo viene salvato in un'area di memoria associata al processo, chiamata descrittore.

Ciò consente una maggiore flessibilità nella politica di assegnazione del processore ai processi, rispetto alla soluzione di salvare le informazioni nello stack.

Quindi la funzione fondamentale del nucleo di un sistema a processi è la **gestione delle transizioni di stato** dei processi. I principali compiti del nucleo sono:

- **Gestire il salvataggio e il ripristino dei contesti dei processi:** quando un processo abbandona il controllo dell'unità di elaborazione fisica, tutte le informazioni contenute nei registri di tale unità devono essere trasferite nel descrittore. Allo stesso modo, quando un processo riprende l'esecuzione tutte le informazioni contenute nel suo descrittore devono essere trasferite nei registri di macchina.
- **Scegliere a quale tra i processi pronti assegnare l'unità di elaborazione (scheduling della CPU):** quando un processo abbandona il controllo dell'unità di elaborazione, il nucleo deve scegliere tra tutti i processi pronti quello da mettere in esecuzione. La scelta può essere o di tipo FIFO, oppure può utilizzare la priorità dei processi.
- **Gestire le interruzioni dei dispositivi esterni:** traducendole in attivazione di processi da bloccato a pronto.
- **Realizzare i meccanismi di sincronizzazione dei processi:** gestendo il passaggio dei processi dallo stato di esecuzione allo stato di bloccato e da bloccato a pronto. (es. P e V)

8.3 Realizzazione del Nucleo: Architettura monoproces- sore

8.3.1 Strutture Dati del Nucleo

Il **Descrittore del processo** è la principale struttura dati del nucleo, e contiene le seguenti informazioni:

- **Identificatore del processo:** nome che identifica univocamente il processo durante il suo tempo di vita;
- **Stato del processo:** pronto, esecuzione, bloccato...
- **Modalità di Servizio:** contiene parametri di scheduling
 - FIFO
 - Priorità (fissa o variabile)
 - Deadline (tempo massimo entro il quale la richiesta può essere soddisfatta)

– Quanto di tempo (sistemi time sharing)

- **Contesto del processo:** program counter, registro di stato, registri generali, indirizzo dell'area di memoria privata del processo.
- **Code di processo:** a seconda dello stato del processo, il suo descrittore può essere inserito in apposite code. Ogni descrittore contiene l'identificatore del processo successivo nella stessa coda.

```
1 typedef struct {
2     int indice_priorita;
3     int delta_t;
4 } modalita_di_servizio; //.
5
6 typedef struct {
7     int nome;
8     modalita_di_servizio servizio;
9     tipo_contesto contesto;
10    tipo_stato stato;    // running, ready, waiting, ecc.
11    int successivo;
12 } descrittore_processo;
13
14 /*array di tutti i descrittori*/
15 descrittore_processo descrittori[num_max_proc];
```

Listing 1: Realizzazione descrittore del processo

8.3.2 Coda dei processi pronti

Esistono sempre una o più **code di processi pronti**. Non appena un processo viene riattivato tramite una **v** viene inserito in fondo alla coda corrispondente alla sua priorità.

La coda dei processi pronti contiene sempre almeno un **dummy process** il quale viene messo in esecuzione solamente quando tutte le altre code sono vuote e rimane in esecuzione fino a quando qualche altro processo diventa pronto. Inoltre il dummy process ha la priorità più bassa ed è sempre nello stato di pronto, eseguendo un ciclo senza fine.

Listing 2: Esempio di realizzazione coda dei processi pronti

```
1 typedef struct {
2     int primo; int ultimo; } descrittore_coda;
3
4 typedef descrittore_coda coda_a_livelli[Npriorita];
5
6 coda_a_livelli coda_processi_pronti;
7
8 // Inserimento => inserisce il processo di indice P nella
   ↪ coda C
```



```

9 void Inserimento(int P, descrittore_coda C) {...}
10
11 // Prelievo => estrae il primo processo dalla coda C e
    ↳ restituisce il suo indice
12 int Prelievo(descrittore_coda C) {...}

```

Coda dei descrittori liberi:

Coda nella quale sono concatenati i descrittori disponibili per la creazione di nuovi processi e nella quale sono reinseriti i descrittori dei processi terminati:

```
descrittore_coda descrittori_liberi;
```

Processo in esecuzione:

Il nucleo necessita di conoscere quale processo è in esecuzione. Questa informazione, rappresentata dall'indice del descrittore del processo, viene contenuta in una particolare variabile del nucleo. Quando il nucleo viene inizializzato, viene creato un processo e l'indice del processo viene assegnato a processo-in-esecuzione.

```
int processo_in_esecuzione;
```

8.4 Funzioni del Nucleo

Le funzioni del nucleo realizzano le operazioni di **transizione di stato** per i singoli processi. Ogni transizione prevede il prelievo da una coda del descrittore del processo coinvolto ed il suo inserimento in un'altra coda.

A tale scopo vengono utilizzate due procedure: **Inserimento** e **Prelievo** di un descrittore da una coda. Se la coda è vuota si adotta l'ipotesi che il campo primo assuma il valore -1 (NIL).

Le funzioni del nucleo vengono suddivise in due livelli:

- **Livello Superiore:** il quale contiene tutte le **funzioni direttamente utilizzabili dai processi** sia esterni che interni, quali risposta ai segnali di interruzione e primitive per la creazione, eliminazione e sincronizzazione dei processi.
- **Livello Inferiore:** il quale realizza tutte le funzionalità di cambio di contesto, ad esempio salvataggio del contesto del processo che si sospende nel suo descrittore, scelta di un nuovo processo da mettere in esecuzione tra quelli pronti e ripristino del suo contesto.

L'ambiente di esecuzione delle funzioni del nucleo ha caratteristiche distinte dal quello dei processi. Infatti, per motivi di protezione, le funzioni del nucleo sono le uniche che:

- possono operare sulle strutture dati che rappresentano lo stato del sistema (descrittori, code di descrittori, semafori, ecc..)

- possono utilizzare istruzioni privilegiate (abilitazione e disabilitazione delle interruzioni, ecc)

Le funzioni del nucleo devono essere eseguite in modo mutuamente esclusivo. Inoltre i due ambienti di esecuzione (nucleo e processi utente) corrispondono a stati diversi di operazione dell'elaboratore (kernel e user). E il meccanismo di passaggio da uno all'altro è basato sul meccanismo delle interruzioni.

- Nel caso di funzioni chiamate da **processi esterni**, il passaggio all'ambiente del nucleo è ottenuto tramite il meccanismo di risposta al segnale di **interruzione**
- Nel caso di funzioni chiamate da **processi interni**, il passaggio è ottenuto tramite l'esecuzione di **system calls**
- In entrambi i casi, al completamento della funzione richiesta, il trasferimento all'ambiente user avviene tramite il meccanismo di **ritorno da interruzione (RTI)**

8.4.1 Funzione di livello inferiore: Cambio di Contesto

La funzione di cambio di contesto è costituita da 3 principali fasi:

Salvataggio dello stato:

Salvataggio del contesto del processo in esecuzione nel suo descrittore e inserimento del descrittore nella coda dei processi bloccati o dei processi pronti.

```

1 void Salvataggio_stato() {
2     int j;
3     j = processo_in_esecuzione;
4     descrittori[j].contesto = <valori dei registri CPU>;
5 }

```

Assegnazione della CPU

Rimozione del processo a maggiore priorità dalla coda dei pronti e caricamento dell'identificatore di tale processo nel registro processo in esecuzione.

```

1 // scheduling: algoritmo con priorit 
2 void Assegnazione_CPU() {
3     int k = 0, j;
4     while(coda_processi_pronti[k].primo == -1) {
5         k++;
6     }
7     j = Prelievo(coda_processi_pronti[k]);
8     processo_in_esecuzione = j;
9 }

```

Ripristino dello stato:

Caricamento del processo del nuovo processo nei registri di macchina.

```

1 void Ripristino_stato() {
2     int j;
3     j = processo_in_esecuzione;
4     <registro-temp> = descrittori[j].servizio.delta_t;
5     <registro-CPU> = descrittori[j].contesto;
6 }

```

Gestione del temporizzatore:

Per consentire la modalità di servizio a divisione di tempo è necessario che il nucleo gestisca un **dispositivo temporizzatore** tramite un'apposita procedura che ad intervalli di tempo fissati, provveda a sospendere il processo in esecuzione ed assegnare l'unità di elaborazione ad un altro processo.

```

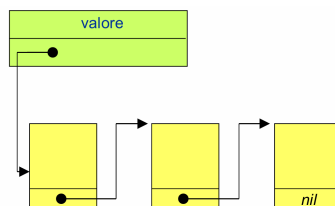
1 void Cambio_di_Contesto() {
2     int j, k;
3     Salvataggio_stato();
4     j = processo_in_esecuzione;
5     k = descrittori[j].servizio.priorita;
6     Inserimento(j, coda_processi_pronti[k]);
7     Assegnazione_CPU();
8     Ripristino_stato();
9 }

```

8.5 Realizzazione Semaforo (caso monoprocesso)

Nel nucleo di un sistema monoprocesso il semaforo può essere implementato:

- una variabile intera che rappresenta il suo valore (≥ 0)
- un puntatore ad una lista di descrittori di processi in attesa sul semaforo (bloccati)



Se non sono presenti semafori in coda, il puntatore alla lista contiene la costante **nil**.

La coda viene gestita con politica FIFO, e quindi i processi risultano ordinati secondo il loro tempo di arrivo nella coda associata al semaforo.

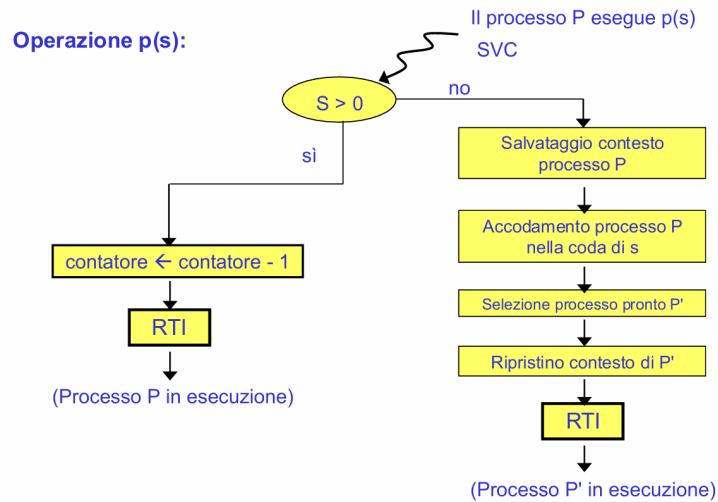
Il descrittore di un processo viene inserito nella coda del semaforo come conseguenza di una primitiva p non passante, e viene prelevato per effetto di una v .

```

1 // gestione dei processi basata su priorit  => associamo al
  // semaforo un insieme di code (una per priorit )
2 typedef struct {
3     int contatore;
4     coda_a_livelli coda;
5 } descr_semaforo;

```

Semafori: operazione p

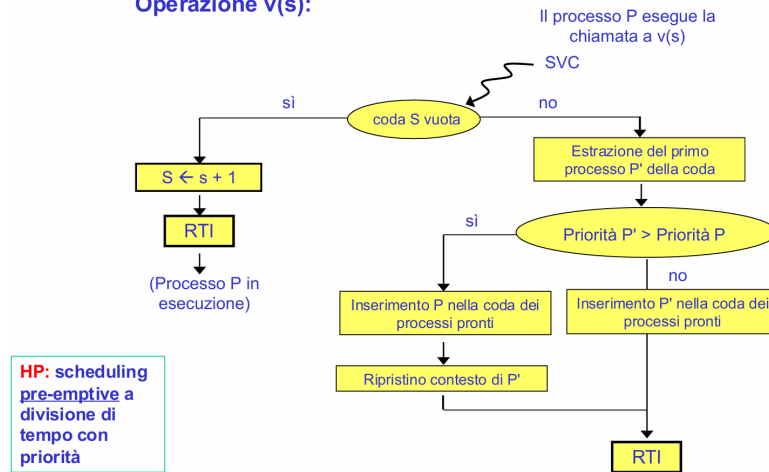


```

1 // insieme di tutti i semafori
2 descr_semaforo semafori[num_max_sem];
3 //ogni semaforo e' rappresentato dall'indice che lo
  //individua nel vettore semafori
4 typedef int semaforo;
5
6 void P(semaforo s) {
7     int j, k;
8     if(semafori[s].contatore == 0) {
9         Salvataggio_stato();
10        j = processo_in_esecuzione;
11        k = descrittore[j].servizio.priorita;
12        Inserimento(j, semafori[s].coda[k]);
13        Assegnazione_CPU();
14        Ripristino_stato();
15    } else { contatore--; } }

```

Operazione v(s):



```

1 void V(semaforo s) {
2     int j, k, p, q; // j,k: processi; p,q: indici prioritari
3     q = 0;
4
5     while(semafori[s].coda[q].primo == -1 && q <
6           ↪ min_priorita) {
7         q++; }
8
9     if(semafori[s].coda[q].primo != -1) {
10        k = Prelievo(semafori[s].coda[q]);
11        j = processo_in_esecuzione;
12        p = descrittori[j].servizio.priorita;
13        if(p < q) {
14            // il processo in esecuzione e' prioritario
15            Inserimento(k, coda_processi_pronti[q]);
16        } else {
17            // preemption
18            Salvataggio_stato();
19            Inserimento(j, coda_processi_pronti[p]);
20            processo_in_esecuzione = k;
21            Ripristino_stato();
22        } } else {
23        semafori[s].contatore++;
24    } }
  
```

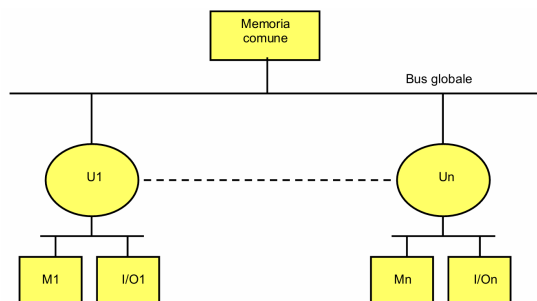
8.6 Architettura Multiprocessore: realizzazione del nucleo, Modello a memoria comune

Sistemi operativi multiprocessore: organizzazione interna

Un sistema operativo che esegue su un'architettura multiprocessore deve gestire una **molteplicità di CPU**, ognuna delle quali può accedere alla stessa **memoria condivisa**.

Esistono principalmente due modelli:

- **Modello SMP**: unica copia del nucleo condivisa tra tutte le CPU
- **Modello a nuclei distinti**: più istanze del nucleo concorrenti



8.6.1 Modello SMP: Simmetric Multi Processing

In questo modello è presente un'unica copia del nucleo del sistema operativo allocata nella memoria comune che si occupa della gestione di tutte le risorse disponibili, comprese le CPU. Ogni processo può essere allocato su una qualunque CPU. Inoltre è possibile che processi che eseguono su CPU diverse richiedano contemporaneamente funzioni del nucleo ad esempio le System Call. È dato che ogni funzione del nucleo comporta un accesso alle strutture dati interne al nucleo, occorre regolare gli accessi al nucleo in modo che avvengano in modo sincronizzato.

Sincronizzazione nell'accesso al nucleo:

Soluzione ad un solo lock:

Viene associato al nucleo un **Lock L**, per garantire la mutua esclusione nell'esecuzione di funzioni del nucleo da parte di processi diversi: l'accesso esclusivo alle sue strutture dati può essere ottenuto delimitando il corpo di ogni richiesta per il nucleo con le primitive **lock** e **unlock** applicate all'unico **Lock L**.

Il problema fondamentale di questa soluzione è la **limitazione del grado di parallelismo**, escludendo a priori ogni possibilità di esecuzione contemporanea di funzioni del nucleo che operano su strutture dati distinte.

Soluzione a più lock:

Un maggiore parallelismo può essere ottenuto individuando all'interno del nucleo diverse classi di sezioni critiche, ognuna associata a una struttura dati separata e sufficientemente indipendente dalle altre. Ad ogni struttura dati viene associato un lock distinto.

Ad esempio la coda dei processi pronti, i singoli semafori, ecc, vengono protetti tramite lock distinti.

Ogni operazione del nucleo che richiederà l'accesso a una particolare struttura S_i protetta dal lock L_i , conterrà una sezione critica il cui prologo ed epilogo saranno rispettivamente $lock(L_i)$ e $unlock(L_i)$.

Scheduling dei processi:

Il modello SMP consente la schedulazione di ogni processo su uno qualunque dei processori attraverso la **load balancing**, ovvero la possibilità di attuare politiche di distribuzione equa del carico sui processori.

Tuttavia, in alcuni casi può risultare più conveniente assegnare un processo ad un determinato processore (affinità):

- i processori possono accedere più rapidamente alla loro memoria privata piuttosto che a quella remota, quindi potrebbe convenire schedulare il processo sul processore la cui memoria privata già contiene il suo codice.
- in sistemi NUMA l'accesso alla memoria "più vicina" è più rapido: conviene schedulare il processo sul processore più vicino alla memoria ove è allocato il suo spazio di indirizzamento.
- i processori hanno memoria cache. Il processo dovrebbe essere assegnato al processore sul quale era stato precedentemente eseguito.

8.6.2 Modello a nuclei distinti

In questo modello la struttura interna del sistema operativo è articolata su più nucleo, ognuno dedicato alla gestione di una diversa CPU.

L'assunzione di base è che l'insieme dei processi che eseguiranno nel sistema sia partizionabile in tanti sottoinsiemi (**nodi virtuali**) lasciamente connessi, cioè con un ridotto numero di interazioni reciproche:

- Ciascun **nodo virtuale** è associato ad un **nodo fisico**: esso è gestito da un nucleo distinto e pertanto tutte le strutture dati del nucleo relative al nodo virtuale vengono allocate sulla **memoria privata** del nodo fisico.
- In questo modo tutte le interazioni locali al nodo virtuale possono avvenire **indipendentemente** e **concorrentemente** a quelle di altri nodi virtuali, facendo riferimento al nucleo del nodo.

Solo le interazioni tra processi appartenenti a nodi virtuali diversi utilizzano la memoria comune.

8.6.3 Organizzazione SMP vs nuclei distinti

- **Grado di parallelismo tra CPU**: il modello a nuclei distinti è più vantaggioso, in quanto il grado di accoppiamento tra CPU è più basso. Maggiore scalabilità.

- **Gestione ottimale delle risorse computazionali:**

- il modello SMP fornisce i presupposti per un migliore bilanciamento del carico tra le CPU perché lo scheduler può decidere di allocare ogni processo su qualunque CPU.
- il secondo modello vincola ogni processo ad essere schedulato sempre sullo stesso nodo.

Realizzazione dei semafori nel modello SMP

Tutte le CPU condividono lo stesso nucleo: per sincronizzare gli accessi al nucleo, le struttura dati del nucleo vengono protette tramite lock.

In particolare, i singoli semafori e la coda dei processi pronti vengono protetti tramite **lock distinti**.

In questo caso due operazioni P su semafori diversi:

- possono operare in modo **contemporaneo** se non risultano sospensive
- in caso contrario, vengono **sequenzializzati** solo gli accessi alla coda dei processi pronti.

Esempio: scheduling pre-emptive basato su priorità ->

L'esecuzione della V può portare al cambio di contesto tra il processo risvegliato (P_r) e uno tra i processi in esecuzione (P_i), se la priorità di P_r è più alta della priorità di P_i : in questo caso il nucleo deve provvedere a revocare l'unità di elaborazione al processo P_i con priorità più bassa ed assegnarla al processo P_r riattivato dalla V.

Occorre che il nucleo mantenga l'informazione del processo in esecuzione a più bassa priorità e del nodo fisico su cui opera.

Inoltre è necessario un meccanismo di segnalazione tra le unità di elaborazione.

SMP: segnalazione inter-processore

Ad esempio, siano:

sem un semaforo, sul quale assumiamo sia inizialmente sospeso un processo (P_j);

P_i il processo che esegue la V(sem) eseguendo sul nodo U_i ;

P_j il processo riattivato per effetto della V(sem);

P_k il processo a più bassa priorità che esegue su U_k ;

P_i chiama V(sem): Il nucleo, attualmente eseguito da U_i con la funzione V(sem), invia un segnale di interruzione a U_k .

2. U_k gestisce l'interruzione utilizzando le funzioni del nucleo: inserisce P_k nella coda dei processi pronti e mette in esecuzione il processo P_j .

8.6.4 Realizzazione dei semafori nel modello a nuclei distinti

Solo le interazioni tra processi appartenenti a nodi virtuali diversi utilizzano la memoria comune. Distinzione tra:

- **Semafori privati** di un nodo U , cioè utilizzati da processi appartenenti al nodo virtuale U . Poiché hanno visibilità confinata al nodo U , vengono realizzati come nel caso monoprocesso.
- **Semafori condivisi** tra nodi, cioè utilizzati da processi appartenenti a nodi virtuali differenti.

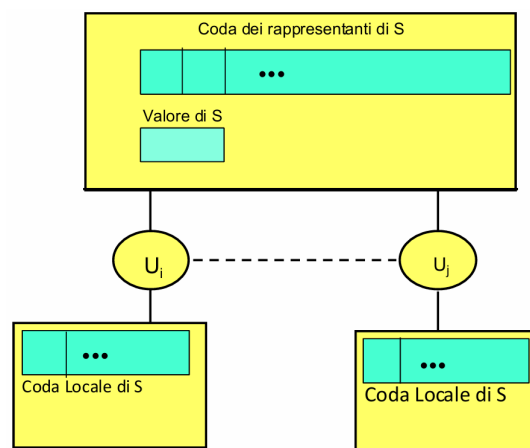
La memoria comune deve contenere tutte le informazioni relative ai semafori condivisi.

8.6.5 Semafori condivisi

Ogni semaforo condiviso viene rappresentato nella memoria comune da un **intero non negativo**; l'accesso al semaforo sarà protetto da un **lock** x , anch'esso memorizzato in memoria comune. **Rappresentante del processo**: Insieme minimo di informazioni sufficienti per identificare sia il **nodo fisico** su cui il processo opera, sia il **descrittore** contenuto nella memoria privata del processore.

Per ogni semaforo condiviso S vengono mantenute varie code:

- Su ogni nodo U_i : una coda locale a U_i contenente i descrittori dei processi locali sospesi su S ; la coda risiede nella memoria privata del nodo e viene gestita esclusivamente dal nucleo del nodo.
- Una **coda globale dei rappresentanti** di tutti i processi sospesi su S , accessibile da ogni nucleo.



Esecuzione di una P sospensiva su un semaforo S (ValS=0) condiviso tra nodi virtuali:

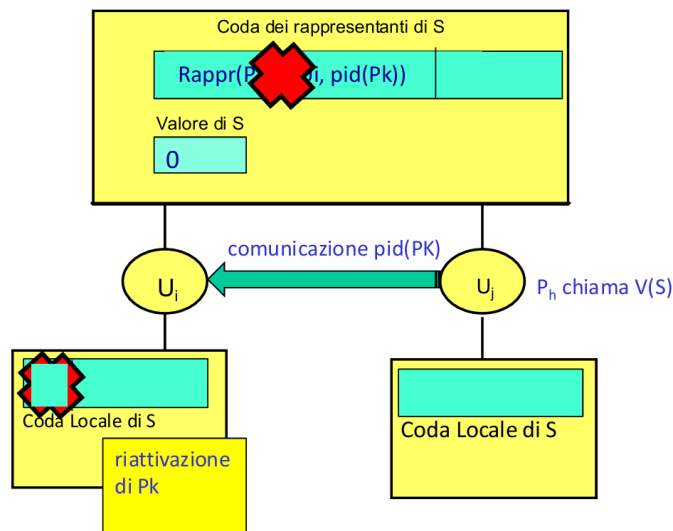
il nucleo del nodo U_i sul quale opera il processo P_k che ha chiamato la P(s) ,
 provvede a:

- inserire il rappresentante di P_k nella coda globale dei rappresentanti associata a S.
- inserire il descrittore di P_i nella coda locale a U_i associata a S.

Esecuzione di una V su un semaforo S (con rappresentanti in coda):

Assumendo che la V(S) sia chiamata dal processo P_h appartenente al nodo U_j :

- Viene verificata le coda dei rappresentanti di S: il primo elemento è il rappresentante del processo P_k che appartiene al nodo U_i , che viene eliminato dalla coda;
- il nucleo di U_j provvede a **comunicare tempestivamente**. l'identità del processo P_k risvegliato al nucleo di U_i ;
- Il nucleo di U_i provvede a riattivare il processo P_k , estraendo il suo descrittore dalla coda locale.



8.6.6 Comunicazione tra nuclei

La comunicazione tra U_j e U_i deve essere tempestiva, cioè: qualunque attività si stia svolgendo in U_i , occorre interromperla per notificare al nucleo di U_i che c'è un processo (P_k) locale a U_i da riattivare $\Rightarrow$ segnale di interruzione da U_j a U_i .



Il nucleo di U_j gestisce l'interruzione accedendo a un buffer in memoria comune dove U_i ha preventivamente scritto il pid del processo da risvegliare (P_k) e quindi estrae il descrittore di P_k dalla coda locale di S , portando il processo risvegliato nello stato ready.

```
1  \\Realizzazione P (modello a nuclei distinti)
2  void P(semaforo sem): {
3      if (<sem appartiene alla memoria privata>)
4          <come nel caso monoprocesso>
5      else {
6          lock(x);
7          <esecuzione della P con eventuale
8          sospensione del rappresentante del
9          processo nella coda di sem>;
10         unlock(x);
11     }
12
13  \\Realizzazione V (modello a nuclei distinti)
14
15  void V (semaforo sem) {
16      if (<sem appartiene alla memoria privata>)
17          <caso monoprocesso>
18      else {
19          lock(x);
20          if (<la coda non e vuota>)
21              if (<il processo appartiene al nodo del segnalante>)
22                  <caso monoprocesso>;
23          else { //semaforo condiviso
24              <estraz. processo dalla coda dei rappresentanti>;
25              <si determina il buffer di comunicazione con il nodo
26              cui il processo appartiene>;
27              if (<area occupata>) <si aspetta>;
28          else <si inserisce nel buffer l identificatore del
29              processo riattivato>;
30              <si invia interrupt al nodo cui appartiene il
```

```

31     processo>;}
32     else <si incrementa il valore del semaforo>;
33     unlock(x);
34 } }

```

8.7 Architettura Monoprocessore: realizzazione del nucleo, Modello a scambio di messaggi

Rispetto al kernel per il modello a memoria comune: il **supporto alla multi-programmazione** (gestione processi, scheduling ecc) può essere realizzato in modo **simile**;

inoltre, **diversa è la modalità di interazione** tra i processi concorrenti: invece di implementare strumenti di sincronizzazione (come il semaforo), il kernel implementa i meccanismi di comunicazione (es. canali e primitive).

8.7.1 Realizzazione delle primitive asincrone

Realizzazione delle primitive di comunicazione de del costruito port utilizzando gli strumenti di comunicazione offerti dal nucleo del sistema operativo.

Riferimento alle **primitive asincrone** in quanto più “primitive”: infatti, le primitive di tipo sincrone possono essere tradotte dal compilatore del linguaggio in termini di primitive asincrone.

Realizzazione in Architetture **mono/multielaboratore e architetture distribuite**.

Architetture mono e multielaboratore

Ipotesi:

- Tutti i messaggi scambiati tra i processi sono di un **unico tipo T** predefinito a livello di nucleo;
- Tutti i canali sono **da molti ad uni (port)** e quindi associati al processo ricevente;
- Essendo le primitive asincrone, ogni porta deve contenere un buffer di lunghezza indefinita.

```

1 typedef struct {
2     T informazione;
3     PID mittente;
4     messaggio *successivo;
5 } messaggio;

```

```

1 typedef struct {
2     messaggio *primo;
3     messaggio *ultimo;
4 } coda_di_messaggi;

```

```

1 void inserisci(messaggio *m, coda_di_messaggi c) {
2     if (c.primo == null) c.primo = m;
3     else c.ultimo -> successivo = m;
4     c.ultimo = m;

```

```

5     m->successivo = null;
6 }

```

```

1 messaggio *estrai(coda_di_essaggi c) {
2     messaggio *pun;
3     pun = c.primo;
4     c.primo = c.primo -> successivo;
5     if (c.primo == null) c.ultimo = null;
6     return pun;
7 }

```

```

1 boolena coda_vuota(coda_di_messaggi c) {
2     if(c.primo == null)
3         return true;
4     return false;
5 }
6
7 typedef struct {
8     coda_di_messaggi coda;
9     p_porta puntatore;
10 } des_porta;
11
12 typedef des_porta *p_porta;
13
14 // descrittore del processo
15 typedef struct {
16     p_porta porte_processo[M];
17     PID nume;
18     modalita_di_servizio servizio;
19     tipo_contesto contesto;
20 //Il campo stato registra se il processo e attivo oppure
21 //↳bloccato; in questo caso tiene traccia delle porte
22 //↳sulle quali e in attesa.
23     tipo_stato stato;
24     PID padre;
25     int N_figli;
26     des_figlio prole[max_figli];
27     p_des successivo;
28 } des_processo;

```

```

1 boolean bloccato_su(p_des p, int ip) {
2     <testa il campo stato nel descrittore del processo di
3     ↳cui p e il puntatore e restituisce il valore true
4     ↳se il processo risulta bloccato in attesa di
5     ↳ricevere messaggi
6     dalla porta il cui indice nel campo porte_processo e ip
7     ↳>;
8 }

```

```

1 void blocca_su(int ip) {
2     <modifica il campo stato del descrittore del
        ↳processo_in_esecuzione per indicare che lo stesso
        ↳si blocca in attesa di messaggi dalla porta il cui
        ↳indice nel campo porte_processo e ip>;
3 }

1 // verifica la presenza di messaggi
2 void testa_porta(int ip) {
3     p_des esec = processo_in_esecuzione;
4     p_porta pr = esec->porte_processo[ip];
5     if(coda_vuota(pr->coda)) { //sospensione
6         // sospensione
7         blocca_su(ip);
8         assegnazione_CPU; // context switch
9     }
10 }

11
12 messaggio *estrai_da_porta(int ip) {
13     messaggio *m;
14     p_des = processo_in_esecuzione;
15     p_porta pr = esec->porte_processo[ip];
16     m = estrai(pr->coda);
17     return m;
18 }

19
20 void inserisci_porta(messaggio *m, PID proc, int ip) {
21     p_des destinatario = descrittore(proc);
22     p_porta pr = destinatario->porte_processo[ip];
23     inserisci(m, pr->coda);
24     //porta il processo destinatario nello stato di ready
25     if(bloccato_su(destinatario, ip)) attiva(destinatario);
26 }

1 void send(T inf, PID proc, int ip) {
2     messaggio *m = new messaggio;
3     m->informazione = inf;
4     m->mittente = processo_in_esecuzione;
5     inserisci_porta(m, proc, ip);
6 }

7
8 void receive(T *inf, PID *proc, int ip) {
9     messaggio *m;
10    testa_porta(ip);
11    m = estrai_da_porta(ip);
12    *proc = m->mittente;
13    *inf = m->informazione;
14 }

```

Ricezione su più canali

```
1 int test_porte(int ip[], int n){
2     p_porta pr; int ris=-1; int indice_porta;
3     p_des esec=processo_in_esecuzione;
4     for (int i=0; i<n; i++){
5         indice_porta=ip[i];
6         pr=esec->porte_processo[indice_porta];
7         if(coda_vuota(pr->coda)) blocca_su(indice_porta);
8         else{
9             ris=indice_porta;
10            esec->stato= <processo attivo>;
11            break;
12        } }
13     if(ris==-1) assegnazione_CPU(); //tutte le porte vuote:
14     ↪sospensione
15     return ris; }
```

In questo caso la scansione procede in ordine di indice, quindi verrà selezionata la porta non vuota con l'indice minimo. Per realizzare una selezione non deterministica, l'ordine di scansione deve essere random.

```
1 int receive_any(T *inf, PID *proc, int ip[],int n){
2     messaggio * mes; int indice_porta;
3     do
4         indice_porta=test_porte(ip,n);
5     while(indice_porta==-1);
6     mes = estrai_da_porta(indice_porta);
7     proc = &(mes->mittente);
8     inf = &(mes ->informazione);
9     return indice_porta;
10 }
```

Architetture distribuite

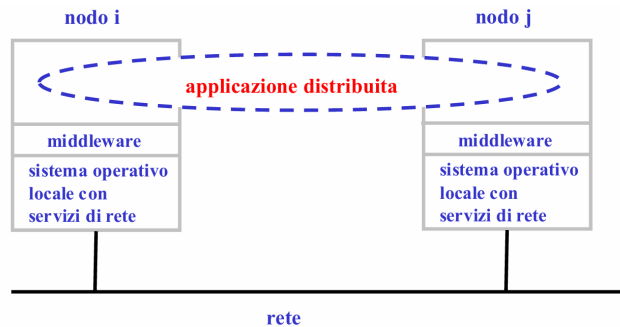
Insieme di nodi tra loro omogenei e tutti dotati dello stesso sistema operativo (stesso nucleo).

Scopo del sistema: gestire tutte le risorse nascondendo all'utente la loro distribuzione sulla rete.

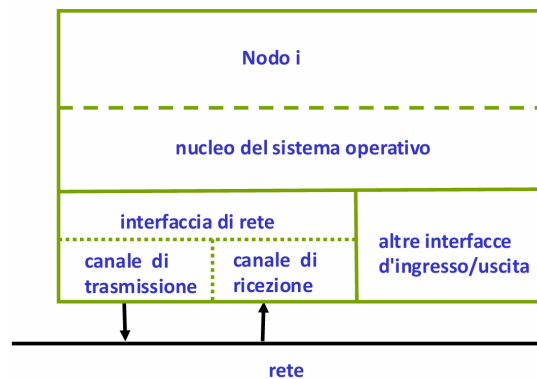
Sistemi operativi di rete (nos - Network operating Systems): insieme di nodi eterogenei, con sistemi operativi diversi e autonomi, nodo per nodo. Ogni nodo della rete è in grado di offrire servizi a clienti remoti presenti su altri nodi della rete.

Trasparenza della distribuzione delle risorse viene ottenuta mediante il **middleware**.

Architetture distribuite - Network Operating System



Sistemi DOS - Interfaccia di rete



Pacchetti, interfacce, canali

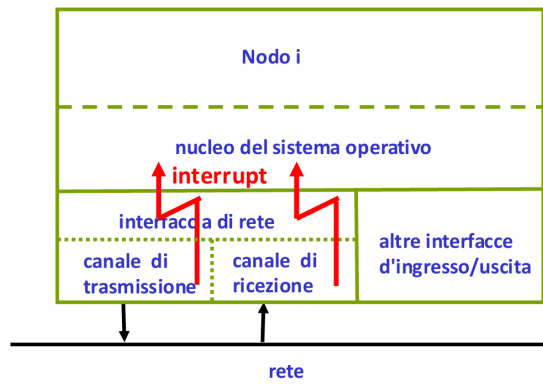
L'unità di trasmissione tra nodi è il **pacchetto**. La struttura del pacchetto dipende dalla realizzazione (cioè dai protocolli di comunicazione adottati dalla rete).

L'interfaccia di rete è strutturata in 2 parti (canali): uno per la **trasmissione** dei messaggi, uno per la **ricezione**.

Il **canale di trasmissione** viene acceduto per realizzare l'invio di pacchetti; ad esso sono associati:

- una **coda di pacchetti**, nella quale ogni processo sender deposita il proprio pacchetto se il canale è occupato da un altro processo.
- un registro **buffer** della dimensione di un pacchetto.

Il **canale di ricezione** viene acceduto per realizzare la ricezione di pacchetti; ad esso è associato un registro **buffer**, nel quale viene depositato ogni pacchetto inviato al nodo. Arrivo/partenza di pacchetti verso/da canali di ricezione/-trasmissione vengono notificati tramite **interruzioni**.



Il pacchetto è l'informazione che viene trasmessa attraverso il canale: contiene, oltre al messaggio (inf, mittente), anche le informazioni relative al processo destinatario e alla porta di destinazione. La struttura del pacchetto dipende dalla realizzazione.

```

1 void invia_pacchetto (packet p) {
2     if (<canale di trasmissione occupato>)
3         packet_queue.inserisci(p);
4     else {
5         <inserimento di p nel registro buffer del canale>;
6         <attivazione trasmissione>; //al termine: interruzione
7     }
8 }

```

```

1 void tx_interrupt_handler() {
2     packet p;
3     salvataggio_stato();
4     if (!packet_queue.vuota()){
5         p = packet_queue.estrai();
6         <inserimento di p nel registro buffer del canale>;
7         <attivazione trasmissione>; }
8     ripristino_stato();
9 }

```

```

1 typedef struct {
2     int indice_nodo;
3     int PID_locale;
4 } PID;
5
6 void send(T inf, PID proc, int ip){
7     if (proc.indice_nodo== nome_nodo)
8         local_send(inf, proc, ip);
9     else remote_send(inf, proc, ip); }
10
11 void remote_send(T inf, PID proc, int ip){
12     packet p;
13     PID mit=processo_in_esecuzione;
14     int indice_nodo_destinatario= proc.indice_nodo;
15     <vengono riempiti i vari campi del pacchetto p: in
        ↳ particolare, viene inserito nel campo relativo al
        ↳ nodo a cui inviare il pacchetto il valore
        ↳ indice_nodo_destinatario. Inoltre, nel campo del
        ↳ pacchetto destinato a contenere le informazioni da
        ↳ inviare vengono inseriti il nome mit del processo che
        ↳ invia, e i tre parametri della funzione inf, proc, e
        ↳ ip>;
16     invia_pacchetto(p); }

```

9 Calcolo parallelo

La capacità di elaborazione parallela è ormai una caratteristica di moltissime architetture: GPU, FPGA, sistemi distribuiti, cluster, sistemi di calcolo ad elevate prestazioni (High Performance Computing systems).

La disponibilità di hardware parallelo rende possibile lo sviluppo di applicazioni che sfruttino contemporaneamente e in modo ottimale le risorse di calcolo a disposizione.

Per sfruttare al meglio il parallelismo disponibile a livello HW gli sviluppatori possono utilizzare tecnologie ad hoc che permettono di esprimere, gestire e massimizzare la capacità di esecuzione parallela.

Architetture ad elevate prestazioni per High performance Computing

ILP e Hardware multithreading hanno permesso un miglioramento delle prestazioni dei processori, tuttavia tali meccanismi sono trasparenti per gli sviluppatori dei programmi. → modello **Von Neumann Esteso**

Nei sistemi HPC, invece, il parallelismo disponibile per l'esecuzione dei programmi è visibile al programmatore, che deve progettare il software in modo da sfruttare al meglio tutte le risorse computazionali a disposizione. → architetture **non Von Neumann**

9.1 Sistemi MIMD

Asincronicità delle attività nei diversi nodi: più processori, ognuno con la sua propria control unit.

Ogni CPU esegue una sequenza di istruzioni diversa dagli altri.

Necessità di interazione tra nodi.

2 categorie di sistemi:

- **shared memory** (UMA, NUMA multiprocessor/multicore)
- **distributed memory** (cluster, supercomputers)

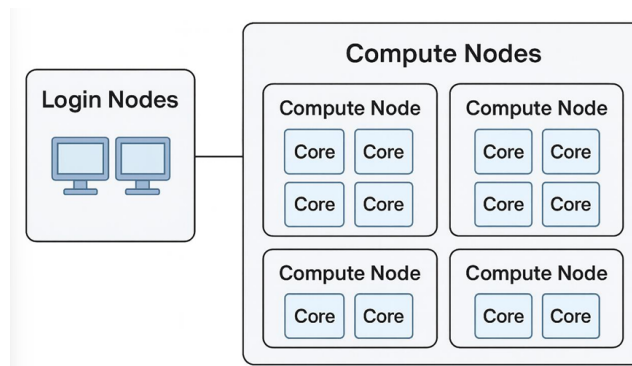
Necessario supporto a livello architetturale per consentire l'interazione:

- memoria e cache -> problema cache coherence
- reti di interconnessione ad alte prestazioni: bassa latenza e banda elevata

Nodi in Cluster HPC

I nodi di un cluster HPC sono generalmente sistemi multiprocessore, ognuno contenente una molteplicità di core.

L'insieme dei nodi di un cluster HPC viene suddiviso funzionalmente in **Compute nodes**: nodi dedicati all'esecuzione di processi applicativi; **Login nodes**: nodi utilizzati per l'interazione con gli utenti;



9.2 Sistemi operativi per HPC

L'obiettivo principale dell'High Performance Computing è massimizzare le prestazioni delle applicazioni parallele. Il sistema operativo che gestisce le risorse di ogni nodo di calcolo deve cercare di **massimizzare la percentuale di tempo di CPU assegnato alle applicazioni**. Normalmente (in contesti <tradizionali>, non HPC), il sistema operativo adotta politiche di gestione delle risorse, ed in particolare della CPU, guidate da principi di interattività ed equità:

- Scheduler pre-emptive → Frequenti cambi di contesto → overhead
- Memoria virtuale → Impatto della paginazione sulle prestazioni
- Accesso a periferiche e file system

Tutti questi meccanismi sottraggono tempo di CPU alle applicazioni. **RU-**

MORE DI SISTEMA: l'influenza dei meccanismi del sistema operativo (interruzioni, cambi di contesto, paginazione, servizi del SO non necessari) sull'esecuzione dei processi applicativi può essere significativamente negativa.

Il Sistema operativo deve essere concepito per abbattere il rumore di sistema.

- Light Weight Kernel, LWK
- Full Weight Kernel, FWK
- Multi-kernel

9.2.1 Light Weight Kernel

I core di ogni nodo di calcolo sono governati da un kernel ad hoc, appositamente progettato in modo che il rumore sia minimo.

Caratteristiche tipiche di questo approccio sono:

- Un solo kernel per la gestione di tutti i core (v. SMP)
- Multiprogrammazione molto limitata, no preemption
- Nessuna gestione di I/O

- Memoria virtuale assente

Non avendo dei servizi standard, il problema principale è la manutenzione e il costo di sviluppo del SO.

9.2.2 Full Weight Kernel

Il kernel che gestisce ogni nodo deriva dal kernel di un SO commerciale (es: Linux), opportunamente adattato.

Caratteristiche tipiche di questo approccio sono:

- Un solo kernel per la gestione di tutti i core (v. SMP)
- Disponibilità di servizi standard → compatibilità/portabilità del software
- Scheduler prioritario con classi a priorità alta specifica per applicazioni HPC:
 - Senza preemption → abbattimento del rumore dovuto ai cambi di contesto
 - Spesso politiche orientate alla sincronizzazione tra processi applicativi (es: Gang scheduling)
- Paginazione basata su **<huge pages>**: si passa dai 4KB dei sistemi tradizionali a pagine di 1MB-1GB. Riduzione drastica dei page fault → abbattimento del rumore imputabile a paginazione.

Rispetto all'LWK è presente un rumore più alto dovuto al mantenimento di servizi in molti casi non necessari (es: I/O).

9.2.3 Multi-kernel

Per sfruttare i benefici di entrambi gli approcci LWK e FWK, è possibile adottare un modello basato su più kernel per nodo.

Ad esempio IHK, utilizza 2 kernel per nodo: **Un kernel linux completo** (base kernel) in esecuzione su uno (o più) core, dedicato all'esecuzione dei servizi standard di SO; **Un LWK minimale** (McKernel) in esecuzione sui rimanenti core, dedicati al calcolo parallelo.

Ogni volta che un processo applicativo richiede un servizio di sistema operativo (es: I/O, accesso al file system, ecc) il kernel LWK comunica con il kernel Linux, che si attiva per servire la richiesta. In questo modo abbiamo un abbattimento quasi totale del rumore nei core dedicati al calcolo e come svantaggio abbiamo un numero ridotto di core dedicati al calcolo perché i core utilizzati dal kernel linux non sono disponibili per i processi applicativi.

9.3 Sviluppo di applicazioni parallele in sistemi HPC

La necessità di adattare il codice alle risorse HPC per sfruttare efficacemente il parallelismo disponibile a livello HW, impone al programmatore di trasformare i propri programmi seriali in codice parallelo.

- **parallelizzazione automatica:** esistono compilatori <parallelizzanti> che traducono codice sequenziale in codice parallelo (es: traduzione parallela di `clcli`). Normalmente le prestazioni ottenibili con questo approccio sono poco soddisfacenti.
- **parallelizzazione esplicita:** il programmatore esprime il parallelismo nel codice dei programmi che sviluppa, con la finalità di sfruttare nel modo migliore possibile le risorse computazionali HW: linguaggi e librerie per il calcolo parallelo. Un programma in esecuzione è un insieme di processi, ognuno in esecuzione su un nodo fisico distinto, che all'occorrenza possono interagire (o tramite lo scambio di messaggi o tramite memoria condivisa).

Nei sistemi HPC chi lancia l'esecuzione di un programma parallelo può contare sulla disponibilità esclusiva di un insieme di nodi di elaborazione da usare per l'esecuzione (differenza con OS multithreading/multitasking). **Task paral-**

lelism: In sistemi di questa categoria il parallelismo si ottiene distribuendo task diversi a processi diversi; ogni processo è assegnato a una CPU a sua completa disposizione.

9.4 Sviluppo di Software Parallelo

Nello sviluppo di sw parallelo è molto utilizzato il paradigma **SPMD** (<Single Program Multiple Data>): ogni core esegue lo stesso programma; per differenziare il codice eseguito dai diversi nodi, si sfrutta il branching condizionale.

```
1 void run(int nodo) {
2     if (nodo==0) // nodo e il rank associato al processo
3         master(); //codice eseguito dal task sul nodo 0
4     else
5         slave(nodo); //codice di task su nodi diversi da 0
6 }
```

Parallelizzazione In molti casi si parte da una versione seriale del programma che si desidera rendere più veloce nell'esecuzione, distribuendo il lavoro a più processi, ciascuno in esecuzione su un'unità di elaborazione diversa. La versione seriale viene parallelizzata, ovvero trasformata in un insieme di processi paralleli, per essere eseguita in un'architettura HPC. La parallelizzazione può essere un compito complesso, ma in alcuni casi, il tipo di algoritmo eseguito rende molto semplice la transizione seriale -> parallelo (algoritmi <embarassingly parallel>)

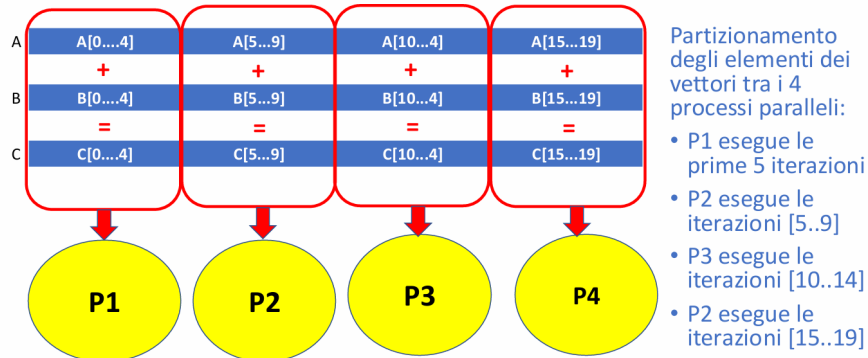
9.4.1 Programmi embarrassingly parallel

La parallelizzazione è ottenuta dividendo il lavoro tra i processi in modo che ognuno esegua le stesse istruzioni su dati diversi in modo indipendente dagli altri. I processi ottenuti sono tra loro **indipendenti**.

```
1 //Esempio: parallelizzaz della somma di due vettori A e B
2   int A[20], B[20], C[20];
3   ...
4   for(i=0; i<20; i++){
5       C[i] = A[i]+ B[i];
6   //ogni iterazione puo essere eseguita in modo indipendente
   ↪dalle altre
```

Programmi embarrassingly parallel

Quindi, se disponiamo di 4 nodi, **allochiamo su ogni nodo un processo**:



Idealmente: il tempo di esecuzione dovrebbe essere $\frac{1}{4}$ del tempo sequenziale.

Il problema è che non tutti gli algoritmi è embarrassingly parallel, la parallelizzazione impone punti di sincronizzazione/comunicazione tra processi paralleli.

Sviluppo di un programma parallelo Bisogna dividere il lavoro tra i processi, in modo da:

- bilanciare il carico tra i nodi
- minimizzare le interazioni

Inoltre, progettare la sincronizzazione tra processi e progettare la comunicazione tra processi;

I problemi di progettazione sono correlati, nei **sistemi shared memory** i processi comunicano grazie alla sincronizzazione; nei **sistemi distributed memory** i processi si sincronizzano per mezzo della comunicazione.

Risoluzione di un problema in minor tempo

Sia N la dimensione del problema; sia p il numero di processori a disposizione:
→ decomposizione del problema in tanti sottoproblemi (possibilmente indipendenti) di dimensione N/p

9.5 Prestazioni del software parallelo

Per valutare il vantaggio derivante dall'esecuzione di programmi paralleli anche nei sistemi HPC si utilizzano le metriche:

- Speedup
- Efficienza

9.5.1 Speedup

Lo speedup è una funzione di p e N che misura quanto è più veloce la versione parallela su p nodi rispetto alla versione sequenziale, ovvero il guadagno derivante dalla parallelizzazione.

$$S(p, N) = \frac{T(1, N)}{T(p, N)}$$

- N è la dimensione del problema (e.g. la dimensione dell'insieme dei dati) e p è il numero di processi utilizzati per il calcolo
- $T(1, N)$ è il tempo di esecuzione del programma nella sua versione sequenziale (su un solo nodo)
- $T(p, N)$ è il tempo di esecuzione del programma nella sua versione parallela su p nodi.

Assumiamo di usare p processori per l'esecuzione parallela; nel caso ideale:

$$T(1, N) = p \cdot T(p, N) \rightarrow S = p$$

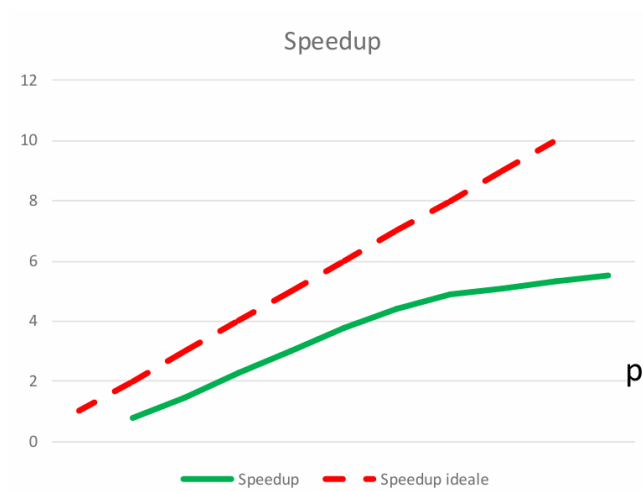
ovvero: nel caso ideale il tempo di esecuzione della versione parallela è $1/p$ del tempo nella versione sequenziale:

$$T(p, N) = \frac{T(1, N)}{p}$$

nei casi non ideali:

$$T(p, N) = \frac{T(1, N)}{p} + T_{\text{overhead}} \rightarrow S < p$$

T -overhead è dovuto a parti non parallelizzabili, creazione/allocazione processi, comunicazione/sincronizzazione, distribuzione non bilanciata del lavoro.



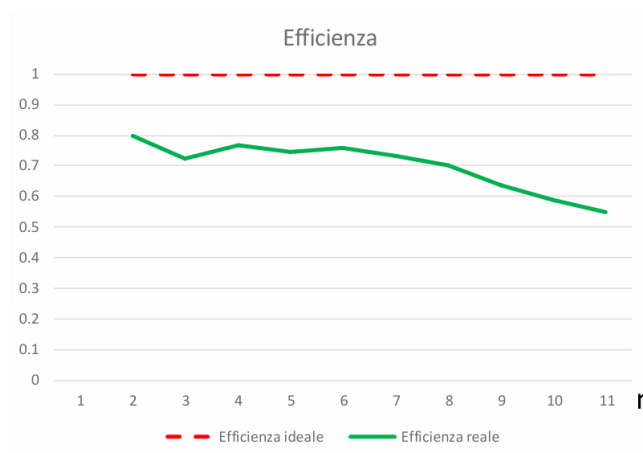
9.5.2 Efficienza

$$E = \frac{S}{p}$$

dove S è lo speedup e p è il numero di processi utilizzati.

- Nel caso ideale $S=p \rightarrow E=1$
- Casi reali: $S < p \rightarrow E < 1$

se un programma mantiene la stessa efficienza al variare del numero di processori utilizzati e/o al variare della quantità di dati da elaborare, si dice che è **idealmente scalabile**



9.6 Legge di Amdahl (1967)

$$S(p, N) = \frac{T(1, N)}{T(p, N)}$$

Dato un programma, osserviamo che, in generale, non tutto il suo codice può essere parallelizzabile. Pertanto il tempo ottenuto dall'esecuzione sequenziale si può quindi esprimere come segue:

$$T(1, N) = T_{seq}(1, N) + T_{par}(1, N)$$

dove T_{seq} è la parte sequenziale e T_{par} è il tempo imputabile alla parte parallelizzabile.

Definendo

$$r = \frac{T_{seq}(1, N)}{T(1, N)}$$

dove $r \in [0, 1]$ ossia la frazione del tempo totale spesa nella parte non parallelizzabile. Chiaramente:

$$T_{seq}(1, N) = r.T(1, N)$$

e quindi:

$$T_{par}(1, N) = T(1, N) - T_{seq}(1, N) = (1 - r).T(1, N)$$

La parte sequenziale non beneficia dell'aumento del numero di processori:

$$T_{seq}(p, N) = T_{seq}(1, N) = \text{costante} = T_{seq}$$

La parte parallelizzabile viene suddivisa equamente:

$$T_{par}(p, N) = \frac{T_{par}(1, N)}{p} = \frac{(1 - r).T(1, N)}{p}$$

Il tempo totale parallelo è quindi:

$$T(p, N) = T_{seq}(p, N) + T_{par}(p, N) = r.T(1, N) + \frac{(1 - r).T(1, N)}{p}$$

Speedup diventa per definizione:

$$S(p, N) = \frac{T(1, N)}{T(p, N)}$$

sostituendo l'espressione di $T(p, N)$:

$$S(p, N) = \frac{T(1, N)}{r.T(1, N) + (1 - r) * \frac{T(1, N)}{p}} = S(p, N) = \frac{1}{r + \frac{1-r}{p}}$$

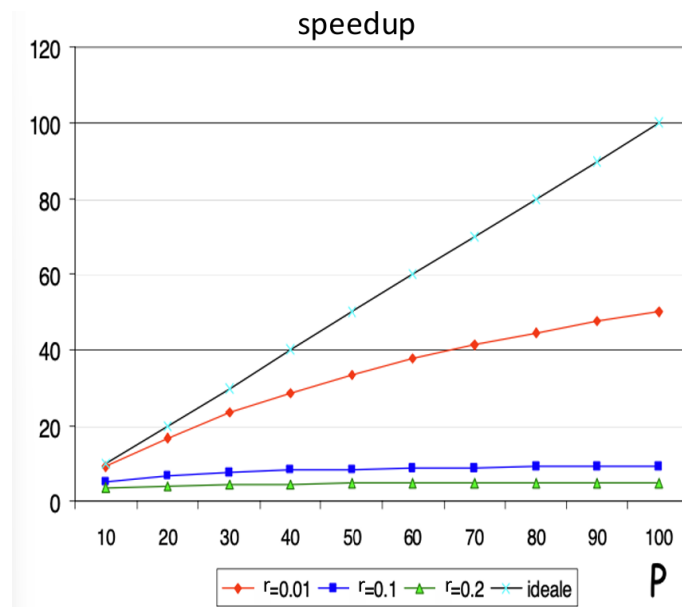
$$S(p, N) = \frac{1}{r + \frac{1-r}{p}}$$

Osserviamo un **comportamento asintotico** su $S(p,N)$ ponendo il limite dello speedup al crescere del numero di processori:

$$\lim_{p \rightarrow \infty} S(p, N) = \lim_{p \rightarrow \infty} \frac{1}{r + \frac{1-r}{p}} = \frac{1}{r}$$

Dunque, anche con un numero arbitrariamente grande di processori, lo speedup massimo è limitato dalla frazione sequenziale del programma:

$$S_{\max} = \frac{1}{r}$$

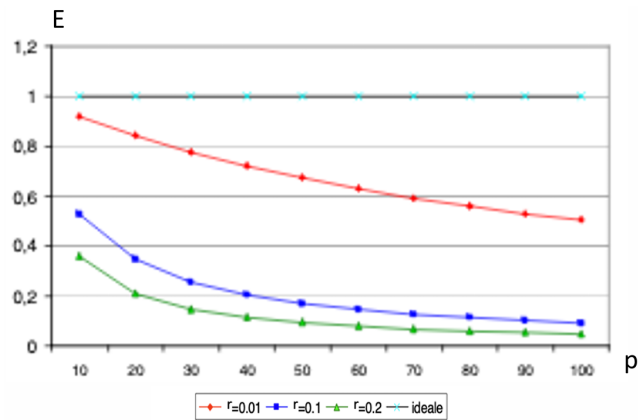


Nel caso non ideale, l'efficienza si esprime:

$$E = \frac{S}{p} = \frac{1}{r + \frac{(1-r)}{p}} * \frac{1}{p} = \frac{1}{pr - r + 1}$$

anche in questo caso il comportamento della funzione $E(p)$ è asintotico $\lim_{p \rightarrow \infty} E(p) = 0$

la legge di Amdahl mostra che all'aumentare del numero dei nodi, l'efficienza cala, tendendo asintoticamente a 0; nel caso **ideale** al crescere di p , l'efficienza si mantiene costante $\lim_{p \rightarrow \infty} E_P = \lim_{p \rightarrow \infty} \frac{S_P}{P} = 0$



Scalabilità strong misura la variazione di efficienza/speedup al variare del numero di processi.

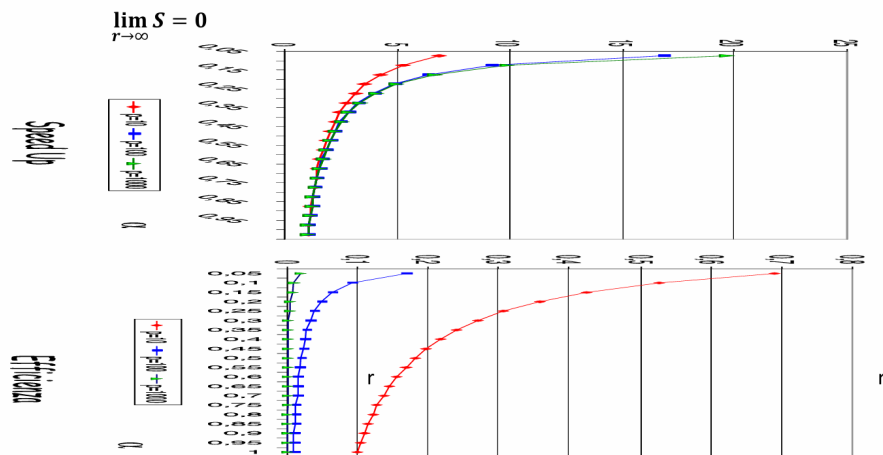


Figure 5: Impatto della componente non parallelizzabile r

Le prestazioni di un programma parallelo dipendono, oltre che dal numero p di CPU a disposizione, anche dalla dimensione N del problema da risolvere (ovvero, dalla quantità di dati da elaborare).

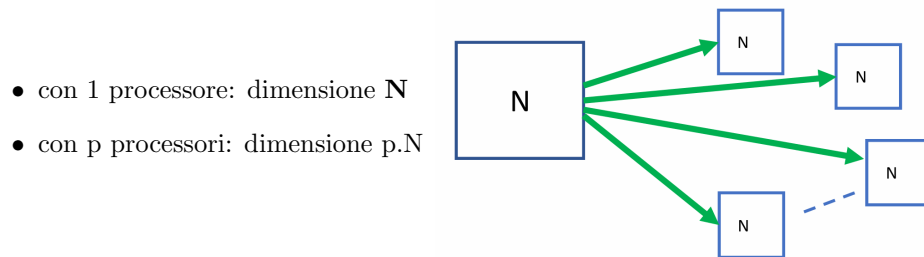
$$\text{TEMPO DI ESECUZIONE} = T(p, N)$$

Es. sistema di equazioni lineari $\rightarrow$ dimensione della matrice dei coefficienti e termini noti In alcuni casi, la dimensione del problema è fissata, in altri è variabile.

9.6.1 Scalabilità strong e weak

- si può valutare quanto si può guadagnare nella sua soluzione mantenendo costante N e aumentando il numero di processori (**scalabilità strong**).
- si può valutare se aumentando la dimensione possiamo risolvere il problema nello stesso tempo (**scalabilità weak**).

Scalabilità weak: Dato un problema di dimensione N risolto su un nodo, aumento linearmente la dimensione del problema al crescere del numero dei processori, mantenendo pari a N il carico di ogni processore



Il concetto di scalabilità, in generale, esprime per un programma parallelo la capacità di mantenere invariata l'efficienza al variare del numero dei nodi e/o della dimensione del problema.

Scalabilità strong: la dimensione del problema è costante, si valuta l'efficienza al crescere del numero dei nodi. (V. legge Amdahl). In questo caso:

- il lavoro totale da eseguire è **costante**
- il lavoro del singolo nodo diminuisce al crescere del numero dei nodi

Se la dimensione del problema aumenta, per mantenere tempi paragonabili **aumentiamo il numero dei nodi**: **Scalabilità weak:** mantenendo costante il carico di lavoro per singolo nodo, si valutano efficienza/speedup al crescere del numero dei nodi. **La dimensione del problema aumenta in proporzione al numero p di nodi utilizzati.**

9.6.2 Legge di Gustafson(1988)

Dato un problema di dimensione $p.N$ consideriamo la sua soluzione parallela con p processori. In generale, una frazione r del tempo di calcolo è imputabile alla parte di lavoro non parallelizzabile.

Tempo totale $T(p, p.N) = T_{seq} + T_{par}$ definendo r la frazione del tempo di esecuzione non parallelizzabile: $r = \frac{T_{seq}}{T(p, p.N)}$, allora

- $T_{seq} = r.T(p, p.N)$
- $T_{par} = T(p, p.N) - r.T(p, p.N) = (1 - r).T(p, p.N)$

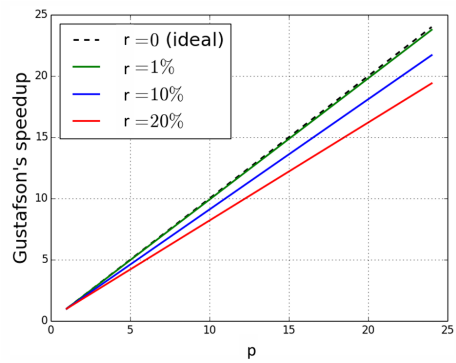
Risolvendo lo stesso problema con un solo processore (T_{seq} non dipende da p):

$$T(1, p.N) = T_{seq} + p.T_{par} = r.T(p, p.N) + p.(1-r).T(p, p.N) = T(p, p.N).(r + p.(1-r))$$

$$\text{Lo speedup diventa: } \mathbf{S(p,p.N)} = \frac{T(1,p.N)}{T(p,p.N)} = \frac{T(p,p.N).(r+p.(1-r))}{T(p,p.N)} = (r + p.(1-r))$$

$$\boxed{S(p, p.N) = r + p.(1-r)}$$

Assegnando ad ogni processore un workload costante $(1-r)$, lo speedup cresce linearmente con il numero dei processori



10 Sviluppo di programmi paralleli

A seconda delle caratteristiche architetturali del sistema utilizzato, nello sviluppo di software parallelo è possibile fare riferimento a due modelli di interazione:

- Se i nodi dell'architettura non condividono memoria (es. cluster HPC), lo sviluppo dei programmi paralleli si fonda sul modello a scambio di messaggi. In questo ambito lo standard è rappresentato dalle librerie MPI.
- Se, invece, è prevista la condivisione di memoria tra tutti i nodi (es. sistemi multicore/multiprocessor), il modello di interazione tra processi è a memoria comune. In questo caso, è possibile utilizzare OpenMP.

10.1 MPI: Message Passing Interface

MPI è uno standard di fatto che stabilisce un protocollo per la comunicazione tra processi in sistemi paralleli. A partire dal 1992, il protocollo si è sviluppato e diffuso, definendo *Language Independent Specifications* (LIS) alle quali tutte le realizzazioni si uniformano.

Lo standard è implementato in diverse librerie di funzioni C/C++, Fortran, Python per lo sviluppo di programmi paralleli nel modello a scambio di messaggi.

Implementazioni di MPI MPICH OpenMPI IntelMPI

Lo standard garantisce portabilità sulle diverse architetture e implementazioni.

10.1.1 Caratteristiche di MPI

- Basato sul paradigma SPMD: l'esecuzione di una applicazione parallela MPI viene portata avanti da molteplici istanze dello stesso programma, ognuna in esecuzione contemporanea su un nodo distinto.
- Offre un ricco set di funzioni per comunicazione punto-punto e collettiva, con semantiche sincrone e asincrone.
- Offre strumenti per *data partitioning* (distribuzione di dati tra nodi) e *data collecting* (raccolta di risultati).
- Gestione dei processi statica e implicita: il grado di parallelismo viene definito a tempo di caricamento.

```
1 #include <mpi.h>
2
3 int main() {
4     // parte sequenziale
5     MPI_Init(NULL, NULL);
6     // codice parallelo con chiamate MPI
7     MPI_Finalize();
8     // parte sequenziale
9 }
```

10.1.2 MPI_Init e MPI_Finalize

- `int MPI_Init(int *argc, char ***argv);`
- `int MPI_Finalize(void);`

Entrambe restituiscono `MPI_SUCCESS` in caso di successo.

10.2 Communicator

Il communicator è un'astrazione che definisce un dominio di comunicazione, ovvero un insieme di processi che possono comunicare tra loro.

Il communicator di default è `MPI_COMM_WORLD`, al quale appartengono tutti i processi creati con `mpirun`. È possibile creare nuovi communicator con funzioni come:

- `MPI_Comm_create`
- `MPI_Comm_spawn`

10.2.1 Size e Rank

- `MPI_Comm_size(comm, &size)`: restituisce il numero di processi nel communicator.
- `MPI_Comm_rank(comm, &rank)`: restituisce l'identificativo del processo all'interno del communicator.

10.3 Funzioni di comunicazione

MPI definisce primitive di comunicazione:

- Comunicazione singola (`MPI_Send`, `MPI_Recv`)
- Comunicazione collettiva (`MPI_Reduce`, `MPI_Bcast`, `MPI_Scatter`, `MPI_Gather`, `MPI_AllGather`)

10.3.1 MPI_Send

```
int MPI_Send(const void* buffer, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm);
```

- `buffer`: indirizzo del messaggio
- `count`: numero di elementi
- `datatype`: tipo degli elementi
- `dest`: rank del destinatario
- `tag`: etichetta della comunicazione

- `comm`: communicator

`MPI_Send` può essere sia sincrona che asincrona, dipendentemente dall'implementazione. La primitiva è `<buffer-safe>` ovvero: lo standard stabilisce che `MPI_Send` blocchi il processo sender almeno fino a quando il messaggio non viene prelevato dal buffer per essere inviato al destinatario; pertanto, al termine della `MPI_Send` si può eventualmente riutilizzare il buffer in maniera `<safe>`.

`MPI_Ssend(buffer, count, datatype, dest, tag, comm); sync`

`MPI_Isend(buffer, count, datatype, dest, tag, comm); async`

10.3.2 `MPI_Recv`

```
int MPI_Recv(void* buffer, int count, MPI_Datatype datatype, int source,
int tag, MPI_Comm comm, MPI_Status *status);
```

Riceve un messaggio da un processo mittente. `MPI_Recv` è bloccante.

`int MPI_Irecv(..)` non è bloccante: se il messaggio non è arrivato, ritorna immediatamente.

10.4 Primitive collettive

`MPI_Reduce` Permette di raccogliere e combinare valori da più processi in un unico risultato, applicando un'operazione (es. somma, prodotto, massimo).

`MPI_Bcast` Distribuisce un dato da un processo a tutti gli altri del communicator.

`MPI_Scatter` Distribuisce blocchi di dati da un processo a tutti gli altri.

`MPI_Gather` Raccoglie blocchi di dati da tutti i processi in un unico processo collettore.

11 Programmazione parallela su sistemi shared memory

11.1 Sistemi HPC a memoria condivisa

Quando l'architettura prevede la condivisione di memoria tra nodi, è possibile utilizzare il modello di interazione tra processi a **memoria comune**. In questo caso, a seconda del modello architetturale, esistono tecnologie di programmazione diverse:

- **sistemi multicore/multiprocessor**: **OpenMP**, pthreads ecc.
- **GP-GPU**: esistono librerie specifiche che consentono lo sviluppo di programmi destinati ad eseguire su GPU: CUDA (libreria proprietaria, tecnologia NVidia), **openCL**.

11.1.1 OpenMP: Open specifications for Multi-Processing

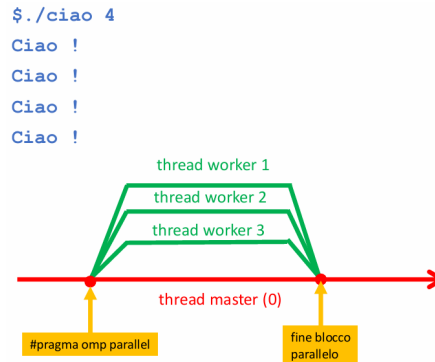
È una libreria che permette di parallelizzare codice C con un approccio dichiarativo.

- Gestione dei thread paralleli
- Informazioni sull'ambiente di esecuzione
- Definizione della visibilità delle variabili (**shared**, **private**, **firstprivate**)
- Sincronizzazione dei thread (sezioni critiche, barriere, ecc.)

Direttive per il compilatore **# pragma omp**

Esempio di creazione di thread:

```
1 #pragma omp parallel
   ↳ num_threads(n)
2 {   printf("Ciao!\n"); }
3 /*
4 gcc -fopenmp -o ciao hello.c
5 ./ciao 4
6 La printf viene eseguita da 4
7 thread paralleli: il processo
8 iniziale (Master) + altri 3
9 (Workers) creati per effetto
10 della direttiva */
```



Attraverso la clausola **num_threads**, si specifica il **grado di parallelismo**, cioè il numero dei thread in esecuzione parallela; ogni processo esegue il blocco parallelo (printf).

La creazione dei thread paralleli è implicita e avviene per effetto della direttiva:

`pragma omp parallel..`; questo è un modello puramente **cobegin-coend**. Al termine del blocco parallelo il processo master attende che tutti i workers siano terminati (**sincronizzazione implicita**).

11.2 Clausole principali

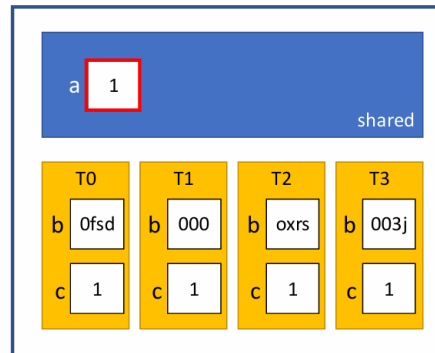
Nel comando # `pragma omp parallel` [eventuali clausole]

- `num_threads(N)`: numero di thread paralleli (specifica il **grado di parallelismo**)
- `shared, private, firstprivate`: scope delle variabili
- `if`: parallelizzazione condizionale
- `for`: parallelizzazione di cicli
- `reduction`: operazioni di reduce
- `schedule`: gestione della distribuzione del workload

11.2.1 Scope delle variabili

Di default ogni variabile definita esternamente a # `pragma omp parallel` è condivisa tra i thread paralleli, e ogni variabile definita **internamente** al blocco parallelo è locale al singolo thread. Si può modificare il default tramite le clausole **shared, private, firstprivate**.

```
#include <omp.h>
#include <stdio.h>
int main( int argc, char *argv[] )
{
    int a=1, b=1, c=1;
    #pragma omp parallel shared(a) \
        private(b) firstprivate(c)
    {
        a++;
        b++;
        c++;
    }
}
```



a,b,c sarebbero shared (default), ma, per effetto delle clausole:

- **private(b)**: b è privata; ogni processo ne usa una istanza locale non inizializzata;
- **firstprivate(c)**: ogni processo usa una copia privata di c inizializzata al valore che c aveva prima di `pragma`
- **shared(a)**: a è condivisa tra tutti i thread paralleli (è il default → la clausola è superflua).

Mutua esclusione

L'accesso a variabili shared da parte di thread paralleli va sincronizzato, quindi è necessario imporre la **mutua esclusione** nell'esecuzione di sezioni critiche.

```
1 main() {  
2     int Risultato=0;  
3     # pragma omp parallel num_threads(20) {  
4         int myRis;  
5         myRis=Calcolo(..);  
6         #pragma omp critical {  
7             //Il blocco di istruzioni succ. alla direttiva  
8             //viene eseguito da un solo processo alla volta.  
9             Risultato += myRis; //sezione critica  
10        }  
    }
```

Il grado di parallelismo viene deciso a runtime. Ci sono funzioni per gestire informazioni sui thread in esecuzione:

- `omp_get_num_threads()` numero di thread paralleli da utilizzare all'interno di un blocco parallelo.
- `omp_get_thread_num()` restituisce il rank del thread che lo invoca (0 è il master).
- `omp_set_num_threads(n)` imposta a n il numero di thread paralleli nei successivi blocchi paralleli.

11.2.2 Clausola reduction

Associa a un blocco parallelo una variabile condivisa `<var>` che viene utilizzata per memorizzare il risultato di un'operazione `<op>` di reduce:

Aggregazione automatica dei risultati:

```
1 #pragma omp parallel ... reduction(<op>: <Var>)  
2 {  
3     ...  
4     <Var><op>=<espressione>;  
5     //ris += Trap(a, b, n);  
6 }
```

L'espressione viene **valutata in parallelo**; l'aggiornamento di `<Var>` avviene in **mutua esclusione**.

11.2.3 Clausola if

É possibile condizionare la parallelizzazione al verificarsi di una condizione con la clausola `if`.

```
#pragma omp parallel if (t > 10) num_threads(t)
```

eseguita in parallelo da t threads, solo se `argv[1]>10` altrimenti viene ignorato il pragma.

11.2.4 Direttiva master

Permette di far eseguire un blocco di istruzioni al thread Master (rank 0).

```
#pragma omp parallel num_threads(n){  
    ...  
    #pragma omp master  
    ...  
}
```

11.2.5 Direttiva single

Fa eseguire un blocco di istruzioni ad uno solo tra i thread paralleli (la scelta non è deterministica)

```
#pragma omp single
```

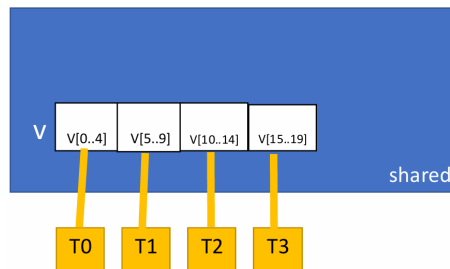
realizza una **barriera di sincronizzazione implicita**: ogni thread, prima di eseguire l'istruzione successiva al blocco single, attende tutti i thread (compreso il single) abbiano terminato le istruzioni precedenti.

11.2.6 Parallelizzazione di cicli

Distribuzione del carico tramite for:

```
1 #pragma omp parallel for [altre clausole...]  
2 for(i=0; i<K; i++)  
3     v[i] = i*i;
```

se n è il num di thread paralleli, ogni thread esegue un insieme (chunk) di K/n iterazioni contigue.



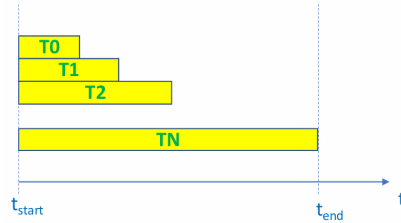
a ogni thread è assegnato un **chunk** di K/n interazioni contigue.

Il carico di iterazioni può essere significativamente differente

```

1 sum = 0.0;
2 for (i = 1; i <= n; i++)
3     for (j=0; j<i; j++)
4         sum += f(i);

```



11.2.7 Clausola schedule

È possibile realizzare distribuzioni diverse dalla block distribution:

```
#pragma omp parallel for schedule(<tipo>, <chunksize>)
```

dove **<tipo>** indica il tipo di distribuzione (static, dynamic); **<chunksize>** indica l'unità di carico (espressa in numero di iterazioni).

il tipo di **scheduling static** prevede che la distribuzione del lavoro venga fatta assegnando i diversi chunks ai thread **<staticamente>** secondo una politica round-robin.

```

1 #pragma omp parallel for num_thread(4) schedule (static, 1)
2 for(i=0; i<20; i++)
3     v[i] = i*i;

```

Lo scheduler assegnerà (chunk di dim=1) le iterazioni

- a th0=0, 4, 8, 12, 16;
- a th1=1, 5, 9, 13, 17;
- a th2=2, 6, 10, 14, 18;
- a th3=3, 7, 11, 15, 19.

il tipo di **scheduling dynamic** quando un thread termina l'elaborazione di un chunk, lo scheduler gli assegna il prossimo chunk (seguendo l'ordine degli indici).

```

1 #pragma omp parallel for num_thread(4) schedule (dynamic, 2)
2 for(i=0; i<20; i++)
3     v[i] = i*i;

```

Assegna le prime 8 iterazioni ai 4 thread; al primo thread che termina il suo primo chunk verrà assegnato il chunk 8,9 e al successivo verrà assegnato il chunk 10,11 e così via.

In questo modo è possibile mantenere il carico bilanciato, anche in presenza di non determinismo nel numero di operazioni da eseguire ad ogni iterazione

11.3 Sincronizzazione tra thread in OMP

oltre alla clausola `critical` per mutua esclusione, OMP offre altri strumenti per realizzare in modo esplicito la sincronizzazione tra thread paralleli:

- direttiva `barrier`: barriera di sincronizzazione in un team di threads.
- gestione di lock: tramite funzioni `omp-set-lock` e `omp-unset-lock` è possibile realizzare schemi di sincronizzazione ad hoc tra threads.

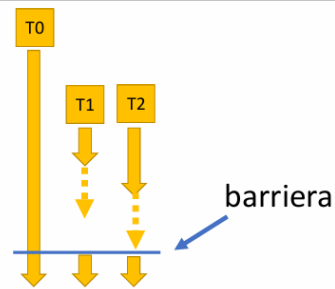
11.3.1 Direttiva barrier

ha senso solo se mi trovo all'interno di un `omp parallel`

```
#pragma omp barrier
```

Esempio:

```
int main( int argc, char *argv[] ) {  
    int v[MAX];  
    a=8;  
    #pragma omp parallel num_threads(3)  
    {  
        int iam = omp_get_thread_num();  
        v[iam] = iam*iam;  
        #pragma omp master  
        a += 800; //ran only by one thread  
        #pragma omp barrier  
        printf("thread %d: valore di a: %d\n", iam,a);  
    }  
}
```



11.3.2 Sincronizzazione lock

È implementato da una variabile del tipo `omp_lock_t` condivisa tra i thread che devono sincronizzarsi, alla quale è possibile applicare le funzioni:

- `omp_set_lock(..)`
- `omp_unset_lock(..)`
- `omp_init_lock(..)`, `omp_destroy_lock(..)`

Un lock è un oggetto che ha 2 stati: libero e occupato. Per inizializzare ogni lock va preventivamente creato e inizializzato (a **libero**) con la funzione:

```
void omp_init_lock(omp_lock_t *L);
```

quando il lock non viene più usato, va eliminato con la funzione:

```
void omp_destroy_lock(omp_lock_t *lock);
```

```

1 // il lock viene utilizzato per realizzare sezioni critiche
2 #include <stdio.h>
3 #include <omp.h>
4 omp_lock_t my_lock; //
5 int main() {
6     omp_init_lock(&my_lock); //
7     #pragma omp parallel num_threads(4)
8     { int i, j, t = omp_get_thread_num( );
9       for (i = 0; i < 5; ++i) {
10         omp_set_lock(&my_lock); //prologo sezione critica
11         printf("Thread %d - inizio sezione critica %d\n", t,
12               ↵, i);
13         printf("Thread %d - fine sezione critica %d\n", t,
14               ↵, i);
15         omp_unset_lock(&my_lock); //epilogo sezione critica
16     } }
17 omp_destroy_lock(&my_lock); } //

```

```

1 // il lock viene utilizzato per imporre un vincolo di
2   ↵precedenza tra il thread 3 ed il thread 2
3 #include <stdio.h>
4 #include <omp.h>
5 omp_lock_t my_lock; //
6 int main() {
7     omp_init_lock(&my_lock); //
8     omp_set_lock(&my_lock); //il master occupa il lock
9     #pragma omp parallel num_threads(4)
10    { int t = omp_get_thread_num( );
11      if (t==3)
12      { printf("Thread %d: prima..\n", t);
13        omp_unset_lock(&my_lock); //
14      }
15      else if (t==2)
16      { omp_set_lock(&my_lock); //
17        printf("Thread %d: dopo..\n", t);
18      }
19      else printf("Thread %d: sono libero da vincoli..", t);
20    } // fine blocco parallelo
21    omp_destroy_lock(&my_lock); } //

```

11.4 Confronto OpenMP vs pthreads

- **Pthreads:** paradigma MPMD ovvero il programmatore definisce la (potenzialmente diversa) sequenza di istruzioni che ogni singolo processo parallelo eseguirà (v. pthread_create), creazione dei processi: modello fork-join, strumenti avanzati di sincronizzazione (mutex, semafori, condition).

- **OpenMP**: approccio SPMD, schema cobegin-coend, Sincronizzazione tramite direttive/clausole che implementano schemi predefiniti (barrier, critical, reduction..). Politiche di sincronizzazione ad hoc possono essere realizzate combinando lock e schemi predefiniti.

11.5 Confronto MPI vs OpenMP

MPI	OpenMP
Message passing	Memoria condivisa
Elevata scalabilità	Scalabilità limitata al numero di CPU
Maggiore complessità	Semplicità d'uso
Portabilità elevata	Solo sistemi shared memory

In openMP la scalabilità è limitata dal numero di cpu disponibili per singolo nodo e non posso eseguire un applicazione openMP su più nodi perchè si deve garantire che i dati si trovino sulla stessa memoria fisica.

11.6 Hybridization

È possibile combinare MPI e OpenMP per beneficiare dei vantaggi di entrambi, specialmente in presenza di carichi di comunicazione elevati.

Hybridization in pratica

```
int main( ... ) {
    MPI_Init_thread(&argc, &argv, requested, &provided);

    #pragma omp parallel
    {
        ...
    }
}
```

livello di
supporto ai
thread
(richiesto e
ottenuto)

al posto di
MPI_Init()

Livello di supporto:

- **MPI_THREAD_SINGLE**: only one thread can run during MPI calls => all MPI calls must be outside the OpenMP pragma sections
- **MPI_THREAD_FUNNELED**: MPI calls can be inside pragmas but they are performed only by the master thread
- **MPI_THREAD_SERIALIZED**: MPI calls by multiple threads are allowed but only one at a time
- **MPI_THREAD_MULTIPLE**: MPI calls can be performed by multiple threads with no restrictions
- ...